Deep Learning — Complete Notes

All topics combined in one searchable file. Use Ctrl+F to search.

Table of Contents

8. Traditional NLP & Sequence Models

N-grams, Katz Backoff, perplexity & empirical entropy, Seq2Seq + Attention, Word2Vec, GloVe

SVG Formulae 14 Q&A

9. Transformer & Modern LLMs

Self-attention, multi-head attention, positional encoding, Beam Search, RAG, model compression

SVG Formulae 12 Q&A

10. Prompting & Instruction Tuning

Zero-shot, few-shot, Chain-of-Thought, prompt tuning, Pre-training vs Fine-tuning, PEFT (LoRA, Adapters)

SVG Formulae 12 Q&A

11. RLHF & Alignment

SFT, reward model, PPO optimization, DPO, and AI Agents (ReAct, Reflexion, HuggingGPT)

SVG Formulae 14 Q&A

PRACTICE PAPERS

Practice Paper 1

100 marks — MCQ (20) + Short Answer (30) + Long Answer (50). Full solutions with SVG diagrams.

100 Marks SVG Solutions

Practice Paper 2

100 marks — MCQ (20) + Short Answer (30) + Long Answer (50). RLHF proofs, attention computation, gradient derivations.

100 Marks SVG Solutions

↑ Back to Top | **Introduction & Motivation**

Topic 1 — Introduction, Motivation & History of DL/NLP

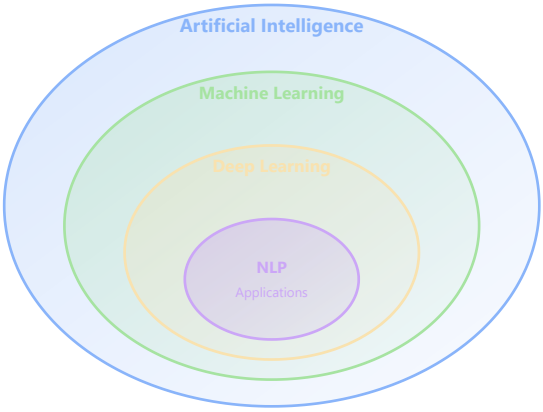
This topic covers the foundations: what AI, ML, DL, and NLP are, why NLP matters, its historical evolution, datasets, preprocessing pipelines, feature extraction, evaluation metrics, and the Python/PyTorch toolkit for deep learning.

1. AI vs ML vs DL vs NLP

Definitions

Field	Definition	Example
AI	Machines performing tasks requiring human intelligence	Self-driving cars, chatbots
ML	Systems learn patterns from data (subset of AI)	Spam detection from labeled emails
DL	Neural networks with multiple layers (subset of ML)	Image recognition, speech synthesis
NLP	Machines understand/generate human language	Translation, sentiment analysis

Hierarchy Diagram



2. Motivation Behind NLP

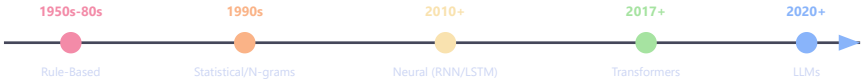
Why NLP Exists

Humans communicate via language, grammar, context, and emotions. Machines understand only numbers. NLP bridges: **human language ↔ machine understanding**.

Key Drivers

Driver	Explanation
Massive Text Data	Emails, tweets, reviews, documents — too much for humans to process
Human-Machine Interaction	Natural language commands: "Book a flight tomorrow"
Automation	Customer support, content moderation, translation at scale

3. History & Evolution of NLP



Era	Approach	Limitation
Rule-Based (1950s–80s)	Hand-written grammar rules	Couldn't scale; failed on ambiguity
Statistical (1990s)	Probabilities, N-grams	Couldn't capture long-range dependencies
Neural (2010+)	RNN, LSTM, GRU	Sequential processing, vanishing gradients
Transformer (2017+)	"Attention Is All You Need"	Compute-intensive for long sequences
LLM (2020+)	GPT, BERT, T5 at scale	Alignment, hallucination, cost

4. Datasets in NLP

Train / Validation / Test Split

Popular NLP Datasets

Dataset	Task	Size
IMDB	Sentiment Analysis	50K reviews
SQuAD	Question Answering	100K+ questions
WMT	Machine Translation	Millions of pairs
GLUE	Multi-task Benchmark	9 tasks
Common Crawl	LLM Pre-training	Petabytes

5. Core NLP Tasks

Task	Input → Output	Example
Sentiment Analysis	Text → Emotion label	"Great movie!" → Positive
Machine Translation	Language A → Language B	"Hello" → "Bonjour"
NER	Text → Entities	"Elon Musk founded SpaceX" → [Person, Org]
Summarization	Long text → Short summary	Article → 2 sentences
Question Answering	Context + Question → Answer	"Capital of India?" → "New Delhi"

6. Challenges in NLP

Challenge	Example	Why It's Hard
Ambiguity	"bank" = river bank or financial bank?	Same word, multiple meanings
Sarcasm	"Great, my laptop crashed again"	Positive word, negative intent
Context/Coreference	"He put trophy in suitcase because *it* was big"	What does "it" refer to?
Data Sparsity	Rare words appear few times	Insufficient statistics
Multilinguality	Different grammars, scripts	One model doesn't fit all

7. NLP Pipeline

8. Tokenization

Breaking text into smaller units (tokens).

Type	Input	Output
Word-level	"I love AI"	["I", "love", "AI"]
Character-level	"hello"	["h","e","l","l","o"]
Subword (BPE)	"playing"	["play", "ing"]

9. Stemming vs Lemmatization

Feature	Stemming	Lemmatization
Method	Cuts suffixes heuristically	Uses dictionary/morphology
Speed	Fast	Slower
Accuracy	Lower ("studies"→"studi")	Higher ("studies"→"study")
Use Case	Search engines	Text understanding tasks

10. Feature Extraction — BoW & TF-IDF

Bag of Words (BoW)

Represents text as word frequency vectors. Order is ignored.

```
# Example
sentences = ["I love AI", "I love Python"]
# Vocabulary: [I, love, AI, Python]
# Vectors:   [1,1,1,0] and [1,1,0,1]
```

TF-IDF

Term Frequency

$$TF(t,d) = \frac{\text{count of } t \text{ in document } d}{\text{total words in } d}$$

t = target word | d = specific document

Inverse Document Frequency

$$IDF(t) = \log\left(\frac{N}{df(t)}\right)$$

N = total number of documents | $df(t)$ = number of documents containing word t

TF-IDF

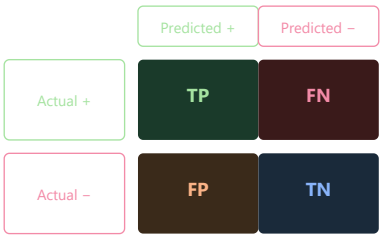
$$\text{TF-IDF}(t,d) = TF(t,d) \times IDF(t)$$

High TF-IDF → word is important in this document but rare across corpus

TF-IDF automatically down-weights common words like "the" (IDF→0) and up-weights discriminative words.

11. Evaluation Metrics

Confusion Matrix



Core Metrics

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

$$F_1 = \frac{2 \cdot P \cdot R}{P + R}$$

TP = True Positive | FP = False Positive | FN = False Negative | TN = True Negative

Accuracy is misleading for imbalanced datasets. A model predicting all samples as the majority class gets high accuracy but zero usefulness. Use Precision/Recall/F1 instead.

12. Python & PyTorch for Deep Learning

Key Libraries

Library	Purpose
NumPy	Fast numerical arrays
Pandas	Data manipulation
Matplotlib	Visualization
PyTorch	Deep Learning (dynamic graphs, GPU)

Tensors

Dimension	Name	Example
0-D	Scalar	5
1-D	Vector	$[1, 2, 3]$
2-D	Matrix	$[[1, 2], [3, 4]]$
n-D	Tensor	Batch of images: (B, C, H, W)

13. Gradients & Training Loop

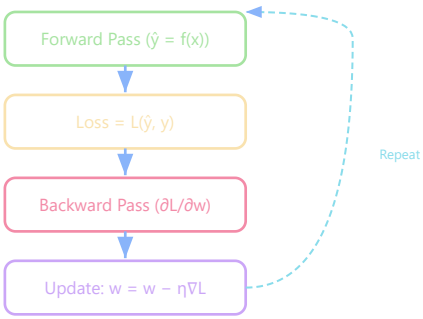
> ⏩ **Preview:** This section gives a high-level look at how neural networks learn. The full math of backpropagation, loss functions, and optimizers is covered in **Topic 3 (FNN)**.

Weight Update Rule (Gradient Descent)

$$w_{new} = w_{old} - \eta \frac{\partial L}{\partial w}$$

w = weight parameter | η = learning rate (step size) | L = loss function | $\frac{\partial L}{\partial w}$ = gradient of loss w.r.t. weight

Training Loop Flowchart



PyTorch Training Example

```
import torch
import torch.nn as nn
```

```
import torch.optim as optim

model = nn.Linear(1, 1)
criterion = nn.MSELoss()
optimizer = optim.SGD(model.parameters(), lr=0.01)

x = torch.tensor([[1.0]])
y = torch.tensor([[2.0]])

pred = model(x)          # Forward
loss = criterion(pred, y) # Loss
loss.backward()              # Gradients
optimizer.step()             # Update weights
```

PyTorch Autograd Example

```
x = torch.tensor(2.0, requires_grad=True)
y = x**2           # y = x²
y.backward()      # dy/dx = 2x
print(x.grad)      # tensor(4.0) → 2×2 = 4
```

Metric Numericals

100 test samples: TP=40, TN=50, FP=5, FN=5

Accuracy = $(40 + 50)/100 = \mathbf{0.90}$

Precision = $40/(40 + 5) = 40/45 = \mathbf{0.889}$

Recall = $40/(40 + 5) = 40/45 = \mathbf{0.889}$

F1 = $2 \times (0.889 \times 0.889)/(0.889 + 0.889) = \mathbf{0.889}$

GPU Training

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)
tensor = tensor.to(device)
```

14. Neuron & Activation Functions

Single Neuron

$$y = f(w \cdot x + b)$$

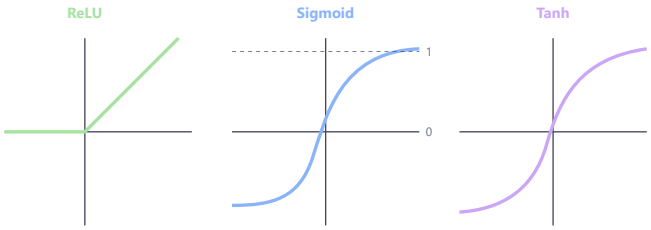
w = weight vector | x = input vector | b = bias | f = activation function

ReLU

$$\mathbf{ReLU}(x) = \mathbf{max}(0, x)$$

ReLU = Rectified Linear Unit — the most popular activation function ("switch") in modern networks. Full comparison of all activation variants in **Topic 2a**.

Activation Functions Comparison



Activation	Range	Best For
ReLU	$[0, \infty)$	Hidden layers (default choice)
Sigmoid	$(0, 1)$	Binary classification output
Tanh	$(-1, 1)$	Centered activation (RNNs)

15. Best Practices & Common Mistakes

Best Practices: Clean data properly • Use validation set • Normalize inputs • Start simple, then add complexity • Monitor overfitting (train vs val loss)

Common Mistakes: Removing stopwords like "not" (changes sentiment) • Using accuracy alone for imbalanced data • Forgetting model.train()/model.eval() modes • Learning rate too high (divergence)

16. Quick Revision

Topic	One-Line Summary
NLP	Machines understand human language
Dataset	Collection of labeled training examples
Tokenization	Split text into tokens
BoW	Word frequency vector (ignores order)
TF-IDF	Word importance = frequency × rarity
PyTorch	Dynamic-graph DL framework by Meta
Tensor	N-dimensional array (GPU-accelerated)
Gradient	Direction & magnitude of steepest loss increase
ReLU	$\max(0, x)$ — default activation

Advanced Questions & Answers

Q1. A binary classifier on a dataset with 950 positives and 50 negatives achieves 95% accuracy. Compute precision, recall, and F1 if it predicts ALL samples as positive.

Given: TP = 950, FP = 50, FN = 0, TN = 0

$$\text{Precision} = \frac{950}{950 + 50} = 0.95$$

$$\text{Recall} = \frac{950}{950 + 0} = 1.0$$

$$F_1 = \frac{2 \times 0.95 \times 1.0}{0.95 + 1.0} = \frac{1.9}{1.95} \approx 0.974$$

Despite seemingly high metrics, this model is **useless** — it never identifies negatives. Specificity = $TN/(TN + FP) = 0/50 = 0$. This demonstrates why accuracy alone is misleading for imbalanced data.

Q2. Given 3 documents, compute TF-IDF of "neural" in D1.
D1: "neural networks use neural computation" (5 words)
D2: "deep learning is powerful" (4 words)
D3: "neural networks learn features" (4 words)

$$TF(\text{"neural"}, D1) = \frac{2}{5} = 0.4$$

$$IDF(\text{"neural"}) = \ln\left(\frac{3}{2}\right) \approx 0.405$$

$$\text{TF-IDF} = 0.4 \times 0.405 = \mathbf{0.162}$$

$N = 3$ documents total, $df = 2$ (word appears in D1 and D3)

Q3. Explain why the gradient formula $w_{new} = w_{old} - \eta \frac{\partial L}{\partial w}$ uses *subtraction*. What happens with addition?

The gradient ∇L points in the direction of steepest **increase** of loss. To **minimize** loss, we move in the opposite direction — hence subtraction.

With addition: we'd move toward higher loss → model diverges, performance degrades every step.

Q4. Training loss decreases every epoch but validation loss increases after epoch 5. Diagnose and propose 3 solutions with mathematical justification.

Diagnosis: Overfitting — model memorizes training data but fails to generalize.

Solution 1 — L2 Regularization (penalizing large weights to keep the model simple — detailed in Topic 3)**:** Add $\lambda \sum w_i^2$ to loss. Gradient becomes $\frac{\partial L}{\partial w} + 2\lambda w$, causing weight decay and constraining complexity.

Solution 2 — Dropout (rate p) (randomly disabling neurons during training so the network can't rely on any single path — detailed in Topic 5)**:** Zero out neurons with probability p during training. Trains an ensemble of 2^n sub-networks, preventing co-adaptation.

Solution 3 — Early Stopping: Stop at epoch 5 (minimum validation loss). Limits effective model complexity by restricting optimization time.

Q5. Compare BoW and TF-IDF. Why does TF-IDF outperform BoW for information retrieval? Provide a mathematical argument.

BoW assigns equal weight to all words. If "the" appears 100 times, it dominates the representation despite zero semantic value.

TF-IDF down-weights via $IDF = \log(N/df)$. A word in ALL documents gets $IDF = \log(1) = 0$, eliminating it entirely.

Information-theoretic argument: TF-IDF weights terms proportional to their information content (Shannon's theory: rare events carry more information). This maximizes signal-to-noise ratio in document representations.

Created by **Kushal Ghosh** (gkushalg01@gmail.com) and **Bharat Das Vaishnav** (bharatvaishnav2025@gmail.com)

↑ Back to Top | **Single Neuron Computation**

Topic 2 — Single Neuron Computation, Gradient Descent & Backpropagation

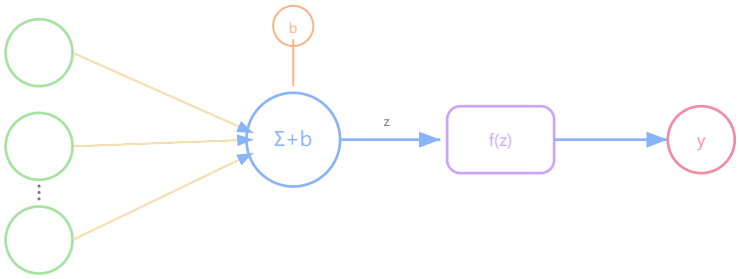
This topic covers: biological vs artificial neurons, weighted sums, activation functions, perceptrons, the XOR problem, Universal Approximation Theorem, loss functions, gradient descent, backpropagation, and optimization algorithms.

1. Artificial Neuron Structure

Biological vs Artificial

Biological Neuron	Artificial Neuron
Dendrites receive signals	Inputs receive data
Cell body processes	Weighted sum computation
Axon sends output	Activation output
Synapses determine strength	Weights determine importance

Neuron Diagram



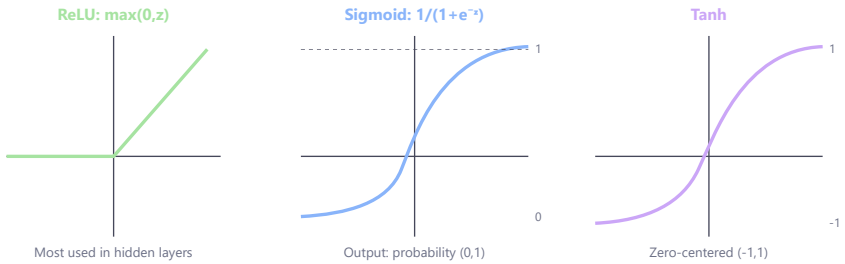
> **Core Neuron Formula**
>
> $$z = \sum_{i=1}^{n} w_i x_i + b \qquad y = f(z)$$
>
> x_i = input features | w_i = weight for input i (importance) | b = bias (shifts decision boundary) | f = activation function | y = output

Role of Each Component

Component	Purpose	Analogy

Weights	Determine input importance	Volume knobs
Bias	Shift activation threshold	Default tendency
Activation	Introduce non-linearity	On/off decision

2. Activation Functions



Sigmoid

$$\sigma(z) = \frac{1}{1+e^{-z}}$$

Output ∈ $(0, 1)$ — interprets as probability. Derivative: $\sigma'(z) = \sigma(z)(1-\sigma(z))$, max = 0.25

Tanh

$$\tanh(z) = \frac{e^{z}-e^{-z}}{e^{z}+e^{-z}}$$

Output ∈ $(-1, 1)$ — zero-centered. Relation: $\tanh(z) = 2\sigma(2z) - 1$

ReLU (Rectified Linear Unit)

$$\text{ReLU}(z) = \max(0, z)$$

Output ∈ $[0, \infty)$ — default choice for hidden layers. Gradient: 1 for $z > 0$, 0 for $z < 0$

Dying ReLU: If $z < 0$ always, gradient = 0 permanently → neuron is "dead." See Topic 2a for Leaky ReLU solution.

3. Perceptron & XOR Problem

Perceptron

The earliest artificial neuron (Frank Rosenblatt, 1958). Uses a **step function** as activation → binary output.

Step Function

$$f(z) = \begin{cases} 1 & z \geq 0 \\ 0 & z < 0 \end{cases}$$

Creates a linear decision boundary: $w_1 x_1 + w_2 x_2 + b = 0$

Logic Gates with Perceptrons

Gate	Weights	Bias	Linearly Separable?
AND	$w_1 = 1, w_2 = 1$	$b = -1.5$	Yes ✓
OR	$w_1 = 1, w_2 = 1$	$b = -0.5$	Yes ✓
XOR	—	—	No ✗

AND Gate Step-by-Step ($w_1 = 1, w_2 = 1, b = -1.5$)

x_1	x_2	$z = w_1x_1 + w_2x_2 + b$	$f(z)$ (step)	Expected
0	0	$0 + 0 - 1.5 = -1.5$	0	0 ✓
0	1	$0 + 1 - 1.5 = -0.5$	0	0 ✓
1	0	$1 + 0 - 1.5 = -0.5$	0	0 ✓
1	1	$1 + 1 - 1.5 = 0.5$	1	1 ✓

Decision boundary: $x_1 + x_2 - 1.5 = 0 \Rightarrow x_2 = -x_1 + 1.5$ (a line that separates (1,1) from rest)

Simple Backprop Example ($L = w^2, w = 3$)

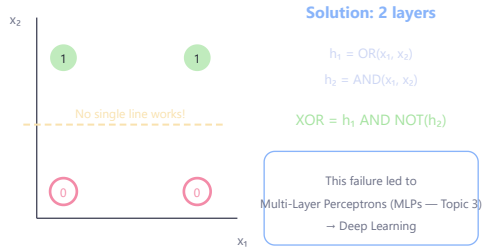
$L = w^2 = 9$

$\frac{\partial L}{\partial w} = 2w = 2(3) = 6$

With $\eta = 0.1$: $w_{new} = w - \eta \cdot \nabla = 3 - 0.1(6) = 2.4$

New loss: $L = 2.4^2 = 5.76$ (reduced from 9 → converging to 0)

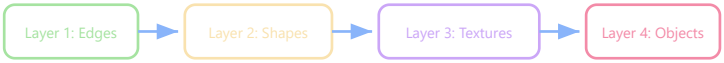
XOR Problem — Why Single Perceptron Fails



4. Deep vs Shallow Networks

Aspect	Shallow (1 hidden layer)	Deep (many layers)
Feature learning	Limited, flat	Hierarchical (edges→shapes→objects)
Parameter efficiency	May need exponential neurons	Efficient reuse of features
Training difficulty	Easier	Harder (vanishing gradients)
Data requirement	Less	More

Feature Hierarchy in Deep Networks



5. Universal Approximation Theorem

Statement: A feedforward network with at least one hidden layer, enough neurons, and a non-linear activation can approximate any continuous function on a compact subset of $\mathbb{R}^n$ to arbitrary accuracy.

Mathematical Form

$$f(x) \approx \sum_{i=1}^{N} a_i \cdot \sigma(w_i \cdot x + b_i)$$

N = number of hidden neurons | a_i = output weights | σ = squashing activation | w_i, b_i = hidden layer parameters

UAT does NOT guarantee: (1) Training will find the right weights, (2) Few neurons suffice, (3) Generalization to unseen data. Deep networks (multiple layers stacked — detailed in Topic 3) are preferred because they achieve the same power with exponentially fewer parameters.

6. Loss Functions

Loss measures *how wrong* the prediction is. Smaller loss = better model.

Mean Squared Error (Regression)

$$MSE = \frac{1}{N} \sum_{i=1}^{N} (y_i - \hat{y}_i)^2$$

y_i = actual value | $\hat{y}_i$ = predicted value | N = sample count. Squaring penalizes large errors more.

Binary Cross-Entropy (Binary Classification)

$$L = -\left[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})\right]$$

$y \in \{0, 1\}$ = true label | $\hat{y} \in (0, 1)$ = predicted probability. Strongly penalizes confident wrong predictions.

Problem Type	Loss Function	Example
Regression	MSE	House price prediction
Binary Classification	Binary Cross-Entropy	Spam detection
Multi-class	Categorical Cross-Entropy	Digit recognition (0-9)

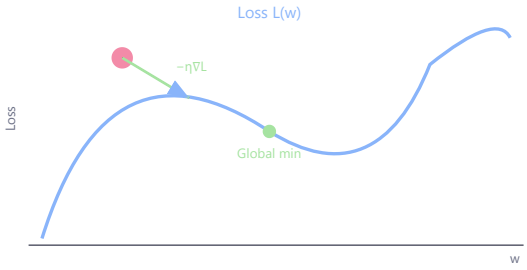
7. Gradient Descent

Weight Update Rule

$$w_{new} = w_{old} - \eta \frac{\partial L}{\partial w}$$

w = weight | η = learning rate (step size, typically 0.001–0.1) | $\frac{\partial L}{\partial w}$ = gradient (direction of steepest loss increase) | Subtract = move opposite to increase = toward minimum

Gradient Descent Visualization



Learning Rate Effects

Learning Rate	Effect
Too small (0.0001)	Very slow convergence, may get stuck
Too large (1.0)	Overshoots minimum, diverges
Just right (0.001–0.01)	Stable, efficient convergence

Types of Gradient Descent

Type	Batch Size	Updates/Epoch	Trait
Batch GD	All data	1	Smooth but slow & memory-heavy
Stochastic GD	1 sample	N	Noisy but fast, good regularization
Mini-batch GD	32–256	N/batch	Best practical balance (standard)

8. Backpropagation

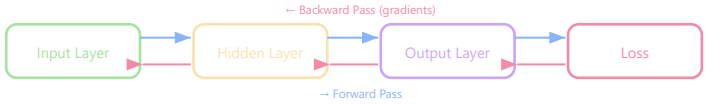
Backpropagation computes *how much each weight contributed to the error* using the chain rule of calculus.

Chain Rule

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial w}$$

Error flows backward: output → hidden → input. Each layer multiplies its local gradient.

Backprop Flow Diagram



9. Optimization Algorithms

Optimizer	Key Idea	Best For
SGD	Basic gradient descent	Simplicity, research
SGD + Momentum	Adds velocity memory $v_t = \beta v_{t-1} + (1-\beta)\nabla L$	Faster convergence, less oscillation
AdaGrad	Adapts lr per parameter (÷ sum of squared grads)	Sparse data (NLP embeddings)
RMSProp	Fixes AdaGrad shrinking (exponential decay)	RNN training
Adam	Momentum + RMSProp combined	Default choice for most tasks

> **Rule of thumb:** Use Adam for new projects. Switch to SGD+Momentum for fine-tuning when you need precise control.

10. Gradient Problems & Solutions

Problem	Symptom	Solution
Vanishing Gradients	Early layers don't learn (gradients → 0)	ReLU, BatchNorm, ResNet skip connections
Exploding Gradients	Weights blow up (gradients → ∞)	Gradient clipping, proper initialization

11. Worked Numericals

Weighted Sum + Sigmoid

Given: $x = [1, 2]$, $w = [0.2, 0.8]$, $b = -1$

$z = 0.2(1) + 0.8(2) + (-1) = 0.2 + 1.6 - 1 = 0.8$

$\sigma(0.8) = \frac{1}{1+e^{-0.8}} = \frac{1}{1.449} \approx \mathbf{0.69}$

MSE Calculation

Given: Actual $y = [2, 4, 6]$, Predicted $\hat{y} = [3, 5, 4]$

$MSE = \frac{(2-3)^2+(4-5)^2+(6-4)^2}{3} = \frac{1+1+4}{3} = \mathbf{2.0}$

Gradient Descent Step

Given: $w = 5$, $\frac{\partial L}{\partial w} = 2$, $\eta = 0.1$

$w_{new} = 5 - 0.1(2) = 5 - 0.2 = \mathbf{4.8}$

Backprop with Chain Rule

Given: $y = wx$, $L = (y-2)^2$, $x = 3$, $w = 1$

Forward: $y = 1 \times 3 = 3$, $L = (3-2)^2 = 1$

$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial w} = 2(y-2) \cdot x = 2(1) \cdot 3 = \mathbf{6}$

Advanced Questions & Answers

AQ1. A single neuron has $w = [0.5, -0.3, 0.8]$, $b = -0.2$, input $x = [1, 2, 3]$. Compute sigmoid output and $\frac{\partial \sigma}{\partial w_1}$.

$z = 0.5(1) + (-0.3)(2) + 0.8(3) + (-0.2) = 0.5 - 0.6 + 2.4 - 0.2 = 2.1$

$\sigma(2.1) = \frac{1}{1+e^{-2.1}} = \frac{1}{1.1225} \approx 0.891$

$\frac{\partial \sigma}{\partial w_1} = \sigma(z)(1 - \sigma(z)) \cdot x_1 = 0.891 \times 0.109 \times 1 \approx \mathbf{0.0971}$

AQ2. Prove a single perceptron cannot solve XOR. Show minimum architecture that can.

Proof: From XOR constraints: $b \leq 0$ (from 0,0→0), $w_2 + b > 0$ (from 0,1→1), $w_1 + b > 0$ (from 1,0→1), $w_1 + w_2 + b \leq 0$ (from 1,1→0).

Adding constraints 2&3: $w_1 + w_2 + 2b > 0$. Constraint 4: $w_1 + w_2 \leq -b$. Combined: $-2b < w_1 + w_2 \leq -b \Rightarrow -2b < -b \Rightarrow b > 0$. Contradicts $b \leq 0$. ■

Solution: 2-layer network: $h_1 = \text{OR}(x_1, x_2)$, $h_2 = \text{AND}(x_1, x_2)$, output = h_1 AND NOT(h_2) = XOR.

AQ3. Network: 2 inputs → 3 hidden (sigmoid) → 1 output. Count parameters and FLOPs.

Parameters: Input→Hidden: $2 \times 3 + 3 = 9$. Hidden→Output: $3 \times 1 + 1 = 4$. **Total: 13**

FLOPs: Hidden: 3×(2 mults + 1 add + bias + sigmoid) ≈ 24 ops. Output: 3 mults + 2 adds + bias + sigmoid ≈ 9 ops. **Total ≈ 33 FLOPs**

AQ4. $L = (w - 3)^2$, $w_0 = 2.0$, $\eta = 0.1$. Perform 5 iterations.

$\frac{\partial L}{\partial w} = 2(w - 3)$

Iter	w	L	∇	w_{new}
1	2.000	1.000	−2.000	2.200
2	2.200	0.640	−1.600	2.360
3	2.360	0.410	−1.280	2.488
4	2.488	0.262	−1.024	2.590
5	2.590	0.168	−0.819	2.672

Weight error reduces by factor $(1 - 2\eta) = 0.8$ each step. Loss reduces by $0.8^2 = 0.64$ per step.

AQ5. Explain UAT and its practical limitations.

Statement: A single hidden layer with finite neurons and squashing activation can approximate any continuous function on compact $\mathbb{R}^n$.

Limitations: (1) Required width may be exponentially large. (2) No guarantee gradient descent finds the right weights. (3) Overfitting — memorizes, doesn't generalize. (4) Deep networks achieve same power with exponentially fewer parameters. (5) Wide single-layer networks have harder optimization landscapes.

Basic Q&A

Q1. What is the output of a perceptron with $w = [1, 1]$, $b = -1.5$ for input $[1, 0]$?

$z = 1(1) + 1(0) - 1.5 = -0.5 < 0 \Rightarrow$ output = **0**

Q2. Why can't a single perceptron solve XOR?

XOR is not linearly separable — no single line can divide (0,0),(1,1) from (0,1),(1,0). Need at least 2 layers.

Q3. What is the derivative of $\tanh(x)$?

$\frac{d}{dx}\tanh(x) = 1 - \tanh^2(x)$. At $x = 0$: derivative = 1. Saturates to 0 for large $|x|$.

Q4. Compute ReLU and its derivative for $x = -3$ and $x = 5$.

$\text{ReLU}(-3) = 0$, derivative = 0 | $\text{ReLU}(5) = 5$, derivative = 1

Q5. What happens if learning rate is too high vs too low?

Too high: Overshoots minimum, loss oscillates/diverges. **Too low:** Extremely slow convergence, may get stuck in local minima.

Quick Revision Sheet

Concept	Key Point
Neuron	$y = f(\sum w_i x_i + b)$
Perceptron	Step activation → binary classifier, linear boundary only
XOR Problem	Not linearly separable → need hidden layers
Gradient Descent	$w \leftarrow w - \eta \nabla L$ — walk downhill
Backprop	Chain rule backwards through layers to get gradients
UAT	1 hidden layer can approximate any function (but may need ∞ neurons)
Sigmoid	$\sigma(x) = \frac{1}{1+e^{-x}}$ → output [0,1], vanishing gradient
ReLU	$\max(0, x)$ → no vanishing gradient, but dying neurons
Learning Rate	Too high = diverge, too low = slow, use schedulers

---

Created by **Kushal Ghosh** (gkushalg01@gmail.com) and **Bharat Das Vaishnav** (bharatvaishnav2025@gmail.com)

Topic 2a — Dying ReLU Problem & Leaky ReLU

This topic covers the Dying ReLU problem — why neurons die, how to detect it, and the Leaky ReLU solution.

1. ReLU Recap

ReLU Activation

$$f(z) = \max(0, z)$$

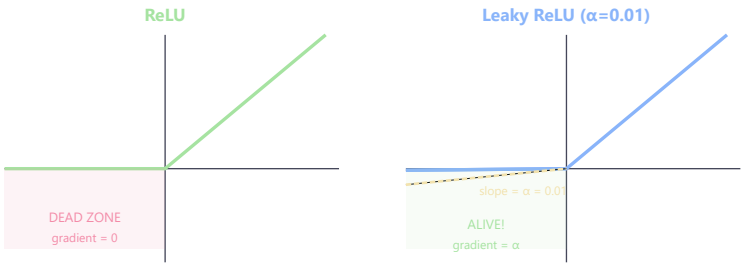
z = weighted sum (pre-activation) | Output: z if positive, 0 if negative

ReLU Derivative

$$f'(z) = \begin{cases} 1 & z > 0 \\ 0 & z \leq 0 \end{cases}$$

Gradient is either full (1) or completely dead (0). No middle ground.

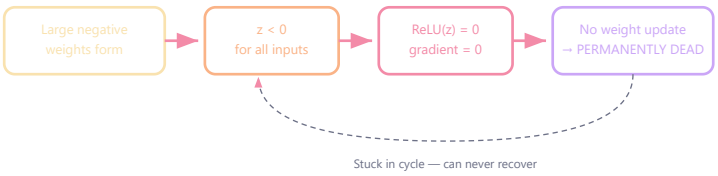
ReLU vs Leaky ReLU — Visual Comparison



2. The Dying ReLU Problem

Definition: A neuron becomes *permanently inactive* — always outputs 0 for every input. It can never recover because its gradient is always 0, so weights never update.

How It Happens — Step by Step



Causes

Cause	Mechanism
Large learning rate	Huge updates push weights strongly negative very quickly

Poor initialization	Weights start negative → neurons dead from the beginning
Large negative weights	$z = \sum w_i x_i + b < 0$ for all inputs in data distribution

Training Scenario — Dead Neuron

Iteration	z	ReLU Output	Gradient	Weight Update
1	−3	0	0	None
2	−5	0	0	None
3	−2	0	0	None
4	−7	0	0	None
Neuron is permanently dead 💀				

3. Leaky ReLU — The Solution

Leaky ReLU

$$f(z) = \begin{cases} z & z > 0 \\ \alpha z & z \leq 0 \end{cases}$$

α = small constant (typically 0.01) | For $z < 0$: output is small but non-zero | Gradient for $z < 0$ = α (non-zero!) → neuron can still learn

Example: Leaky ReLU at z = −5

ReLU: $f(-5) = \max(0, -5) = 0$ gradient = 0 → dead

Leaky ReLU (α=0.01): $f(-5) = 0.01 \times (-5) = -0.05$ gradient = 0.01 → alive!

ReLU vs Leaky ReLU Comparison

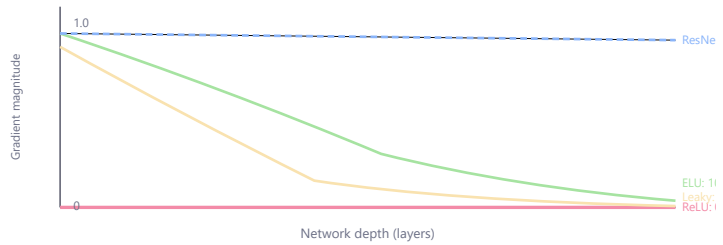
Feature	ReLU	Leaky ReLU
Negative output	Always 0	Small negative (αz)
Dead neuron risk	High	Much lower
Gradient for $z < 0$	0 (dead)	α (alive)
Computation	Simplest	Nearly identical cost

4. All Solutions to Dying ReLU

Method	How It Helps
Leaky ReLU	Non-zero gradient for negative inputs
He Initialization	$W \sim \mathcal{N}(0, \sqrt{2/n_{in}})$ — keeps activations balanced
Smaller learning rate	Prevents extreme weight updates
Batch Normalization	Stabilizes activations, prevents extreme negatives

ELU / GELU	Smooth negative region with non-zero gradients
Parametric ReLU (PReLU)	Learns optimal α per neuron during training

5. Gradient Flow Comparison (50-Layer Network)



> **Key insight:** For very deep networks (50+ layers), even Leaky ReLU cannot prevent vanishing gradients alone. Residual/skip connections (ResNet) are essential — they provide a gradient highway that maintains magnitude ≈ 1 regardless of depth.

6. Best Practices

- Use **He initialization** with ReLU networks: $W \sim \mathcal{N}(0, \sqrt{2/n_{in}})$
- Start with learning rate 0.001 (not 0.1)
- Monitor neuron activations — if many are always 0, switch to Leaky ReLU
- For very deep networks, combine Leaky ReLU with BatchNorm (normalizes layer outputs — Topic 5) + skip connections (shortcuts that bypass layers — Topic 5)
- PReLU (learned α) is best when you can afford the extra parameters

Advanced Questions & Answers

Q1. 1000 neurons with ReLU. After training (η=0.1), 300 output zero for ALL samples. Calculate dead neuron % and capacity loss.

Dead neurons = 300/1000 = **30%**. Effective capacity loss = 30%.

If original capacity = C features, now only ≈ $0.7C$ features can be learned. The model's expressive power is significantly degraded.

Q2. Neuron with $w = [-3, -2]$, $b = -1$, inputs $x_i \in [0, 1]$. Prove it is permanently dead under ReLU.

Maximum possible z:

$$z_{max} = (-3)(1) + (-2)(1) + (-1) = -6$$

Since maximum $z = -6 < 0$, ReLU(z) = 0 for ALL inputs. Gradient = 0 always → weights never update → **provably dead.**

Q3. 50-layer network, all $z = -2$. Compute gradient reaching layer 1 for ReLU, Leaky ReLU (α=0.01), and ELU.

ReLU: $0^{50} = 0$ — total death

Leaky ReLU: $0.01^{50} = 10^{-100}$ — technically non-zero but useless

ELU: $f'(-2) = e^{-2} \approx 0.135$, so $0.135^{50} \approx 10^{-43}$ — better but still vanishing

Conclusion: Only skip connections (ResNet) maintain gradient ≈ 1 through arbitrary depth.

Q4. What is the optimal α for Leaky ReLU? Discuss the trade-off.

Trade-off: Larger α → better gradient flow but more linearity (less non-linear power).

At $\alpha = 1$: identity function (useless). At $\alpha = 0$: standard ReLU (dying risk).

Empirically: $\alpha \in [0.01, 0.3]$ works well. PReLU *learns* α per neuron during training — best of both worlds.

Q5. 4 hidden layers (100 neurons each), ReLU, lr=0.01. After 1000 steps, 40% dead. Propose recovery and explain why He initialization prevents this.

Recovery: "Neuron resurrection" — re-initialize dead neurons from He distribution while preserving alive neurons' trained weights.

He initialization: $W \sim \mathcal{N}(0, \sqrt{2/n_{in}})$. For $n_{in} = 100$: std = $\sqrt{2/100} = 0.141$

This ensures Var(z) ≈ Var(x) at each layer. With balanced variance, ~50% of pre-activations are positive (≈ symmetric around 0), so ~50% activate at init — far from dead.

Standard init $W \sim \mathcal{N}(0, 1)$: variance *explodes* through layers, pushing many z to extreme negatives → mass death.

Basic Q&A

BQ1. What is ReLU(−5)? What is its derivative?

ReLU(−5) = max(0, −5) = **0**. Derivative = **0** (negative region).

BQ2. What is Leaky ReLU(−5) with α=0.01?

$0.01 \times (-5) = -0.05$. Derivative = **01**.

BQ3. Why is ReLU preferred over sigmoid in hidden layers?

---

(1) No vanishing gradient for positive inputs (derivative = 1). (2) Computationally cheaper (just max). (3) Induces sparsity (many outputs = 0).

BQ4. What is the "dying ReLU" problem?

If a neuron's pre-activation is always negative (due to bad weights or high learning rate), ReLU outputs 0 forever. Gradient = 0 → neuron never updates → permanently dead.

BQ5. How does He initialization help ReLU networks?

$W \sim \mathcal{N}(0, \sqrt{2/n_{in}})$. Keeps variance stable across layers → ~50% neurons activate at initialization → prevents mass death.

Final Intuition: Neurons as Switches

> Think of ReLU neurons as **on/off switches**. Each neuron "decides" whether a particular feature is present (output > 0) or absent (output = 0). A deep network is like a vast switchboard — different combinations of active/inactive neurons encode different features. This sparsity (many neurons off) makes the network both efficient and interpretable. Leaky ReLU keeps the switch slightly "leaky" so it can still recover if wrongly turned off.

Softplus Activation

Softplus Definition

$$\text{Softplus}(x) = \ln(1 + e^x)$$

A smooth approximation of ReLU. Unlike ReLU (which has a kink at 0), Softplus is differentiable everywhere. Range: $(0, \infty)$.

Softplus Derivative (EXAM IMPORTANT)

$$\frac{d}{dx}\text{Softplus}(x) = \frac{d}{dx}\ln(1 + e^x) = \frac{e^x}{1 + e^x} = \frac{1}{1 + e^{-x}} = \sigma(x)$$

$\sigma(x)$ = sigmoid function | The derivative of Softplus IS the sigmoid function | This elegant relationship means Softplus is the antiderivative (integral) of sigmoid

> **EXAM:** "The derivative of Softplus is the sigmoid function." This is a frequently tested fact. Conversely: $\int \sigma(x)\,dx = \text{Softplus}(x) + C$.

Softplus vs ReLU Comparison

Property	ReLU	Softplus
Formula	$\max(0, x)$	$\ln(1 + e^x)$
Smooth?	No (kink at 0)	Yes (everywhere differentiable)
Derivative	Step function (0 or 1)	Sigmoid (smooth 0→1)
At $x = 0$	$f(0) = 0$	$f(0) = \ln(2) \approx 0.693$
Computation	Cheap (just max)	Expensive (exp + log)
Use case	Default hidden layer activation	When smoothness required (e.g., variance in VAE)

Numerical Example

Problem: Compute Softplus(2) and its derivative.

Solution:

- $\text{Softplus}(2) = \ln(1 + e^2) = \ln(1 + 7.389) = \ln(8.389) = \mathbf{2.127}$
- $\frac{d}{dx}\text{Softplus}(2) = \sigma(2) = \frac{1}{1+e^{-2}} = \frac{1}{1+0.135} = \frac{1}{1.135} = \mathbf{0.881}$

Note: For large x, Softplus$(x) \approx x$ (same as ReLU). For large negative x, Softplus$(x) \approx 0$ (same as ReLU).

Quick Revision Sheet

Function	Formula	Range	Key Property
ReLU	$\max(0, x)$	$[0, \infty)$	No vanishing gradient (pos), dying neurons (neg)
Leaky ReLU	$\max(\alpha x, x)$	$(-\infty, \infty)$	No dying neurons, small negative gradient
PReLU	$\max(\alpha x, x)$, α learned	$(-\infty, \infty)$	α adapts per neuron during training
ELU	x if $x > 0$, else $\alpha(e^x - 1)$	$[-\alpha, \infty)$	Smooth at 0, pushes mean toward 0
GELU	$x \cdot \Phi(x)$	$\approx (-0.17, \infty)$	Smooth, used in Transformers (the attention-based architecture behind BERT/GPT — Topic 9)
Swish	$x \cdot \sigma(x)$	$\approx (-0.28, \infty)$	Self-gated, smooth non-monotonic
Softplus	$\ln(1 + e^x)$	$(0, \infty)$	Smooth ReLU; derivative = sigmoid

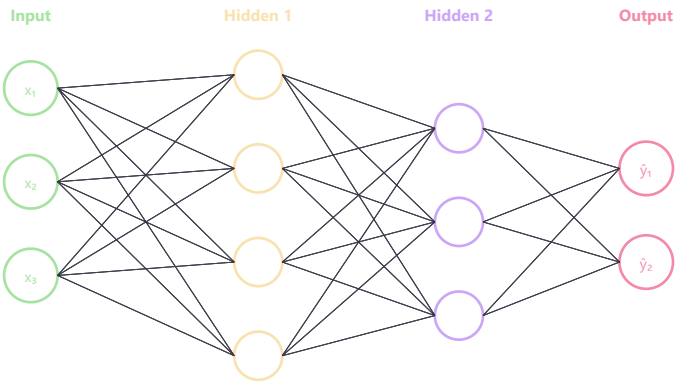
---

Created by **Kushal Ghosh** (gkushalg01@gmail.com) and **Bharat Das Vaishnav** (bharatvaishnav2025@gmail.com)

Topic 3 — Feed-Forward Neural Network (FNN)

FNNs are the foundational deep learning architecture — information flows forward only (input → hidden → output), with no cycles or feedback connections.

1. FNN Architecture



Fully-connected (dense) layers — every neuron connects to all next-layer neurons

Layer	Role	Activation
Input	Receives raw features	None
Hidden	Learns features & patterns	ReLU (default)
Output	Produces final prediction	Softmax (classification) / Linear (regression)

2. FNN vs MLP

FNN	MLP
Broad category: data flows forward	Specific type of FNN
May include CNNs (networks using sliding filters for images — Topic 4)	Only fully-connected (dense) layers
Connections not necessarily all-to-all	All neurons connected to all in next layer

> **Relationship:** MLP ⊂ FNN — Every MLP is an FNN, but not every FNN is an MLP. Think: FNN = "how data flows", MLP = "what layers are used".

3. Forward Propagation

> **Layer-wise Forward Pass**
>
> $$\mathbf{a}^{[l]} = f\left(\mathbf{W}^{[l]}\mathbf{a}^{[l-1]} + \mathbf{b}^{[l]}\right)$$
>
> $\mathbf{a}^{[l]}$ = activation vector of layer l | $\mathbf{W}^{[l]}$ = weight matrix (rows = neurons in l, cols = neurons in $l-1$) | $\mathbf{b}^{[l]}$ = bias vector | f = activation function

Forward Pass Example

Given: $x_1 = 1$, $x_2 = 2$, $w_1 = 0.5$, $w_2 = 1$, $b = 1$, activation = ReLU

$z = 0.5(1) + 1(2) + 1 = 3.5$

$\text{ReLU}(3.5) = \mathbf{3.5}$

4. Softmax Function

Softmax

$$P(y_i) = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

z_i = logit (raw output score) for class i | K = total number of classes | Output: probability distribution that sums to 1

Softmax Visualization

5. Loss Functions

Cross-Entropy Loss (with Softmax)

$$L = -\sum_{i=1}^{K} y_i \log(\hat{y}_i)$$

y_i = one-hot true label (1 for correct class, 0 otherwise) | $\hat{y}_i$ = predicted probability for class i | K = number of classes

Task	Loss Function	Output Activation
Regression	MSE	Linear
Binary classification	Binary Cross-Entropy	Sigmoid
Multi-class	Categorical Cross-Entropy	Softmax

6. Backpropagation & Training

Weight Update (Gradient Descent)

$$w_{new} = w_{old} - \eta \frac{\partial L}{\partial w}$$

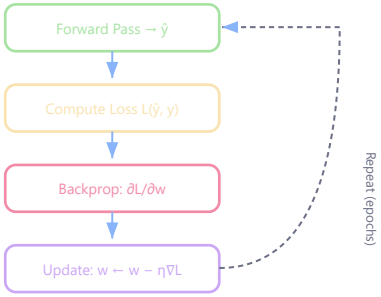
η = learning rate | $\frac{\partial L}{\partial w}$ = gradient of loss w.r.t. weight (computed via chain rule backward through layers)

Training Loop Diagram

Key Training Terminology

Term	Definition	Example (1000 samples, batch=100)
Epoch	One complete pass through entire training dataset	1 epoch = all 1000 samples seen once
Batch Size	Number of samples processed before one weight update	100 samples per batch
Iteration	One weight update (one batch processed)	10 iterations per epoch

Common batch sizes: 16, 32, 64, 128, 256. Powers of 2 align with GPU memory. Larger = smoother gradient but more memory. Smaller = noisier but better generalization.



Backprop Example

Given: $w = 2$, $\frac{\partial L}{\partial w} = 3$, $\eta = 0.1$

$$w_{new} = 2 - 0.1(3) = 2 - 0.3 = \mathbf{1.7}$$

7. Overfitting & Regularization

Sign	Training Accuracy	Validation Accuracy
Underfitting	Low	Low
Good fit	High	High (close)
Overfitting	Very High	Low (gap)

Solutions

Method	How It Helps
Dropout	Randomly disables neurons during training → forces robust features (detailed in Topic 5)
L2 Regularization	Penalizes large weights → simpler model
Early Stopping	Stop when validation loss increases
More Data	Harder to memorize → better generalization
Data Augmentation	Artificial diversity in training data

8. Machine Learning Recipe

9. Example: MNIST Digit Classifier

Parameter Count: [784, 128, 64, 10]

Layer 1: $784 \times 128 + 128 = 100,480$

Layer 2: $128 \times 64 + 64 = 8,256$

Layer 3: $64 \times 10 + 10 = 650$

Total: 109,386 parameters ≈ 0.44 MB (float32)

10. Best Practices & Common Mistakes

- Use ReLU in hidden layers, Softmax for multi-class output
- Normalize inputs (zero mean, unit variance) → faster convergence
- Start simple (few layers), scale gradually
- Monitor validation loss to detect overfitting
- Use Adam optimizer as default, switch to SGD+Momentum for fine-tuning

Common mistakes:

- Too many layers for simple problems → overfitting
- Large learning rate → divergence
- No normalization → unstable training
- Wrong loss function (e.g., MSE for classification)

Advanced Questions & Answers

Q1. FNN architecture [784, 256, 128, 10]. Compute total trainable parameters and memory (float32).

Layer 1: $784 \times 256 + 256 = 200,960$

Layer 2: $256 \times 128 + 128 = 32,896$

Layer 3: $128 \times 10 + 10 = 1,290$

Total: 235,146 parameters. Memory = $235,146 \times 4$ bytes = 940,584 ≈ **94 MB**

Q2. Complete forward+backward pass: input $x = 2$, hidden (1 neuron, ReLU), output (1 neuron, linear), target $y = 1$, η=0.01. Weights: $w_1 = 0.5$, $b_1 = 0.1$, $w_2 = 0.8$, $b_2 = -0.1$.

Forward: $z_1 = 0.5(2) + 0.1 = 1.1 \rightarrow h_1 = \text{ReLU}(1.1) = 1.1 \rightarrow \hat{y} = 0.8(1.1) - 0.1 = 0.78$

Loss: $L = (0.78 - 1)^2 = 0.0484$

Backward: $\frac{\partial L}{\partial \hat{y}} = 2(0.78 - 1) = -0.44$

$\frac{\partial L}{\partial w_2} = -0.44 \times 1.1 = -0.484$ | $\frac{\partial L}{\partial b_2} = -0.44$

$\frac{\partial L}{\partial w_1} = -0.44 \times 0.8 \times 1 \times 2 = -0.704$ | $\frac{\partial L}{\partial b_1} = -0.352$

Updated: $w_1 = 0.507$, $b_1 = 0.1035$, $w_2 = 0.8048$, $b_2 = -0.0956$

Q3. Logits $z = [2.0, 1.0, 0.1]$, true class = 0. Compute softmax + cross-entropy gradient w.r.t. each logit.

$e^{2.0} = 7.389$, $e^{1.0} = 2.718$, $e^{0.1} = 1.105$ → Sum = 11.212

$p = [0.659, 0.242, 0.099]$. Loss $= -\log(0.659) = 0.417$

Gradient of softmax+CE: $\frac{\partial L}{\partial z_i} = p_i - y_i$

$\nabla z = [0.659 - 1,\ 0.242 - 0,\ 0.099 - 0] = [-0.341,\ 0.242,\ 0.099]$

Pushes correct class logit higher, wrong class logits lower.

Q4. 10-layer FNN with sigmoid. Estimate max gradient reaching layer 1.

$\sigma'(z)_{max} = 0.25$ (at $z = 0$). Gradients multiply through 9 layers:

Best case: $0.25^9 \approx 3.8 \times 10^{-6}$ — gradient is ~260,000× smaller at layer 1.

This is why sigmoid networks can't go deep. ReLU (gradient = 1 for $z > 0$) solved this.

Q5. Compare Batch GD, Mini-batch (32), and SGD (1) on 10,000 samples: updates/epoch, convergence.

Method	Updates/Epoch	Gradient Quality	Convergence
Batch GD	1	Exact	Smooth but slow, memory-heavy
Mini-batch (32)	313	Noisy (32 samples)	Good balance — standard choice
SGD (1)	10,000	Very noisy	Fast early, oscillates near minimum

Mini-batch 32 is standard: powers-of-2 align with GPU, noise provides implicit regularization, and 313 updates/epoch gives fast convergence.

Basic Q&A

BQ1. What makes a network "feed-forward"?

Information flows in one direction only: input → hidden → output. No cycles or loops (unlike RNNs — networks with loops for sequential data, covered in Topic 7).

BQ2. How many parameters in a layer with 10 inputs and 5 outputs?

Weights: $10 \times 5 = 50$. Biases: 5. Total = **55** parameters.

BQ3. What is the difference between MLP and FNN?

FNN = any network without cycles. MLP = specific FNN with ≥1 hidden layers, fully connected, using non-linear activations. All MLPs are FNNs, but a single perceptron (FNN with 0 hidden layers) is not an MLP.

BQ4. Why do we need non-linear activation functions?

Without them, stacking layers is equivalent to one linear layer ($W_2 \cdot W_1 \cdot x = W_{combined} \cdot x$). Non-linearity lets networks learn complex, non-linear decision boundaries.

BQ5. What is the role of the bias term?

Bias shifts the activation function, allowing the neuron to activate even when all inputs are zero. Without bias, the decision boundary must pass through the origin.

PyTorch Autograd & Exam Numericals

PyTorch Gradient Computation (Mid-Term Q2)

Problem: Predict the exact printed output:

```
import torch

a = torch.tensor(2.0, requires_grad=True)
b = torch.tensor(3.0, requires_grad=True)

y = a * b + b ** 2

y.backward()

print("a.grad:", a.grad.item())
print("b.grad:", b.grad.item())
```

Solution:

$y = ab + b^2$

Partial derivatives:

$$\frac{\partial y}{\partial a} = b = 3.0$$

$$\frac{\partial y}{\partial b} = a + 2b = 2.0 + 6.0 = 8.0$$

Output:

```
a.grad: 3.0
b.grad: 8.0
```

PyTorch autograd builds a computation graph and applies chain rule automatically. `.backward()` computes gradients for all tensors with `requires_grad=True`.

MSE Loss Computation (Mid-Term Q3)

Problem: A two-layer feedforward neural network (no bias) has:

$$W_1 = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}, \quad W_2 = [1\ 1]$$

Input: $x = [1, 1, 1]^T$. Ground truth: $y = 50$. Find MSE loss.

Step 1 — Hidden layer:

$$h = W_1 x = \begin{bmatrix} 1+2+3 \\ 4+5+6 \end{bmatrix} = \begin{bmatrix} 6 \\ 15 \end{bmatrix}$$

Step 2 — Output:

$$\hat{y} = W_2 h = [1\ 1] \begin{bmatrix} 6 \\ 15 \end{bmatrix} = 6 + 15 = 21$$

Step 3 — MSE Loss:

$$\text{MSE} = (\hat{y} - y)^2 = (21 - 50)^2 = (-29)^2 = \mathbf{841}$$

SGD Properties (Mid-Term Q7 MSQ)

Which statements about SGD vs Batch Gradient Descent are TRUE?

(a) SGD updates using entire dataset at each iteration

(b) SGD noise helps escape sharp local minima

(c) SGD always converges to global minimum more efficiently

(d) SGD with momentum accumulates exponentially decaying moving average of past gradients

---

Correct: (b), (d)

- (a) FALSE — SGD uses ONE sample (or mini-batch), not entire dataset. Batch GD uses entire dataset.
- (b) TRUE — Gradient noise from random sampling helps escape sharp minima → finds flatter minima with better generalization.
- (c) FALSE — SGD does NOT guarantee convergence to global minimum. It's noisy and may oscillate. But it often generalizes better.
- (d) TRUE — Momentum: $v_t = \gamma v_{t-1} + \eta \nabla L$. Exponential decay ($\gamma$ typically 0.9) of past gradients accelerates consistent directions.

Why Deep Networks Over Wide? (Mid-Term Q4)

Why are deep networks (many layers, moderate width) preferred over wide networks (few layers, very large width) for image recognition?

Correct answer (c): Deep networks learn **hierarchical representations** (edges → textures → shapes → objects), requiring exponentially fewer neurons than a single wide layer to represent the same compositional function.

- **Compositionality:** A 10-layer network can represent a function requiring 2^{10} neurons in a single wide layer
- **Feature hierarchy:** Layer 1 detects edges, Layer 2 combines edges into textures, Layer 3 combines textures into parts → each layer REUSES lower features
- **Parameter efficiency:** Sharing features across layers means far fewer total parameters
- **Universal Approximation Theorem:** Both can approximate any function, but deep networks are exponentially MORE EFFICIENT for compositional functions

Why NOT (a): Wide networks CAN represent non-linear functions (with activation). **Why NOT (b):** Not purely computational — deep is actually more parameter-efficient. **Why NOT (d):** Deep doesn't always have fewer parameters (VGG-19 has 138M).

Quick Revision Sheet

Concept	Key Point
FNN	No cycles; info flows input→output. Universal with enough neurons.
Forward pass	$a^{(l)} = f(W^{(l)}a^{(l-1)} + b^{(l)})$ layer by layer
Backprop	Chain rule from loss backward; $\delta^{(l)} = (W^{(l+1)^T}\delta^{(l+1)}) \odot f'$
Epoch	One full pass through training data
Batch size	Samples per weight update (32 standard)
Overfitting	Train acc ≫ val acc → use dropout, regularization, more data
Dropout	Randomly zero neurons during training (p=0.5 typical)
Adam	Adaptive LR per parameter using momentum + RMSprop

Created by **Kushal Ghosh** (gkushalg01@gmail.com) and **Bharat Das Vaishnav** (bharatvaishnav2025@gmail.com)

↑ Back to Top | **CNN Fundamentals**

Topic 4 — Convolutional Neural Networks (CNN)

1. Introduction

Convolutional Neural Networks (CNNs) are specialized neural networks designed primarily for:

- image processing
- computer vision
- spatial pattern recognition

CNNs are extremely powerful for:

- object detection
- face recognition
- medical imaging
- autonomous vehicles
- OCR
- video analysis

2. Why CNNs Were Needed

Traditional Feed-Forward Neural Networks (FNNs):

- become inefficient for images
- require huge numbers of parameters

Example: A (256×256) RGB image contains: $256 \times 256 \times 3$ pixels. Fully connecting this to dense layers becomes computationally enormous.

CNNs solve this by:

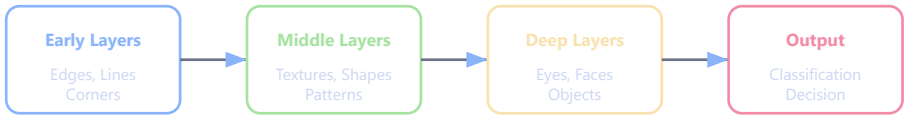
- sharing weights
- learning local patterns
- extracting hierarchical features

3. Learning Progression

```
Image Representation
    ↓
Convolution Operation
    ↓
Feature Maps
    ↓
Convolution Layer
    ↓
Activation Function
    ↓
Pooling Layer
    ↓
Flattening
    ↓
Fully Connected Layer
    ↓
CNN Architecture
    ↓
CNN Training
```

4. What CNNs Learn

CNNs learn hierarchical visual features.



Early Layers Learn

- edges
- lines
- corners

Middle Layers Learn

- textures
- shapes
- patterns

Deep Layers Learn

- eyes
- faces
- objects

5. Core Components of CNN

Component	Purpose
Convolution Layer	Feature extraction
Activation Function	Non-linearity
Pooling Layer	Dimensionality reduction
Fully Connected Layer	Final classification

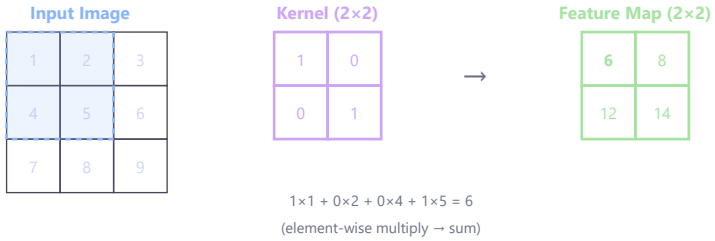
PART 1 — Convolution Operation

6. Convolution Operation

Convolution is the core mathematical operation in CNNs. It helps detect: patterns, edges, textures from images.

7. Basic Idea

A small matrix called **Kernel / Filter** slides over the image. At each position: multiplication occurs, values are summed. This produces a **Feature Map**.



8. Example Input Image

Example (3×3) image:

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

9. Example Filter

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

10. Convolution Step

Multiply overlapping values:

$$(1)(1) + (2)(0) + (4)(0) + (5)(1) = 1 + 5 = 6$$

This becomes first output value.

11. Convolution Formula

$$(I * K)(x, y) = \sum_m \sum_n I(x - m, y - n) K(m, n)$$

I = input image | K = kernel/filter | (x, y) = output position | (m, n) = kernel indices

12. What Filters Detect

Filter Type	Detects
Edge filter	Edges
Blur filter	Smoothing
Sharpen filter	Sharp details

13. Why Convolution Is Powerful

Instead of learning every pixel individually, CNN learns reusable visual patterns. This dramatically reduces parameters.

14. Important Terms

Term	Meaning
Kernel/Filter	Small learnable matrix
Feature Map	Output after convolution
Stride	Step size of filter
Padding	Extra border added

15. Stride

Stride determines **how many pixels filter moves at a time**.

Stride = 1: move one pixel. Stride = 2: skip every alternate position.

16. Padding

Padding adds zeros around image borders. Purpose: preserve image size, avoid information loss.

17. Output Size Formula

$$\text{Output} = \left\lfloor \frac{N - F + 2P}{S} \right\rfloor + 1$$

N = input size | F = filter/kernel size | P = padding | S = stride | $\lfloor \cdot \rfloor$ = floor function (round down to nearest integer)

18. Output Size Numerical

Problem: Input $N = 5$, Filter $F = 3$, Padding $P = 0$, Stride $S = 1$. Find output size.

Solution:

$$\text{Output} = \frac{(5 - 3 + 0)}{1} + 1 = 2 + 1 = 3$$

Final Answer: 3×3

PART 2 — Convolution Layer

19. Convolution Layer

A convolution layer applies multiple filters, extracts different features. Each filter learns different patterns.

20. Multiple Filters

- Filter 1 → edges
- Filter 2 → textures
- Filter 3 → curves

21. Activation After Convolution

Usually ReLU is applied:

$$f(z) = \max(0, z)$$

22. Why ReLU Is Used

Advantages: fast, reduces vanishing gradients, introduces non-linearity.

23. Feature Maps

Each filter produces **one feature map**. More filters → richer feature extraction.

24. Parameter Sharing

A single filter is reused across image. Benefits: fewer parameters, better efficiency.

25. Sparse Connectivity

Each neuron connects only to local image regions. Unlike FNNs — not every pixel connects everywhere.

PART 3 — Pooling

26. Pooling Layer

Pooling reduces: feature map size, computation, overfitting.

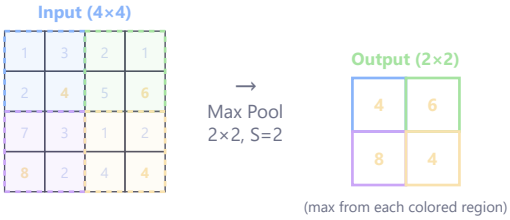
27. Why Pooling Matters

Pooling compresses information, retains important features.

28. Types of Pooling

Pooling Type	Operation
Max Pooling	Takes maximum
Average Pooling	Takes average

29. Max Pooling Example



Input: $\begin{bmatrix} 1 & 3 \\ 2 & 4 \end{bmatrix}$ → Max Pooling Output: 4

30. Why Max Pooling Is Popular

Because strongest features often matter most.

31. Pooling Numerical

Problem: Apply (2×2) max pooling. Input: $\begin{bmatrix} 1 & 5 \\ 7 & 2 \end{bmatrix}$

Solution: Maximum value = 7

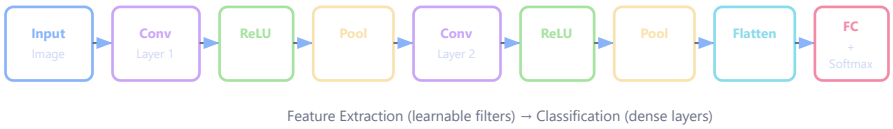
Final Answer: 7

32. Pooling Benefits

Benefit	Explanation
Reduces computation	Smaller feature maps
Reduces overfitting	Less parameters
Translation invariance	Robustness to movement

PART 4 — Full CNN Architecture

33. Full CNN Pipeline



Feature Extraction (learnable filters) → Classification (dense layers)

```
Input Image
    ↓
Convolution Layer
    ↓
ReLU
    ↓
Pooling
    ↓
Convolution Layer
    ↓
ReLU
    ↓
Pooling
    ↓
Flatten
```

```
    ↓
Fully Connected Layer
    ↓
Softmax Output
```

34. Flattening

Flatten converts 2D feature maps into 1D vector. Needed before dense layers.

35. Fully Connected Layer

Dense layers perform final classification.

36. Softmax Output Layer

Softmax converts outputs into probabilities:

> $$P(y_i) = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$
>
> z_i = raw logit for class i | K = total number of classes

37. CNN Workflow Example

Problem: Cat vs Dog Classification

```
Image Input
    ↓
Convolution detects edges
    ↓
Pooling compresses features
    ↓
Deeper layers detect shapes
    ↓
Fully connected layers classify
    ↓
Output: Cat or Dog
```

38. Why CNNs Work So Well for Images

Images contain spatial locality and repeating patterns. CNNs exploit this using local filters and shared weights.

PART 5 — CNN Implementation & Training

39. CNN Training Workflow

```
Input Image
    ↓
Forward Pass
    ↓
Prediction
    ↓
Loss Calculation
    ↓
Backpropagation
```

```
    ↓
Weight Update
```

40. Loss Functions in CNNs

Task	Loss Function
Binary classification	BCE
Multi-class classification	Cross Entropy

41. Optimizers Used

Optimizer	Usage
SGD	Traditional
Adam	Most popular

42. CNN Hyperparameters

Hyperparameter	Meaning
Filter size	Kernel dimensions
Number of filters	Feature detectors
Stride	Filter movement
Padding	Border extension
Learning rate	Update step size

43. CNN Numerical — Parameter Count

Problem: Input channels: 3, Filter size: 3×3, Number of filters: 10. Find total parameters.

Formula:

Parameters per filter: $3 \times 3 \times 3 = 27$

Add bias: $27 + 1 = 28$

For 10 filters: $28 \times 10 = \mathbf{280}$

44. Common CNN Architectures

> ⏩ These are introduced briefly here. Full details on each architecture (ResNet skip connections, VGG depth strategy, etc.) are in **Topic 5**.

Architecture	Importance
LeNet	Early CNN
AlexNet	Deep learning breakthrough
VGG	Very deep CNN
ResNet	Residual learning

45. Best Practices

Use ReLU

Improves: training speed, gradient flow.

Normalize Images

Common range: $[0, 1]$ or $[-1, 1]$.

Use Max Pooling

Helps reduce dimensions, preserve important features.

Start With Small CNN

Avoid over-complex models initially.

Use Data Augmentation

Examples: rotation, flipping, cropping. Helps reduce overfitting.

46. Common Beginner Mistakes

Mistake	Problem
Too many filters	High computation
No pooling	Large memory usage
Large learning rate	Unstable training
Very deep network initially	Hard debugging

47. Most Asked Interview Questions

Q1. Why are CNNs better than FNNs for images?

CNNs use local connectivity, share weights, require far fewer parameters.

Q2. What does convolution do?

It extracts visual features like edges, textures, patterns.

Q3. Why is pooling used?

Pooling reduces dimensions, reduces overfitting, improves robustness.

Q4. What is stride?

Stride determines how far filter moves each step.

Q5. Why is ReLU used in CNNs?

Because: computationally efficient, reduces vanishing gradients.

Q6. Why use multiple filters?

Different filters learn different visual patterns.

48. CNN vs FNN

CNN	FNN
Best for images	General-purpose

Uses convolution	Uses dense layers
Fewer parameters	Large parameter count
Learns spatial features	No spatial understanding

49. Quick Revision Sheet

Concept	Key Idea
Convolution	Feature extraction
Filter	Learnable pattern detector
Feature Map	Convolution output
Pooling	Dimension reduction
ReLU	Non-linearity
Flatten	Convert to vector
Softmax	Probability output

50. Final End-to-End CNN Flow

```
Image
  ↓
Convolution
  ↓
ReLU
  ↓
Pooling
  ↓
Feature Maps
  ↓
Flatten
  ↓
Dense Layers
  ↓
Softmax
  ↓
Prediction
```

PART 6 — Object Detection

51. What Is Object Detection?

Object detection means: **locating and classifying multiple objects in an image simultaneously**.

Unlike classification (one label per image), detection outputs:

- bounding boxes (where objects are)
- class labels (what each object is)
- confidence scores (how certain)

52. Object Detection vs Classification vs Segmentation

Task	Output
Classification	One label for entire image
Detection	Multiple bounding boxes + labels
Segmentation	Label for every pixel

53. Two-Stage vs One-Stage Detectors

Type	Approach	Examples
Two-stage	First propose regions, then classify	R-CNN, Faster R-CNN
One-stage	Directly predict boxes from image	YOLO, SSD

Two-stage: more accurate but slower. One-stage: faster but slightly less precise.

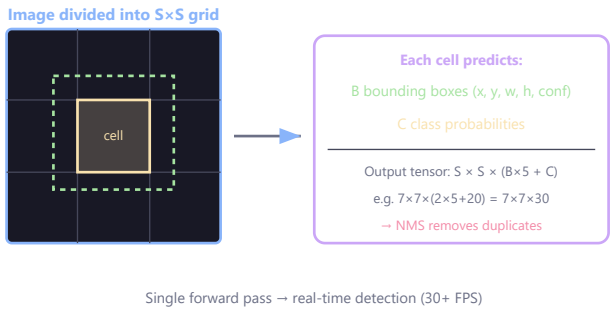
54. Two-Stage Detector Pipeline

```
Input Image
    ↓
Region Proposal Network (RPN)
    ↓
Candidate Regions
    ↓
CNN Feature Extraction per region
    ↓
Classification + Bounding Box Regression
    ↓
Output: Objects with locations
```

55. YOLO (You Only Look Once)

YOLO is the most famous one-stage detector. Core idea: **divide image into grid, predict boxes and classes in one forward pass**.

56. How YOLO Works




```
Input Image
    ↓
Divide into S×S grid
    ↓
Each grid cell predicts:
  - B bounding boxes
  - Confidence scores
  - C class probabilities
    ↓
Non-Max Suppression (NMS)
```

```
    ↓
Final Detections
```

57. YOLO Grid Prediction

Each grid cell predicts for each bounding box: (x, y, w, h, c)

x, y = center of bounding box relative to grid cell | w, h = width and height relative to full image | c = confidence score = P(object) × IoU(pred, truth)

58. IoU (Intersection over Union)

$$IoU = \frac{\text{Area of Overlap}}{\text{Area of Union}}$$

Area of Overlap = intersection of predicted and ground truth boxes | Area of Union = total area covered by both boxes combined | IoU = 1 means perfect match, IoU = 0 means no overlap

59. Non-Max Suppression (NMS)

When multiple boxes detect the same object:

1. Keep box with highest confidence
2. Remove boxes with high IoU overlap with the kept box
3. Repeat until no redundant boxes remain

60. YOLO Loss Function (Simplified)

$$L_{YOLO} = \lambda_{coord} \sum (x - \hat{x})^2 + \lambda_{coord} \sum (w - \hat{w})^2 + \sum (C - \hat{C})^2 + \sum (p - \hat{p})^2$$

λ_{coord} = weight for localization loss (typically 5) | $(x - \hat{x})^2$ = error in predicted center position | $(w - \hat{w})^2$ = error in predicted box dimensions | $(C - \hat{C})^2$ = confidence score error | $(p - \hat{p})^2$ = class probability error

61. Why YOLO Is Fast

- Single forward pass (no region proposals)
- Entire image processed at once
- Real-time detection possible (30+ FPS)

62. YOLO Versions

Version	Key Improvement
YOLOv1	Grid-based detection
YOLOv2	Batch norm, anchor boxes
YOLOv3	Multi-scale detection
YOLOv4+	Better backbones, training tricks

63. Object Detection Metrics

Metric	Meaning
Precision	Fraction of detections that are correct
Recall	Fraction of true objects that are detected

mAP (mean Average Precision)	Average precision across all classes
IoU threshold	Minimum overlap to count as correct

64. Numerical Example — IoU

Problem: Predicted box area: 100 pixels. Ground truth box area: 120 pixels. Overlap area: 80 pixels.

Solution: Union = 100 + 120 − 80 = 140

$$IoU = \frac{80}{140} = 0.571$$

At IoU threshold 0.5: this counts as a correct detection.

Advanced Questions & Answers

Q1. A 5×5 input image is convolved with a 3×3 filter (stride=1, no padding). Compute the output size. Then apply the filter to the input.

```
Input:
1  0  1  0  1
0  1  0  1  0
1  0  1  0  1
0  1  0  1  0
1  0  1  0  1

Filter:
 1  0 -1
 0  1  0
-1  0  1
```

Output size: (5−3)/1 + 1 = 3×3

Computing each output element (sum of element-wise products):

Position (0,0): 1(1)+0(0)+1(−1)+0(0)+1(1)+0(0)+1(−1)+0(0)+1(1) = 1+0−1+0+1+0−1+0+1 = **1**

Position (0,1): 0(1)+1(0)+0(−1)+1(0)+0(1)+1(0)+0(−1)+1(0)+0(1) = **0**

Position (0,2): 1(1)+0(0)+1(−1)+0(0)+1(1)+0(0)+1(−1)+0(0)+1(1) = **1**

Position (1,0): 0(1)+1(0)+0(−1)+1(0)+0(1)+1(0)+0(−1)+1(0)+0(1) = **0**

Position (1,1): 1(1)+0(0)+1(−1)+0(0)+1(1)+0(0)+1(−1)+0(0)+1(1) = **1**

Position (1,2): 0(1)+1(0)+0(−1)+1(0)+0(1)+1(0)+0(−1)+1(0)+0(1) = **0**

Position (2,0): 1(1)+0(0)+1(−1)+0(0)+1(1)+0(0)+1(−1)+0(0)+1(1) = **1**

Position (2,1): 0(1)+1(0)+0(−1)+1(0)+0(1)+1(0)+0(−1)+1(0)+0(1) = **0**

Position (2,2): 1(1)+0(0)+1(−1)+0(0)+1(1)+0(0)+1(−1)+0(0)+1(1) = **1**

```
Output:
1  0  1
0  1  0
1  0  1
```

This filter detects diagonal patterns (acts as diagonal edge detector).

Q2. A CNN layer has input of size 224×224×3 with 64 filters of size 7×7, stride=2, padding=3. Compute: (a) output dimensions, (b) total parameters, (c) total multiply-add operations.

(a) Output dimensions:

$$H_{out} = \frac{224 - 7 + 2(3)}{2} + 1 = \frac{223}{2} + 1 = 112$$

Output: **112 × 112 × 64**

(b) Parameters:

Each filter: 7 × 7 × 3 = 147 weights + 1 bias = 148

Total: 148 × 64 = **9,472 parameters**

(c) Multiply-add operations:

Per output pixel: 7 × 7 × 3 = 147 multiply-adds

Total output pixels: 112 × 112 × 64 = 802,816

Total MACs: 147 × 802,816 = **118,013,952 ≈ 118M MACs**

Q3. YOLO divides an image into a 7×7 grid. Each cell predicts 2 bounding boxes with 5 parameters each, plus 20 class probabilities. What is the final output tensor shape? How does NMS reduce the 98 predictions to final detections?

Output tensor:

Per cell: 2 boxes × 5 params (x, y, w, h, confidence) + 20 classes = 30

Total output: 7 × 7 × 30 = **1470 values** (tensor shape: 7×7×30)

Total boxes: 7 × 7 × 2 = **98 bounding boxes**

NMS process:

1. For each class, filter boxes with confidence × class_prob < threshold (e.g., 0.5) → removes ~80 weak boxes
2. Sort remaining boxes by score (descending)
3. Take highest-scoring box as detection
4. Compute IoU of all other boxes with this box
5. Remove boxes with IoU > 0.5 (they detect same object)
6. Repeat from step 3 until no boxes remain
7. Typical result: 3-10 final detections per image

Q4. Compare the receptive field of a single 7×7 conv layer vs. three stacked 3×3 conv layers. Compute parameters for each assuming 64 input/output channels.

Receptive field:

- Single 7×7: receptive field = 7×7 ✓
- Three 3×3: receptive field = 3 + 2 + 2 = 7×7 ✓ (same!)

Each 3×3 layer adds 2 to receptive field (3−1=2 per layer, total = 1+2+2+2 = 7).

Parameters:

- Single 7×7: 7×7×64×64 + 64 = **200,768**
- Three 3×3: 3×(3×3×64×64 + 64) = 3×(36,864 + 64) = **110,784**

Comparison: Three 3×3 uses **45% fewer parameters** for the same receptive field, PLUS adds two extra non-linearities (ReLU between layers), increasing representational power. This is why VGGNet uses exclusively 3×3 filters.

Q5. An object detector produces the following predictions on an image containing 3 true cars (GT boxes A, B, C). Compute precision, recall, and mAP@0.5.

Box	Confidence	IoU with nearest GT	Match
P1	0.95	0.72 with A	TP
P2	0.90	0.61 with B	TP
P3	0.85	0.45 with C	FP (IoU<0.5)
P4	0.70	0.30 with A	FP (already matched)
P5	0.60	0.55 with C	TP

At IoU threshold 0.5:

- P1: IoU=0.72>0.5, first match with A → **TP**
- P2: IoU=0.61>0.5, first match with B → **TP**
- P3: IoU=0.45<0.5 → **FP**
- P4: IoU=0.30<0.5 (also A already matched) → **FP**
- P5: IoU=0.55>0.5, first match with C → **TP**

Final: TP=3, FP=2, FN=0

Precision = 3/5 = **60**

Recall = 3/3 = **00**

For mAP, compute precision at each recall level:

Cumulative TP	Precision	Recall
1 (P1)	1/1=1.0	1/3=0.33
2 (P2)	2/2=1.0	2/3=0.67
2 (P3)	2/3=0.67	2/3=0.67
2 (P4)	2/4=0.50	2/3=0.67
3 (P5)	3/5=0.60	3/3=1.0

AP = area under P-R curve (11-point interpolation):

At each recall r, take max precision at recall ≥ r:

- At recall 0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6: max precision = 1.0
- At recall 0.7, 0.8, 0.9, 1.0: max precision = 0.60

AP = (7×1.0 + 4×0.60)/11 = (7+2.4)/11 ≈ **855**

Q6. CNN Shape Tracing with MaxPool (Mid-Term Q5)

Predict the output of this PyTorch model:

```
model = nn.Sequential(
    nn.Conv2d(1, 8, kernel_size=3, stride=1, padding=1),
    nn.ReLU(),
    nn.MaxPool2d(2, 2),
    nn.Conv2d(8, 16, kernel_size=3, stride=1, padding=1),
    nn.ReLU(),
    nn.MaxPool2d(2, 2),
)
x = torch.randn(1, 1, 28, 28)
out = model(x)
print("Shape:", out.shape)
print("Total elements:", out.numel())
```

Tracing dimensions step by step:

Formula: Conv with stride=1, padding=1, kernel=3 → output size = input size (same padding)

MaxPool2d(2,2) → halves spatial dimensions

Layer	Output Shape	Reasoning
Input	(1, 1, 28, 28)	Given
Conv2d(1→8, k=3, s=1, p=1)	(1, 8, 28, 28)	Same padding: $\lfloor (28+2-3)/1 \rfloor + 1 = 28$
ReLU	(1, 8, 28, 28)	No shape change
MaxPool2d(2,2)	(1, 8, 14, 14)	28/2 = 14
Conv2d(8→16, k=3, s=1, p=1)	(1, 16, 14, 14)	Same padding: $\lfloor (14+2-3)/1 \rfloor + 1 = 14$
ReLU	(1, 16, 14, 14)	No shape change
MaxPool2d(2,2)	**(1, 16, 7, 7)**	14/2 = 7

Output:

```
Shape: torch.Size([1, 16, 7, 7])
Total elements: 784
```

> Key pattern: Conv with same padding (p=k//2, s=1) preserves spatial size. MaxPool2d(2,2) halves it. So 28 → 14 → 7. Total elements = 16×7×7 = 784.

Created by **Kushal Ghosh** (gkushalg01@gmail.com) and **Bharat Das Vaishnav** (bharatvaishnav2025@gmail.com)

Topic 5 — Different CNN Architectures

1. Introduction

As CNNs became deeper and more powerful, researchers faced major challenges:

- vanishing gradients
- exploding gradients
- overfitting
- computational complexity

To solve these issues, several advanced CNN architectures were developed: AlexNet, VGGNet, GoogLeNet, DenseNet.

These architectures progressively improved: accuracy, training stability, parameter efficiency, feature reuse.

2. Learning Progression

```
Basic CNN
    ↓
Deep CNN Problems
    ↓
Vanishing/Exploding Gradients
    ↓
Normalization
    ↓
Regularization
    ↓
AlexNet
    ↓
VGGNet
    ↓
GoogLeNet
    ↓
DenseNet
    ↓
Real-world Defect Detection Pipeline
```

PART 1 — Vanishing & Exploding Gradients

3. What Are Gradients?

Gradients tell the network **how much weights should change**. During backpropagation, gradients flow backward through layers.

4. Vanishing Gradient Problem

In deep networks, gradients become extremely small. Result: earlier layers learn very slowly or stop learning.

5. Why Vanishing Gradients Happen

Repeated multiplication of small derivatives:

$$0.1 \times 0.1 \times 0.1 \times 0.1 = 0.0001$$

Gradients shrink rapidly.

6. Effects of Vanishing Gradients

Problem	Effect
Slow learning	Training becomes inefficient
Dead early layers	Features not learned
Poor accuracy	Weak representation

7. Exploding Gradient Problem

Gradients become extremely large:

$$10 \times 10 \times 10 = 1000$$

8. Effects of Exploding Gradients

Problem	Effect
Huge weight updates	Instability
Divergence	Training fails
NaN losses	Numerical overflow

9. Most Asked Interview Question

Q: Why do deep networks suffer from vanishing gradients?

Answer: Because gradients are repeatedly multiplied through many layers, causing them to shrink exponentially.

10. Solutions to Gradient Problems

Problem	Solution
Vanishing gradients	ReLU
Vanishing gradients	Batch Normalization
Exploding gradients	Gradient Clipping
Both	Proper Initialization

11. Why ReLU Helps

$$f'(z) = \begin{cases} 1 & z > 0 \\ 0 & z \leq 0 \end{cases}$$

$f'(z)$ = derivative of ReLU with respect to input z | Positive activations maintain stronger gradients (derivative = 1 instead of shrinking)

12. Numerical Example — Vanishing Gradient

Problem: Compute 0.2^5

Solution: $0.2 \times 0.2 \times 0.2 \times 0.2 \times 0.2 = 0.00032$

Observation: Gradient becomes extremely small.

PART 2 — Normalization & Regularization

13. Why Normalization Is Needed

During training, activations may vary wildly. Normalization stabilizes: training, gradients, convergence.

14. Batch Normalization

BatchNorm normalizes activations across mini-batch samples.

15. BatchNorm Formula

$$\hat{x} = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

$\hat{x}$ = normalized activation | x = original activation value | μ = mean of activations across the mini-batch | σ^2 = variance of activations across the mini-batch | ϵ = small constant (e.g., 1e-5) to prevent division by zero

16. Why BatchNorm Helps

Benefits: faster convergence, stable gradients, allows higher learning rates.

17. Layer Normalization

LayerNorm normalizes across features instead of batch dimension. Mostly used in Transformers (the attention-based architecture covered in Topic 9) and NLP models.

18. BatchNorm vs LayerNorm

BatchNorm	LayerNorm
Across batch	Across features
CNNs	Transformers/RNNs
Depends on batch size	Independent of batch size

19. Regularization

Regularization prevents **overfitting**.

20. Overfitting

Model memorizes training data instead of learning general patterns.

21. Common Regularization Techniques

Technique	Purpose
Dropout	Random neuron removal

L2 Regularization	Penalize large weights
Data Augmentation	Increase dataset diversity

22. Dropout

Dropout randomly disables neurons during training.

23. Why Dropout Helps

Prevents neuron co-dependency and memorization. Encourages robust feature learning.

24. L2 Regularization Formula

$$L_{total} = L + \lambda \sum w^2$$

L_{total} = total loss used for optimization | L = original task loss (e.g., cross-entropy) | λ = regularization strength (hyperparameter) | $\sum w^2$ = sum of squares of all weight values in the network

25. Most Asked Interview Question

Q: Why is BatchNorm important?

Answer: Because it stabilizes activations and improves gradient flow, enabling faster and more stable training.

PART 3 — AlexNet

26. AlexNet

Introduced by Alex Krizhevsky and team. Won ImageNet 2012 competition. Marked the beginning of modern deep learning revolution.

27. Why AlexNet Was Revolutionary

Before AlexNet, deep CNNs were difficult to train. AlexNet introduced: ReLU, Dropout, GPU training, Data augmentation.

28. AlexNet Architecture


```
Input
  ↓
Conv → ReLU → Pooling
  ↓
Conv → ReLU → Pooling
  ↓
Fully Connected
  ↓
Softmax
```

29. Key Features of AlexNet

Feature	Importance

ReLU	Faster training
Dropout	Reduced overfitting
GPU usage	Practical deep learning
Data augmentation	Better generalization

30. Most Asked Interview Question

Q: Why was AlexNet historically important?

Answer: It demonstrated that deep CNNs trained on GPUs could dramatically outperform traditional computer vision methods.

PART 4 — VGGNet

31. VGGNet

Developed by Visual Geometry Group.

32. Main Idea of VGG

Use very small filters repeatedly instead of large filters.

33. VGG Architecture Idea

Repeated 3×3 convolutions.

34. Why Small Filters Help

Multiple small filters: reduce parameters, increase non-linearity, improve feature extraction.

35. VGG Example

Instead of 7×7, use three 3×3 layers.

36. Drawback of VGG

Very large parameter count. Requires high memory and large computation.

37. Most Asked Interview Question

Q: Why did VGG use small filters?

Answer: Small filters reduce parameters while increasing depth and representational power.

PART 5 — GoogLeNet (Inception)

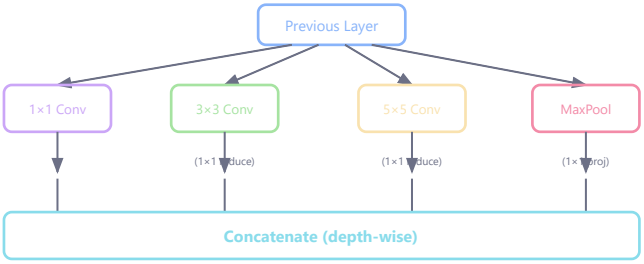
38. GoogLeNet

Introduced the **Inception Module**. Main goal: computational efficiency.

39. Inception Idea

Instead of choosing one filter size, use multiple filter sizes simultaneously.

40. Inception Module




```
1x1 Conv
3x3 Conv
5x5 Conv
Pooling
    ↓
Concatenate Outputs
```

41. Why GoogLeNet Was Efficient

It used 1×1 convolutions to reduce dimensions. This reduced computation and parameters.

42. Most Asked Interview Question

Q: Why are 1x1 convolutions useful?

Answer: They reduce channel dimensions and computational cost while preserving important information.

PART 6 — DenseNet

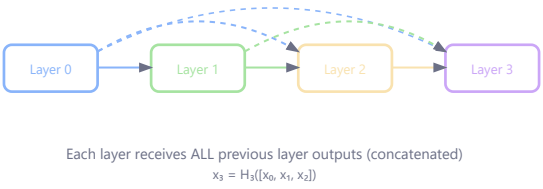
43. DenseNet

DenseNet introduced **dense connectivity**.

44. Main Idea

Each layer receives inputs from all previous layers.

45. Dense Connectivity



Each layer receives ALL previous layer outputs (concatenated)
$x_3 = H_3([x_0, x_1, x_2])$

```
Layer1 → Layer2
  ↓        ↓
Layer3 ←——

Every layer shares features.
```

46. Why DenseNet Helps

- better gradient flow
- feature reuse
- fewer parameters

- reduced vanishing gradients

47. DenseNet Formula

> $$x_l = H_l([x_0, x_1, \ldots, x_{l-1}])$$
>
> All previous outputs are concatenated and fed as input to layer l | H_l = composite function (BN + ReLU + Conv)

48. DenseNet Advantages

Advantage	Explanation
Feature reuse	Earlier features reused
Better gradients	Easier training
Fewer parameters	Efficient learning

49. Most Asked Interview Question

Q: Why does DenseNet improve gradient flow?

Answer: Because every layer directly receives outputs from earlier layers, making gradient propagation easier.

PART 7 — Comparison of Architectures

50. Architecture Comparison

Architecture	Main Innovation
AlexNet	ReLU + GPU training
VGG	Deep small filters
GoogLeNet	Inception modules
DenseNet	Dense connectivity

51. Parameter Efficiency Comparison

Architecture	Parameter Count
AlexNet	High
VGG	Very High
GoogLeNet	Moderate
DenseNet	Efficient

PART 8 — End-to-End Defect Detection Model

52. What Is Defect Detection?

Defect detection identifies: cracks, scratches, dents, manufacturing faults using computer vision.

53. Real-World Applications

Industry	Example
Automotive	Surface defects
Semiconductor	Wafer inspection
Manufacturing	Crack detection
Steel	Rust/scratch analysis

54. End-to-End CNN Workflow

```
Defect Images
    ↓
Preprocessing
    ↓
CNN Feature Extraction
    ↓
Classification Layer
    ↓
Defect / No Defect
```

55. Step-by-Step Workflow

Step 1 — Collect Images

Examples: damaged parts, normal parts.

Step 2 — Label Data

Classes: defect, non-defect.

Step 3 — Preprocess Images

Tasks: resize, normalize, augment.

Step 4 — Train CNN

Common choices: ResNet, DenseNet, EfficientNet.

Step 5 — Evaluate

Metrics: accuracy, precision, recall.

56. Why CNNs Are Good for Defect Detection

CNNs automatically learn textures, irregularities, local anomalies without manual feature engineering.

57. Common Challenges in Defect Detection

Challenge	Problem
Small defects	Hard to detect
Limited data	Overfitting
Lighting variation	Poor generalization

58. Solutions

Problem	Solution
Limited data	Data augmentation
Small defects	Higher resolution
Imbalanced dataset	Weighted loss

59. Most Asked Interview Question

Q: Why are CNNs effective for defect detection?

Answer: Because CNNs automatically learn local spatial patterns and visual anomalies from images.

60. Best Practices

Use Batch Normalization

Improves: stability, convergence.

Use Data Augmentation

Examples: rotation, flipping, brightness adjustment.

Use Transfer Learning

Pretrained CNNs: reduce training time, improve performance.

Monitor Validation Metrics

Helps detect overfitting.

61. Common Beginner Mistakes

Mistake	Problem
Very deep networks initially	Hard debugging
No normalization	Unstable training
Ignoring class imbalance	Biased model
Large learning rate	Divergence

62. Quick Revision Sheet

Concept	Key Idea
Vanishing Gradient	Tiny gradients
Exploding Gradient	Huge gradients
BatchNorm	Stabilizes activations
Dropout	Prevents overfitting
AlexNet	First major deep CNN
VGG	Deep small filters
GoogLeNet	Multi-scale convolutions
DenseNet	Dense feature reuse

63. Final End-to-End Flow

```
Input Images
    ↓
CNN Layers
    ↓
Feature Extraction
    ↓
Normalization
    ↓
Pooling
    ↓
Dense Layers
    ↓
Prediction
    ↓
Defect Classification
```

PART 9 — Transfer Learning

64. What Is Transfer Learning?

Transfer learning means: **using a model pretrained on a large dataset and adapting it to a new (often smaller) dataset**.

65. Why Transfer Learning Works

Deep CNNs learn hierarchical features:

- Early layers: edges, textures (universal)
- Middle layers: shapes, patterns
- Deep layers: task-specific features

Early/middle features transfer well across tasks.

66. Transfer Learning Workflow

```
Large Dataset (e.g., ImageNet)
      ↓
Pretrain CNN (AlexNet, VGG, ResNet, etc.)
      ↓
Remove final classification layer
      ↓
Add new task-specific layer(s)
      ↓
Fine-tune on target dataset
      ↓
Prediction on new task
```

67. Two Approaches

Approach	What Changes?	When to Use?
Feature Extraction	Freeze all layers, train only new head	Very small target dataset
Fine-Tuning	Unfreeze some/all layers, train end-to-end	Medium target dataset

68. When Transfer Learning Helps Most

Scenario	Benefit
Small target dataset	Avoids overfitting
Similar domain	Features transfer directly
Limited compute	No need to train from scratch
Fast prototyping	Quick results with pretrained model

69. Transfer Learning Best Practices

- Use lower learning rate for fine-tuning (pretrained weights are already good)
- Freeze early layers if target dataset is small
- Unfreeze more layers if target dataset is large
- Use data augmentation to compensate for small datasets

PART 10 — EfficientNet

70. What Is EfficientNet?

EfficientNet is a CNN architecture that **systematically scales network width, depth, and resolution together using a compound scaling method**.

71. The Scaling Problem

Scaling Type	What Changes	Problem
Depth	More layers	Vanishing gradients
Width	More channels	Diminishing returns
Resolution	Larger input image	More compute

Scaling one dimension alone gives limited improvement.

72. Compound Scaling (Key Idea)

EfficientNet scales ALL three dimensions simultaneously:

> $$\text{depth: } d = \alpha^\phi \quad \text{width: } w = \beta^\phi \quad \text{resolution: } r = \gamma^\phi$$
>
> Subject to: $\alpha \cdot \beta^2 \cdot \gamma^2 \approx 2$
>
> α = depth scaling coefficient | β = width scaling coefficient | γ = resolution scaling coefficient | ϕ = compound coefficient (user-specified, controls total compute) | Constraint ensures compute roughly doubles when ϕ increases by 1

73. EfficientNet Family

Model	ϕ	Parameters	Top-1 Accuracy
EfficientNet-B0	0	5.3M	77.1%
EfficientNet-B3	3	12M	81.6%
EfficientNet-B7	7	66M	84.3%

74. Why EfficientNet Matters

Advantage	Explanation
Parameter efficient	Better accuracy per parameter
Systematic design	No manual architecture search
Scalable	Easy to adjust compute budget
State-of-the-art baseline	Strong results across vision tasks

75. Updated Architecture Comparison

Architecture	Year	Key Innovation	Params
AlexNet	2012	ReLU + GPU + Dropout	60M
VGG	2014	Deep 3×3 filters	138M
GoogLeNet	2014	Inception (multi-scale)	6.8M
ResNet	2015	Skip connections	25M
DenseNet	2017	Dense connectivity	8M
EfficientNet	2019	Compound scaling	5–66M

Regularization: Dropout & Batch Normalization

76. Dropout

During training, randomly set activations to zero with probability p. Forces network to learn redundant representations.

> **Dropout (Training):**
>
> $$h_i^{(\text{drop})} = \begin{cases} 0 & \text{with probability } p \\ h_i/(1-p) & \text{with probability } 1-p \end{cases}$$
>
> p = dropout rate (typically 0.2–0.5) | h_i = activation of neuron i | Division by $(1-p)$ = **inverted dropout** (ensures expected value unchanged) | At test time: NO dropout, use all neurons

Why Inverted Dropout?

Without scaling: expected training activation = $(1-p) \cdot h_i$. At test time: full h_i. This creates a train-test mismatch. Inverted dropout divides by $(1-p)$ during training so expectations match:

$$\mathbb{E}[\text{train}] = (1-p) \cdot \frac{h_i}{1-p} = h_i = \mathbb{E}[\text{test}]$$

Dropout as Ensemble:

Each training step uses a different random subnetwork. N neurons → 2^N possible subnetworks. At test time, using all neurons approximates ensemble averaging over all 2^N networks.

77. Batch Normalization (BN)

Normalize activations across the **batch dimension** within each layer during training.

Batch Normalization Formula:

$$\mu_B = \frac{1}{m}\sum_{i=1}^{m} x_i \qquad \sigma_B^2 = \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_B)^2$$

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

$$y_i = \gamma \hat{x}_i + \beta$$

μ_B = batch mean | σ_B^2 = batch variance | m = batch size | ϵ = small constant for numerical stability (1e-5) | $\hat{x}_i$ = normalized activation | γ, β = learnable scale and shift (restore representational power) | Applied per-channel in CNNs

Why BN Works:

Benefit	Explanation
Reduces internal covariate shift	Each layer sees stable input distribution
Allows higher learning rates	Gradients don't explode/vanish as easily
Slight regularization	Mini-batch noise acts as regularizer
Faster convergence	Smoother loss landscape

78. Batch Norm vs Layer Norm

Aspect	Batch Normalization	Layer Normalization
Normalize over	Batch dimension (across samples)	Feature dimension (within each sample)
Depends on batch size?	Yes (fails with very small batches)	No (independent of batch)
Used in	CNNs (image tasks)	Transformers (Topic 9), RNNs (Topic 7) — sequence tasks
At inference	Uses running mean/var from training	Same as training
Why not BN in Transformers?	Variable sequence lengths + small batches make batch statistics unreliable	

Attention Mechanisms in CNNs

79. Spatial Attention

Learn **WHERE** to focus in a feature map. Produces a 2D weight map highlighting important spatial regions.

Spatial Attention (CBAM):

$$M_s(F) = \sigma\left(f^{7\times7}([\text{AvgPool}(F);\ \text{MaxPool}(F)])\right)$$

$F \in \mathbb{R}^{C\times H\times W}$ = input feature map | AvgPool/MaxPool along channel axis → $\mathbb{R}^{1\times H\times W}$ each | Concatenate → $\mathbb{R}^{2\times H\times W}$ | $f^{7\times7}$ = 7×7 convolution → $\mathbb{R}^{1\times H\times W}$ | σ = sigmoid → weights in [0,1] per spatial position

Apply:

$$F' = M_s(F) \otimes F$$

$\otimes$ = element-wise multiplication (broadcasting over channels) | Highlights important regions, suppresses background

80. Channel Attention (SE-Net)

Learn **WHICH channels** are important. Produces a weight per channel.

> **Squeeze-and-Excitation (SE) Block:**
>
> $$z_c = \text{GlobalAvgPool}(F_c) = \frac{1}{H \times W} \sum_{i,j} F_c(i,j)$$
>
> $$s = \sigma(W_2 \cdot \text{ReLU}(W_1 \cdot z))$$
>
> $$\tilde{F}_c = s_c \cdot F_c$$
>
> $z \in \mathbb{R}^C$ = channel descriptor (squeeze) | $W_1 \in \mathbb{R}^{C/r \times C}$ = reduction layer ($r = 16$ typical) | $W_2 \in \mathbb{R}^{C \times C/r}$ = expansion layer | $s \in \mathbb{R}^C$ = channel attention weights | Each channel is re-weighted by its importance

81. CBAM (Convolutional Block Attention Module)

Combines BOTH channel and spatial attention sequentially:

> $$F' = M_c(F) \otimes F \quad \textbf{(channel attention first)}$$
>
> $$F'' = M_s(F') \otimes F' \quad \textbf{(then spatial attention)}$$
>
> M_c = channel attention map ($\mathbb{R}^{C \times 1 \times 1}$) | M_s = spatial attention map ($\mathbb{R}^{1 \times H \times W}$) | Sequential application: first select WHAT (channels), then WHERE (locations)

82. Spectral Attention

Operates in the **frequency domain**. Transforms features to frequency space (using FFT/DCT), applies attention in that domain, then transforms back.

> **Spectral Attention Process:**
>
> $$\hat{F} = \text{FFT}(F) \quad \textbf{(spatial} \rightarrow \textbf{frequency)}$$
>
> $$\hat{F}' = A(\hat{F}) \odot \hat{F} \quad \textbf{(frequency-domain attention)}$$
>
> $$F' = \text{IFFT}(\hat{F}') \quad \textbf{(frequency} \rightarrow \textbf{spatial)}$$
>
> FFT = Fast Fourier Transform | $A(\hat{F})$ = learned attention weights in frequency domain | Low-frequency components = smooth structure, global patterns | High-frequency = edges, textures, fine details | Spectral attention can selectively enhance/suppress different frequency bands

83. Spatial vs Spectral Attention

Aspect	Spatial Attention	Spectral Attention
Domain	Spatial (pixel/location)	Frequency (Fourier/DCT)
What it selects	Important regions in image	Important frequency bands
Computation	Simple (convolution + sigmoid)	Requires FFT/IFFT
Use case	Object detection, segmentation	Texture recognition, denoising
Example	CBAM spatial module	FFC (Fast Fourier Convolution)

Transfer Learning in CNNs

84. Why Transfer Learning Works

CNN layers form a hierarchy: early layers learn UNIVERSAL features (edges, textures), deep layers learn TASK-SPECIFIC features (car parts, faces).

Layer Depth	Features Learned	Transferability
Conv1-2 (early)	Edges, gradients, colors	Highly transferable (universal)
Conv3-4 (middle)	Textures, parts, patterns	Moderately transferable
Conv5+ (deep)	Object-specific, task-specific	Low (must fine-tune)
FC layers	Task decisions	Not transferable (replace)

85. Transfer Learning Strategies

Strategy	When to Use	How
Feature extraction	Small target dataset, similar domain	Freeze all conv, train new FC head
Fine-tune last layers	Moderate target dataset	Freeze early layers, fine-tune last conv + FC
Full fine-tuning	Large target dataset, different domain	Fine-tune entire network with low LR

> EXAM: "Why freeze early layers and fine-tune later layers?" Because early layers learn generic features (edges, textures) that transfer across tasks and domains. Later layers learn task-specific features that need updating. Freezing early layers also prevents overfitting on small datasets and is computationally cheaper.

Advanced Questions & Answers

Q1. A ResNet block has input x with 256 channels. The block applies: Conv1×1 (256→64) → Conv3×3 (64→64) → Conv1×1 (64→256). Compute parameters saved compared to a direct Conv3×3 (256→256) block.

Bottleneck block parameters:

- Conv1×1 (256→64): 256×64×1×1 + 64 = 16,448
- Conv3×3 (64→64): 64×64×3×3 + 64 = 36,928
- Conv1×1 (64→256): 64×256×1×1 + 256 = 16,640
- Total: **70,016**

Direct Conv3×3 (256→256):

256×256×3×3 + 256 = **590,080**

Savings: 590,080 − 70,016 = 520,064 parameters (88% reduction!)

The bottleneck design reduces dimensionality, performs expensive spatial convolution in low-dimensional space, then projects back — achieving similar representational power with 8× fewer parameters.

Q2. In BatchNorm, given a mini-batch of 4 values [2, 4, 6, 8] at a single neuron, compute the normalized output. Then apply learned parameters γ=2, β=0.5.

Step 1: Compute mean: μ = (2+4+6+8)/4 = 5

Step 2: Compute variance: $\sigma^2 = [(2-5)^2 + (4-5)^2 + (6-5)^2 + (8-5)^2]/4 = [9+1+1+9]/4 = 5$

Step 3: Normalize (ε=1e-5):

- $\hat{x}_1 = (2-5)/\sqrt{5} = -3/2.236 = -1.342$
- $\hat{x}_2 = (4-5)/\sqrt{5} = -1/2.236 = -0.447$
- $\hat{x}_3 = (6-5)/\sqrt{5} = 1/2.236 = 0.447$
- $\hat{x}_4 = (8-5)/\sqrt{5} = 3/2.236 = 1.342$

Step 4: Scale and shift (γ=2, β=0.5):

- $y_1 = 2(-1.342) + 0.5 = -2.184$
- $y_2 = 2(-0.447) + 0.5 = -0.394$
- $y_3 = 2(0.447) + 0.5 = 1.394$
- $y_4 = 2(1.342) + 0.5 = 3.184$

Note: If γ=√5 and β=5, we recover the original values — showing BatchNorm can learn the identity if needed.

Q3. Explain the vanishing gradient problem in VGG-19 vs. ResNet-152. Mathematically show how skip connections solve it.

VGG-19 (no skip connections):

Gradient at layer l: $\frac{\partial L}{\partial x_l} = \prod_{i=l}^{L-1} \frac{\partial F_i}{\partial x_i}$

If each factor $|\partial F_i/\partial x_i| < 1$ (common with weight decay): For 19 layers: gradient $\propto 0.9^{18} \approx 0.15$ (significant shrinkage)

ResNet (with skip connections):

Each block: y = F(x) + x

Gradient: $\frac{\partial L}{\partial x_l} = \frac{\partial L}{\partial x_L} \cdot \prod_{i=l}^{L-1}(1 + \frac{\partial F_i}{\partial x_i})$

The "1+" ensures gradient **NEVER vanishes** — even if $\partial F_i/\partial x_i = 0$, the gradient still flows through the identity path.

For 152 layers: minimum gradient factor = 1 per block (not 0.9). This is why ResNet-152 trains while plain networks deeper than ~20 layers fail.

Q4. Transfer learning: Fine-tune ResNet-50 (pretrained on ImageNet, 1000 classes) for medical imaging with 5 classes and 500 training images. Describe optimal strategy with mathematical justification.

Strategy: Freeze early layers, replace and train final layers

1. Remove final FC layer (1000 outputs) → replace with FC(2048→5)

2. Freeze all layers except last residual block + new FC
3. Use small lr (1e-4) for unfrozen conv, larger (1e-3) for new FC
4. Apply heavy augmentation (rotation, flip, color jitter)

Mathematical justification:

- Full ResNet-50: ~25.6M params / 500 samples = 51,200 params/sample → massive overfitting
- Freezing to last block: ~2.1M trainable → 4,200 params/sample → manageable
- New FC only: 2048×5 + 5 = 10,245 params → 500/10,245 ≈ 0.05 (use regularization)
- With augmentation (5× effective data): 2,500/10,245 ≈ 0.24 (feasible with dropout)

Why early layers transfer: Early CNN layers learn universal features (edges, textures) that apply across domains. Medical-specific features emerge only in deeper layers.

Q5. EfficientNet uses compound scaling: depth α^φ, width β^φ, resolution γ^φ with constraint $\alpha \cdot \beta^2 \cdot \gamma^2 \approx 2$. If $\alpha=1.2$, $\beta=1.1$, $\gamma=1.15$, verify the constraint. Then compute scaled dimensions from B0 (depth=18, width=256, resolution=224) for $\varphi=2$.

Verify constraint:

$\alpha \cdot \beta^2 \cdot \gamma^2 = 1.2 \times 1.1^2 \times 1.15^2 = 1.2 \times 1.21 \times 1.3225 = 1.919 \approx 2$ ✓

Scaling for $\varphi=2$:

- Depth: $\alpha^2 = 1.2^2 = 1.44 \rightarrow 18 \times 1.44 \approx$ **26 layers**
- Width: $\beta^2 = 1.1^2 = 1.21 \rightarrow 256 \times 1.21 \approx$ **310 channels**
- Resolution: $\gamma^2 = 1.15^2 = 1.3225 \rightarrow 224 \times 1.3225 \approx$ **296×296**

FLOP increase:

FLOPS $\propto$ depth $\times$ width^2 $\times$ resolution^2 $= (\alpha \cdot \beta^2 \cdot \gamma^2)^\varphi = 2^\varphi = 2^2 =$ **4× more compute**

Elegance: doubling compute ($\varphi+1$) always gives systematic improvement across all three dimensions.

Q6. (End-Term) Explain spatial attention in CNNs. How does it differ from the self-attention in Transformers?

Spatial Attention in CNNs (CBAM-style):

- Pools features across channels → 2D attention map per spatial location
- Uses convolution (typically 7×7) on pooled features → sigmoid weight map
- Applied element-wise to feature map
- Complexity: $O(H \times W)$ — linear in spatial size

Self-Attention in Transformers (a different architecture — fully covered in Topic 9; briefly compared here)**:**

- Every position attends to every other position via Q, K, V (query, key, value projections)
- Complexity: $O(N^2)$ — quadratic in sequence length
- Learned query/key/value projections

Key difference: CNN spatial attention is LOCAL and EFFICIENT (one conv layer), while Transformer self-attention is GLOBAL and EXPENSIVE (quadratic). CNN attention answers "where to look" cheaply; Transformer attention models all pairwise relationships.

Q7. (End-Term) What is the difference between dropout and batch normalization? Can they be used together?

Dropout:

- **Purpose:** Regularization (prevent overfitting)
- **Mechanism:** Randomly zero activations during training
- **Effect:** Forces redundant representations, ensemble averaging
- **When:** Training only (disabled at test time)

Batch Normalization:

- **Purpose:** Training stabilization (not primarily regularization)
- **Mechanism:** Normalize activations to zero mean, unit variance
- **Effect:** Allows higher LR, smoother loss landscape
- **When:** Both training (batch stats) and test (running stats)

Can they be used together?

Yes, but with caution. Dropout AFTER BatchNorm can cause train-test variance mismatch because dropout changes the scale of activations that BN was calibrated on. Best practice: use BN in conv layers, dropout in FC layers. In modern architectures (ResNet+), often BN without dropout suffices.

Q8. (Mid-Term) Given a CNN encoder with Conv2d layers: (3→32, k=3, s=2, p=1), (32→64, k=3, s=2, p=1), (64→128, k=3, s=2, p=1). Input: (1, 3, 64, 64). Trace the output shape.

Using formula: $o = \lfloor (n + 2p - k)/s \rfloor + 1$

- **Layer 1:** $(64 + 2 - 3)/2 + 1 = 32$ → output: **(1, 32, 32, 32)**
- **Layer 2:** $(32 + 2 - 3)/2 + 1 = 16$ → output: **(1, 64, 16, 16)**
- **Layer 3:** $(16 + 2 - 3)/2 + 1 = 8$ → output: **(1, 128, 8, 8)**

Final output: (1, 128, 8, 8)

Total elements: $128 \times 8 \times 8 = 8192$

Created by **Kushal Ghosh** (gkushalg01@gmail.com) and **Bharat Das Vaishnav** (bharatvaishnav2025@gmail.com)

↑ Back to Top | **Autoencoders & Segmentation**

Topic 6 — Auto-Encoder and Image Segmentation

1. Introduction

Auto-encoders are neural networks designed to **learn efficient representations of data**. They are mainly used for: feature learning, compression, denoising, anomaly detection, image segmentation, generative learning.

2. Core Idea

An auto-encoder learns how to reconstruct its own input. Input: $\boldsymbol{x}$, Output: $\hat{\boldsymbol{x}}$, Goal: $\boldsymbol{x} \approx \hat{\boldsymbol{x}}$.

3. Learning Progression

```
Basic Auto-encoder → Encoder → Latent Representation → Decoder → Loss Function → Regularized Auto-encoders → Class-enc
```

PART 1 — Auto-Encoder

4. What Is an Auto-Encoder?

A neural network with two main parts: (1) Encoder, (2) Decoder.

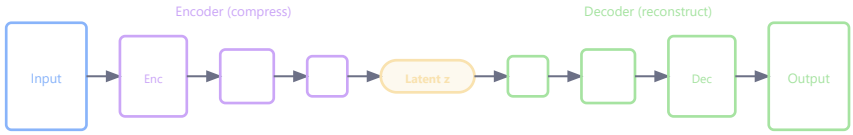
5. Encoder

Encoder compresses input into **latent representation** (compressed features).

6. Decoder

Decoder reconstructs original input from compressed representation.

7. Basic Architecture




```
Input Image → Encoder → Latent Space → Decoder → Reconstructed Image
```

8. Why Auto-Encoders Matter

Auto-encoders learn meaningful compressed representations without requiring labels. This is **unsupervised learning**.

9. Latent Space

Latent space is compressed feature representation. Contains essential information while removing redundancy/noise.

10. Compression Analogy

Think of ZIP file compression. Encoder compresses data, Decoder reconstructs data.

11. Encoder Formula

> $$h = f(Wx + b)$$
>
> x = input | W = weights | b = bias | h = latent representation | f = activation function

12. Decoder Formula

> $$\hat{x} = g(W'h + b')$$
>
> $\hat{x}$ = reconstructed input | W' = decoder weights | h = latent representation | g = activation function

13. Reconstruction Loss

Goal: minimize reconstruction error.

14. MSE Reconstruction Loss

> $$L = \frac{1}{N}\sum(x - \hat{x})^2$$

15. Numerical Example — Reconstruction Loss

Problem: Original: $x = [1, 2, 3]$, Reconstructed: $\hat{x} = [1, 1, 4]$. Find MSE loss.

$L = \frac{(1-1)^2+(2-1)^2+(3-4)^2}{3} = \frac{0+1+1}{3} = 0.67$

16. Bottleneck Layer

The middle compressed layer is called the **bottleneck**. Purpose: force network to learn efficient representations.

17. Why Bottleneck Is Important

Without compression, network may simply memorize input. Bottleneck forces feature learning.

18. Applications of Auto-Encoders

Application	Purpose
Compression	Reduce dimensions
Denoising	Remove noise
Anomaly detection	Detect abnormal data
Feature extraction	Learn representations
Segmentation	Pixel-level understanding

PART 2 — Regularized Auto-Encoders

19. Why Regularization Is Needed

Basic auto-encoders may memorize data or learn identity mapping. Regularization forces better representations.

20. Types of Regularized Auto-Encoders

Type	Main Idea
Sparse Auto-encoder	Few neurons active
Denoising Auto-encoder	Learn from noisy input
Variational Auto-encoder (VAE)	Probabilistic latent space

21. Sparse Auto-Encoder

Encourages only small number of neurons to activate.

22. Sparse Constraint

$$L_{total} = L_{reconstruction} + \lambda L_{sparsity}$$

λ = sparsity weight | $L_{sparsity}$ = KL divergence penalty on average neuron activations

23. Why Sparse Auto-Encoders Help

Benefits: better feature learning, reduced redundancy, more interpretable representations.

24. Denoising Auto-Encoder

Input: noisy image. Output: clean image.

25. Denoising Workflow

```
Noisy Image → Encoder → Latent Features → Decoder → Clean Reconstructed Image
```

26. Why Denoising Auto-Encoders Matter

They learn robust features and noise-resistant representations.

27. Variational Auto-Encoder (VAE)

VAE learns **probability distributions in latent space**. Used in: generative AI, image synthesis, diffusion model foundations.

28. VAE Key Idea

Instead of single latent point, learn latent distribution.

29. VAE Loss

VAE uses: (1) Reconstruction loss, (2) KL-divergence loss.

30. KL Divergence Formula

> $$D_{KL}(P||Q) = \sum P(x) \log \frac{P(x)}{Q(x)}$$

31. Most Asked Interview Question

Q: Why regularize auto-encoders?

Answer: Because regularization prevents trivial memorization and forces meaningful feature learning.

PART 3 — Class-Encoder

32. What Is a Class-Encoder?

A class-encoder learns **class-specific representations**. Instead of reconstructing same sample, it reconstructs another sample from same class.

33. Example

Input: image of digit 3. Target reconstruction: another image of digit 3.

34. Why Class-Encoders Matter

They learn intra-class similarity and class-aware representations. Useful for: classification, face recognition, metric learning.

35. Class-Encoder Workflow

```
Input Image → Encoder → Latent Features → Decoder → Another Sample of Same Class
```

36. Advantages of Class-Encoders

Advantage	Explanation
Better class separation	Improved representation
Robust features	Learns common patterns
Improved classification	Better embeddings

PART 4 — Image Segmentation

37. What Is Image Segmentation?

Image segmentation means **dividing image into meaningful regions**. Unlike classification, segmentation predicts every pixel.

38. Classification vs Segmentation

Classification	Segmentation
One label for image	Label for every pixel

39. Example

Input: medical scan. Output: tumor boundaries.

40. Types of Segmentation

Type	Meaning
Semantic Segmentation	Same-class pixels grouped
Instance Segmentation	Separate individual objects

Semantic Segmentation

Assigns a class label to every pixel. Answers: "What is this pixel?" All objects of same class treated as one combined region (does NOT distinguish Car 1 vs Car 2).

Application	Example
Self-driving cars	Road segmentation
Medical imaging	Tumor segmentation
Satellite imagery	Land classification

Output shape: For 256×256 image with 3 classes → $256 \times 256 \times 3$ (one probability per class per pixel).

Loss functions: Cross Entropy (pixel classification), Dice Loss (imbalanced regions), IoU Loss (overlap quality).

Instance Segmentation

Goes one step further. Answers: (1) What object class? (2) Which specific object? Each object gets separate mask and identity.

Semantic Segmentation	Instance Segmentation
Groups same-class objects	Separates individual objects
"Car pixels"	"Car 1", "Car 2"
Simpler	More complex

Task	Popular Models
Semantic Segmentation	U-Net, FCN, DeepLab
Instance Segmentation	Mask R-CNN

Most Asked Interview Question

Q: Difference between semantic and instance segmentation?

Semantic segmentation labels pixels by class only, while instance segmentation also distinguishes separate objects of the same class.

41. Why Segmentation Matters

Applications: medical imaging, autonomous driving, industrial inspection, satellite analysis.

42. Segmentation Architecture

Most segmentation models use Encoder-Decoder structure, very similar to auto-encoders.

43. Segmentation Workflow

```
Input Image → Encoder (Feature Extraction) → Latent Representation → Decoder (Upsampling) → Pixel-wise Prediction
```

44. U-Net Architecture

Most popular segmentation architecture. Especially important in medical imaging.

45. U-Net Key Idea

U-Net combines encoder features with decoder reconstruction using **skip connections**.

46. Skip Connections

Skip connections are one of the most important ideas in Deep Learning. Used in U-Net, ResNet (Topic 5), DenseNet (Topic 5), and Transformers (Topic 9).

Core Idea

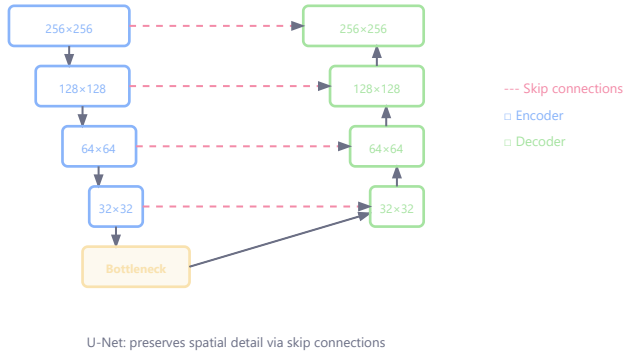
Instead of sending information only layer-by-layer, directly connect earlier layers to later layers. Information "skips" intermediate layers.

Why Skip Connections Are Needed

Deep networks suffer from vanishing gradients and information loss. Skip connections preserve information and improve gradient flow.

Skip Connections in U-Net

During encoding, spatial details get lost due to pooling. Skip connections send high-resolution encoder features directly to decoder.



U-Net: preserves spatial detail via skip connections

```
Encoder Layer → Pooling → Bottleneck → Decoder ← Skip Connection
```

Decoder receives detailed spatial information (edges, boundaries). Improves segmentation precision.

Example Intuition

Encoder detects exact tumor boundary. Pooling may blur this. Skip connection directly transfers boundary details to decoder → sharper segmentation mask.

Skip Connections in ResNet

> $$y = F(x) + x$$
>
> x = input | $F(x)$ = learned transformation | Allows gradients to flow directly backward → very deep networks become trainable

U-Net vs ResNet Skip Connections

U-Net	ResNet
Concatenate features	Add features
Preserve spatial detail	Improve gradient flow
Used in segmentation	Used in classification

Most Asked Interview Questions on Skip Connections

Q1: Why are skip connections important? → Preserve information and improve gradient flow.

Q2: Why useful in U-Net? → Recover fine spatial details lost during downsampling.

Q3: Why reduce vanishing gradients? → Gradients flow directly through shortcut paths.

Q4: Difference in U-Net vs ResNet? → U-Net concatenates, ResNet adds feature maps.

Final Intuition

Semantic Segmentation: "What class is this pixel?"

Instance Segmentation: "Which exact object does this pixel belong to?"

Skip Connections: "Send earlier information directly to deeper layers so important details are not lost."

47. U-Net Workflow

```
Encoder → Bottleneck → Decoder (with Skip Connections)
```

48. Segmentation Loss Functions

Loss Function	Usage
Cross Entropy	Pixel classification
Dice Loss	Overlapping regions
IoU Loss	Region matching

49. Dice Coefficient

Measures overlap between predicted mask and ground truth mask.

50. Dice Formula

$$Dice = \frac{2|A \cap B|}{|A| + |B|}$$

51. Dice Numerical

Problem: $|A| = 6$, $|B| = 8$, $|A \cap B| = 4$. Find Dice.

$Dice = \frac{2(4)}{6+8} = \frac{8}{14} = \mathbf{0.57}$

52. IoU (Intersection over Union)

$$IoU = \frac{|A \cap B|}{|A \cup B|}$$

54. Most Asked Interview Question

Q: Why are auto-encoders useful for segmentation?

Answer: Because encoder-decoder architectures efficiently learn compressed features and reconstruct spatial outputs for pixel-level prediction.

55. Real-World Segmentation Applications

Industry	Application
Healthcare	Tumor segmentation
Automotive	Lane detection
Manufacturing	Defect segmentation
Agriculture	Crop analysis

56. End-to-End Segmentation Pipeline

```
Input Images → Preprocessing → Encoder → Latent Features → Decoder → Segmentation Mask → Post-processing
```

57. Best Practices

Use Skip Connections

Preserve spatial information.

Normalize Images

Improves convergence and stability.

Use Data Augmentation

Examples: flipping, rotation, scaling.

Use Dice Loss for Imbalanced Segmentation

Especially when foreground is small.

Use Transfer Learning

Pretrained encoders improve performance and reduce training time.

58. Common Beginner Mistakes

Mistake	Problem
No bottleneck compression	Memorization
Ignoring imbalance	Poor segmentation
Very deep decoder	High computation
No augmentation	Overfitting

59. Most Asked Interview Questions

Q1: What is an auto-encoder? → Neural network that reconstructs input through compressed latent representations.

Q2: Why is bottleneck important? → Forces network to learn meaningful compressed features.

Q3: Difference CNN vs Auto-Encoder? → CNN: classification. Auto-Encoder: reconstruction.

Q4: What is image segmentation? → Pixel-level classification of images.

Q5: Why skip connections in U-Net? → Preserve fine spatial details lost during downsampling.

Q6: Why Dice Loss? → Handles class imbalance effectively in segmentation.

60. Auto-Encoder vs VAE

Auto-Encoder	VAE
Learns compressed features	Learns probability distributions
Deterministic	Probabilistic
Reconstruction	Generation + reconstruction

61. Quick Revision Sheet

Concept	Key Idea
Encoder	Compresses input
Decoder	Reconstructs input
Latent Space	Compressed representation
Bottleneck	Forces feature learning
Denoising AE	Removes noise
VAE	Generative latent space
Segmentation	Pixel-wise prediction
U-Net	Encoder-decoder segmentation

62. Final End-to-End Flow

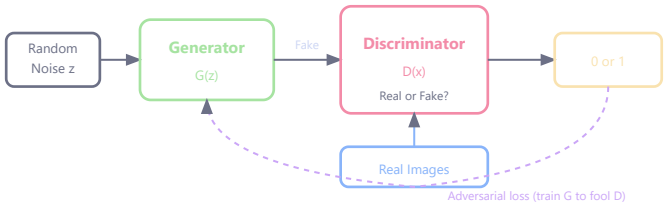
```
Input Image → Encoder → Latent Representation → Decoder → Reconstructed Image / Segmentation Mask → Loss → Backprop →
```

PART 5 — Generative Adversarial Networks (GANs)

63. What Is a GAN?

A GAN is a generative model consisting of two networks: **a Generator and a Discriminator competing against each other**.

64. GAN Architecture




```
Random Noise (z) → Generator (G) → Fake Image → Discriminator (D) ← Real Images → Real or Fake?
```

65. How GANs Work

- Generator tries to produce realistic images to fool the Discriminator
- Discriminator tries to distinguish real images from fakes
- Both improve over time through adversarial training

66. GAN Objective Function

> $$\min_G \max_D \; V(D,G) = \mathbb{E}_{x \sim p_{data}}[\log D(x)] + \mathbb{E}_{z \sim p_z}[\log(1 - D(G(z)))]$$
>
> G = generator | D = discriminator | x = real data from p_{data} | z = random noise from p_z (usually Gaussian) | $D(x)$ = probability that x is real | $G(z)$ = generated fake image | $D(G(z))$ = probability generated image is real

67. GAN Training Intuition

Component	Goal
Generator	Maximize $D(G(z))$ (fool D)
Discriminator	Maximize $D(x)$, minimize $D(G(z))$

Training alternates between updating D and G.

68. GAN Training Problems

Problem	Description
Mode collapse	Generator produces limited variety
Training instability	D and G oscillate, don't converge
Vanishing gradients	D becomes too strong, G gets no signal

69. GAN Applications

Application	Example
Image generation	Faces, artwork
Image-to-image	Sketch → Photo (Pix2Pix)

Super-resolution	Low-res → High-res (SRGAN)
Style transfer	Photo → Painting
Data augmentation	Generate synthetic training data

70. GAN vs Autoencoder vs VAE

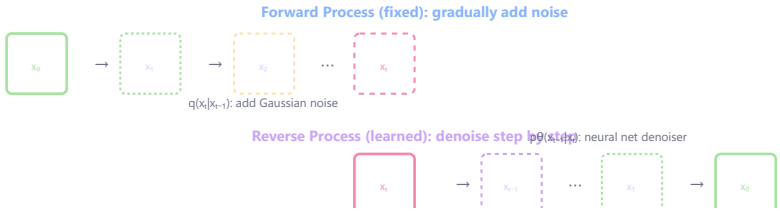
Model	Training Signal	Generates?	Quality
Autoencoder	Reconstruction loss	No (encodes)	N/A
VAE	Reconstruction + KL	Yes	Blurry
GAN	Adversarial loss	Yes	Sharp/realistic

PART 6 — Diffusion Models

71. What Are Diffusion Models?

Diffusion models generate data by **learning to reverse a gradual noise-adding process**. Backbone of DALL-E 2, Stable Diffusion, Midjourney.

72. Core Idea




```
Forward Process (Fixed): Clean Image → Add noise → ... → Pure Noise
Reverse Process (Learned): Pure Noise → Denoise → ... → Clean Image
```

73. Forward Diffusion Process

$$q(x_t|x_{t-1}) = \mathcal{N}(x_t;\ \sqrt{1-\beta_t}\,x_{t-1},\ \beta_t I)$$

x_t = noisy image at step t | β_t = noise schedule (0.0001 to 0.02) | After T steps, $x_T \approx$ pure Gaussian noise | Fixed process (not learned)

74. Reverse Process (Learned)

$$p_\theta(x_{t-1}|x_t) = \mathcal{N}(x_{t-1};\ \mu_\theta(x_t,t),\ \Sigma_\theta(x_t,t))$$

μ_θ = predicted mean (neural network output) | Σ_θ = predicted variance | Network learns to denoise one step at a time

75. Simplified Training Objective

$$L = \mathbb{E}_{t,x_0,\epsilon}\left[||\epsilon - \epsilon_\theta(x_t,t)||^2\right]$$

ϵ = actual noise added | $\epsilon_\theta(x_t,t)$ = noise predicted by neural network | x_t = noisy version of x_0 at step t | Model learns to predict what noise was added

76. Why Diffusion Models Work Well

Advantage	Explanation
Stable training	No adversarial instability
High quality	Very detailed images
Diversity	No mode collapse
Controllable	Easily guided with text/conditions

77. Diffusion vs GAN

Aspect	GAN	Diffusion Model
Training	Adversarial (unstable)	Noise prediction (stable)
Speed	Fast generation	Slow (many steps)
Quality	High	Very high
Diversity	Mode collapse risk	High diversity
Control	Harder	Easy (text prompts)

PART 7 — Panoptic Segmentation

78. What Is Panoptic Segmentation?

Combines **semantic + instance segmentation**. Every pixel gets a class label AND an instance ID.

79. Comparison of Segmentation Types

Type	Assigns Class?	Separates Instances?	Example
Semantic	Yes	No	All cars = one "car" mask
Instance	Yes	Yes (countable only)	Car 1, Car 2 separately
Panoptic	Yes	Yes (for things)	Car 1, Car 2 + road + sky

80. Things vs Stuff

Category	Definition	Examples	Segmentation
Things	Countable objects	Cars, people, dogs	Instance
Stuff	Amorphous regions	Sky, road, grass	Semantic

Panoptic handles BOTH.

81. Panoptic Quality (PQ) Metric

$$PQ = \frac{\sum_{(p,g)\in TP} IoU(p,g)}{|TP| + \frac{1}{2}|FP| + \frac{1}{2}|FN|}$$

TP = matched pairs with IoU > 0.5 | FP = predicted segments with no match | FN = ground truth segments with no match

PART 8 — Attention U-Net

82. What Is Attention U-Net?

Adds attention gates to standard U-Net. **Attention gates learn to focus on relevant spatial regions and suppress irrelevant features.**

83. Why Attention U-Net?

Standard U-Net skip connections pass ALL features (including noise). Attention gates filter skip connections — only pass relevant features.

84. Attention Gate Mechanism

```
Skip Connection (from encoder)
        ↓
    Attention Gate ← Gating signal (from decoder)
        ↓
Filtered Features (relevant only)
        ↓
Concatenate with decoder features
```

85. Attention Gate Formula

> $$\alpha_i = \sigma(W^T(\sigma_1(W_x x_i + W_g g_i + b)) + b_\alpha)$$
>
> α_i = attention coefficient (0 to 1) | x_i = skip feature at position i | g_i = gating signal from decoder | σ_1 = ReLU | σ = sigmoid | Output: $\hat{x}_i = \alpha_i \cdot x_i$

86. Attention U-Net vs Standard U-Net

Standard U-Net	Attention U-Net
Passes all skip features	Filters skip features
May include irrelevant regions	Focuses on relevant regions
Simpler	Slightly more complex
Good baseline	Better for noisy/complex data

PART 9 — Medical Image Segmentation

87. Why Medical Image Segmentation Matters

Aims to delineate anatomical structures or pathological regions. Applications: tumor boundary detection, organ segmentation, cell counting, surgical planning, disease progression tracking.

88. Common Medical Imaging Modalities

Modality	Full Form	Used For
CT	Computed Tomography	Organs, bones
MRI	Magnetic Resonance Imaging	Soft tissue, brain
X-ray	Radiography	Fractures, lungs

US	Ultrasound	Fetal, cardiac

89. Why Medical Segmentation Is Challenging

Challenge	Reason
Small datasets	Annotations require expert doctors
Class imbalance	Tumors are tiny vs background
Low contrast	Boundaries are subtle
3D volumes	CT/MRI are volumetric
Variability	Patient anatomy differs

90. Common Architectures for Medical Segmentation

Architecture	Key Feature
U-Net	Skip connections, works with few samples
Attention U-Net	Attention-filtered skip connections
V-Net	3D U-Net for volumetric data
nnU-Net	Self-configuring U-Net
TransUNet	Transformer (Topic 9) + U-Net hybrid

91. Dice Loss (Common in Medical Segmentation)

$$L_{Dice} = 1 - \frac{2\sum_i p_i g_i}{\sum_i p_i + \sum_i g_i}$$

p_i = predicted probability at pixel i | g_i = ground truth label (0 or 1) | Dice=1 means perfect overlap

92. Why Dice Loss for Medical Images?

Standard cross-entropy fails when foreground (tumor) is tiny. Dice loss directly optimizes overlap, handles class imbalance, focuses on small target region.

93. Medical Segmentation Pipeline

```
Medical Scan (CT/MRI) → Preprocessing (normalize, crop, augment) → U-Net / Attention U-Net → Predicted Mask → Post-pro
```

Advanced Questions & Answers

Q1. An autoencoder has architecture [784, 128, 32, 128, 784]. Input is 28×28 flattened. Compute compression ratio. If MSE is 0.02 per pixel, what is total reconstruction error?

Compression ratio: 784/32 = **5:1**

Each latent dimension encodes ~24.5 original dimensions.

Total reconstruction error: 0.02 × 784 = **68**

RMSE per pixel = √0.02 = 0.141 (~14.1% error on [0,1] scale)

Q2. In a VAE, given μ = [0.5, −0.3] and σ^2 = [0.8, 1.2], compute the KL divergence term.

$$D_{KL} = -\frac{1}{2}\sum_{j=1}^{d}(1 + \log(\sigma_j^2) - \mu_j^2 - \sigma_j^2)$$

Dim 1: $-0.5(1 + \log(0.8) - 0.25 - 0.8) = -0.5(1 - 0.223 - 0.25 - 0.8) = -0.5(-0.273) = 0.137$

Dim 2: $-0.5(1 + \log(1.2) - 0.09 - 1.2) = -0.5(1 + 0.182 - 0.09 - 1.2) = -0.5(-0.108) = 0.054$

Total KL = 0.137 + 0.054 = 0.191

Q3. Compute Dice coefficient and Dice loss for 4×4 binary masks:

```
Ground truth G:        Prediction P:
0 0 1 1                0 0 1 1
0 1 1 1                0 0 1 1
0 1 1 0                0 1 1 1
0 0 0 0                0 0 1 0
```

|P ∩ G| (both=1): positions (0,2),(0,3),(1,2),(1,3),(2,1),(2,2) = **6**

|P| = 8, |G| = 7

Dice = 2(6)/(8+7) = 12/15 = **80**

Dice Loss = 1 − 0.80 = **20**

IoU = 6/(8+7−6) = 6/9 = 0.667. Note: Dice ≥ IoU always.

Q4. GAN training produces these discriminator outputs. Analyze whether mode collapse or training instability is occurring.

Iter	D(real)	D(G(z))	G loss
1	0.9	0.1	2.3
2	0.85	0.3	1.2
3	0.6	0.6	0.7
4	0.9	0.1	2.3
5	0.6	0.6	0.7

Diagnosis: Training oscillation (instability), NOT mode collapse

D(real) oscillates 0.9↔0.6, D(G(z)) oscillates 0.1↔0.6, G loss cycles 2.3→0.7→2.3

Classic GAN "see-saw": D gets strong → G penalized → G adapts → D confused → D re-adapts → repeat

Not mode collapse (D(G(z)) does change — mode collapse shows constant G output).

Solutions: Spectral normalization, Wasserstein loss (WGAN), two-timescale learning rates.

Q5. In U-Net, explain why skip connections are critical. Compute feature map sizes for 256×256 input through 4 downsampling stages and back up.

Encoder (↓ max-pool): 256×256×1 → 128×128×64 → 64×64×128 → 32×32×256 → 16×16×512 (bottleneck)

Decoder (↑ up-conv):

- Up 1: 32×32×256, skip-concat → 32×32×512
- Up 2: 64×64×128, skip-concat → 64×64×256
- Up 3: 128×128×64, skip-concat → 128×128×128
- Up 4: 256×256×32, skip-concat → 256×256×64
- Final 1×1 conv: 256×256×num_classes

Why critical: Without skip connections, decoder must reconstruct from 16×16 — a 256× spatial compression. For pixel-accurate segmentation (65,536 positions from 256 bottleneck locations), decoder must "hallucinate" 256× more detail. Skip connections directly provide high-resolution features, combining deep semantic "what" with shallow spatial "where".

Q6. Autoencoder Gradient Derivation (Quiz 1 Q3 Style)

An autoencoder combines word embeddings $x_1, x_2 \in \mathbb{R}^{D_x}$ into a parent vector:

Concatenate: $x = [x_1; x_2] \in \mathbb{R}^{2D_x}$

Parent: $p = \text{ReLU}(W_1 x + b_1)$, **where** $W_1 \in \mathbb{R}^{D_p \times 2D_x}$

Reconstruction: $x' = W_2 p + b_2$

Reconstruction loss: $J_1 = \frac{1}{2}\|x' - x\|^2$

Compute: (a) $\frac{\partial J_1}{\partial p}$, **(b)** $\frac{\partial J_1}{\partial h}$ **where** $h = W_1 x + b_1$, **(c)** $\frac{\partial J_1}{\partial W_1}$

(a) $\delta_1 = \frac{\partial J_1}{\partial p}$:

By chain rule: $\frac{\partial J_1}{\partial p} = \frac{\partial J_1}{\partial x'} \cdot \frac{\partial x'}{\partial p}$

- $\frac{\partial J_1}{\partial x'} = (x' - x)$
- $\frac{\partial x'}{\partial p} = W_2$

$$\boxed{\delta_1 = W_2^T (x' - x)}$$

(b) $\delta_2 = \frac{\partial J_1}{\partial h}$:

Since $p = \text{ReLU}(h)$:

$$\frac{\partial p}{\partial h} = \text{diag}(\mathbb{1}[h > 0])$$

$$\boxed{\delta_2 = \delta_1 \odot \mathbb{1}[h > 0]}$$

$\mathbb{1}[h > 0]$ = indicator vector (1 where $h_i > 0$, else 0) | $\odot$ = element-wise multiplication | ReLU passes gradient only for positive pre-activations

(c) $\frac{\partial J_1}{\partial W_1}$:

Since $h = W_1 x + b_1$:

$$\boxed{\frac{\partial J_1}{\partial W_1} = \delta_2 \cdot x^T}$$

$\delta_2 \in \mathbb{R}^{D_p \times 1}$, $x^T \in \mathbb{R}^{1 \times 2D_x}$ → result $\in \mathbb{R}^{D_p \times 2D_x}$ (same shape as W_1)

Created by **Kushal Ghosh** (gkushalg01@gmail.com) and **Bharat Das Vaishnav** (bharatvaishnav2025@gmail.com)

Topic 7 — RNN and Its Variants

This topic covers Recurrent Neural Networks and their advanced variants: LSTM, GRU, and Bi-LSTM. These architectures are designed for sequential data processing where order and temporal dependencies matter.

1. Introduction & Key Terms

Full Forms

Abbreviation	Full Form
RNN	Recurrent Neural Network
LSTM	Long Short-Term Memory
GRU	Gated Recurrent Unit
Bi-LSTM	Bi-Directional Long Short-Term Memory
BPTT	Backpropagation Through Time

2. Why Sequential Models?

In many tasks, order matters. Example:

```
"The movie was not good"
```

The meaning of *good* depends on *not*. Thus: order matters.

3. Sequential Learning Problem

Sequential learning means:

learning dependencies across time or ordered data.

4. Examples of Sequential Data

Domain	Sequence
NLP	Words in a sentence
Speech	Audio frames
Video	Image frames
Finance	Stock prices over time
Sensors	Time-series signals

5. Why FNNs Fail for Sequences

Feed-Forward Neural Networks:

- do not remember previous inputs
- process everything independently

Thus they cannot model temporal relationships or context dependencies.

PART 1 — RNN Fundamentals

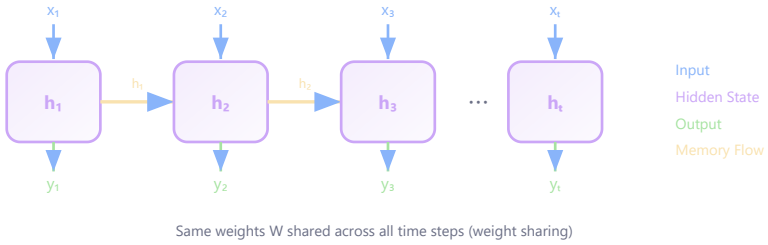
6. Main Idea of RNN

RNN introduces:

hidden memory state.

The hidden state stores past information and sequence context, enabling the network to "remember."

7. RNN Basic Architecture



Same weights W shared across all time steps (weight sharing)

8. Weight Sharing

RNNs reuse same weights at every time step. This enables sequence processing of variable length.

9. RNN Hidden State Formula

$$h_t = f(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$

x_t = current input at time step t | h_{t-1} = previous hidden state | W_{xh} = input-to-hidden weight matrix | W_{hh} = hidden-to-hidden (recurrent) weight matrix | b_h = hidden bias | f = activation function (typically tanh)

10. Output Formula

$$y_t = W_{hy}h_t + b_y$$

y_t = output at time step t | W_{hy} = hidden-to-output weight matrix | h_t = hidden state at time t | b_y = output bias

11. Why Hidden State Matters

Hidden state acts like **short-term memory**. It carries contextual information across sequence steps.

12. Types of RNN Tasks

Architecture	Output Type	Example
One-to-One	Single output	Image classification
One-to-Many	Sequence	Image captioning
Many-to-One	Single output	Sentiment analysis
Many-to-Many	Sequence	Machine translation

13. RNN Numerical Example

Problem: Given $x_t = 2$, $h_{t-1} = 1$, $W_{xh} = 0.5$, $W_{hh} = 0.8$, $b_h = 0$. Using identity activation $f(x) = x$. Find h_t.

Solution:

$$h_t = W_{xh}x_t + W_{hh}h_{t-1} = (0.5)(2) + (0.8)(1) = 1 + 0.8 = \mathbf{1.8}$$

PART 2 — Backpropagation Through Time (BPTT)

14. Why BPTT Is Needed

RNNs contain repeated time dependencies. Normal backpropagation is insufficient. Thus: Backpropagation Through Time (BPTT) is used — the RNN is unrolled through time and gradients are propagated backward across all time steps.

15. Vanishing Gradient Problem

Long-term dependencies become difficult because repeated multiplication of small gradients causes exponential decay:

$$\frac{\partial L_T}{\partial h_t} = \frac{\partial L_T}{\partial h_T} \prod_{k=t+1}^{T} \frac{\partial h_k}{\partial h_{k-1}}$$

If each factor < 1 (e.g. 0.9): $0.9^{50} \approx 0.005$ — only 0.5% of gradient survives 50 steps

16. Exploding Gradient

Repeated large multiplications: 10^{10} become extremely large, destabilizing training.

17. Gradient Clipping

$$g = \frac{g}{\|g\|} \times \text{threshold}$$

g = gradient vector | $\|g\|$ = L2 norm (magnitude) | threshold = maximum allowed gradient norm

Standard RNNs struggle with long-term dependencies. This motivated LSTM and GRU.

PART 3 — LSTM (Long Short-Term Memory)

18. Why LSTM Was Developed

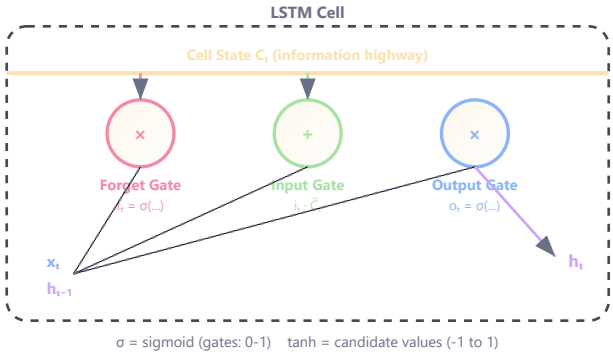
LSTM solves the vanishing gradient problem and enables learning of long-term dependencies through gated memory mechanisms.

19. Core Components of LSTM

Component	Purpose	Controls
Forget Gate (f_t)	Remove unnecessary memory	What to discard from cell state
Input Gate (i_t)	Add new information	What new info to store
Output Gate (o_t)	Generate output	What to expose as hidden state

Cell State (C_t)	Long-term memory highway	Carries information across time

20. LSTM Architecture



σ = sigmoid (gates: 0-1) tanh = candidate values (-1 to 1)

21. Forget Gate Formula

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

f_t = forget gate output (0–1) | σ = sigmoid | W_f = forget gate weights | $[h_{t-1}, x_t]$ = concatenated input | b_f = bias | Near 0 = forget, near 1 = keep

22. Input Gate Formula

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$

i_t = input gate output (controls what new info to store) | W_i = input gate weights | b_i = input gate bias

23. Cell Candidate Formula

$$\tilde{C}_t = \tanh(W_c[h_{t-1}, x_t] + b_c)$$

$\tilde{C}_t$ = candidate cell state (proposed new information) | W_c = candidate weights | tanh outputs between -1 and 1

24. Cell State Update

$$C_t = f_t \cdot C_{t-1} + i_t \cdot \tilde{C}_t$$

C_t = new cell state | $f_t \cdot C_{t-1}$ = old memory scaled by forget gate | $i_t \cdot \tilde{C}_t$ = new info scaled by input gate

25. Output Gate & Hidden State

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

$$h_t = o_t \cdot \tanh(C_t)$$

o_t = output gate (controls what to expose) | h_t = hidden state output | $\tanh(C_t)$ = cell state squashed to [-1, 1]

LSTM preserves long-term information via the cell state "highway" — gradients flow without vanishing because $\partial C_t / \partial C_{t-1} = f_t \approx 1$.

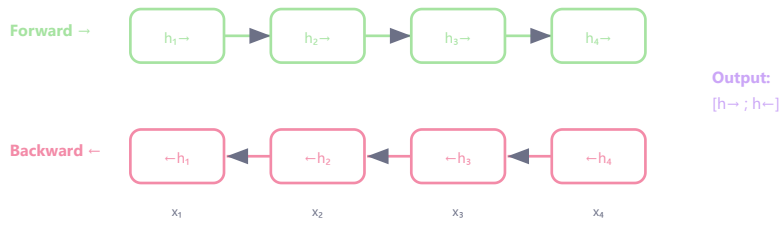
PART 4 — Bi-Directional RNN & Bi-LSTM

26. Problem with Standard RNN

Standard RNN sees only past context. But many tasks need future context too.

Example: "I went to the *bank*" — meaning depends on upcoming words (river? financial?).

27. Bi-RNN Architecture



28. Bi-LSTM

Bi-LSTM combines LSTM and bidirectional processing. Extremely powerful for:

- NLP (named entity recognition, POS tagging)
- Speech recognition
- Machine translation

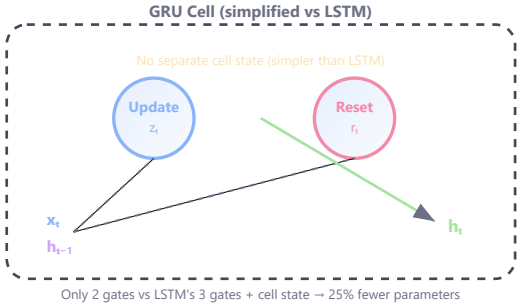
Output dimension = $2 \times \mathbf{hidden\_size}$ (forward + backward concatenated).

PART 5 — GRU (Gated Recurrent Unit)

29. Why GRU Was Developed

LSTM is powerful but computationally heavy. GRU simplifies LSTM by combining forget and input gates into a single **update gate**.

30. GRU Architecture



31. Update Gate Formula

$$z_t = \sigma(W_z[h_{t-1}, x_t])$$

z_t = update gate (controls how much old state to keep vs replace) | W_z = update gate weight matrix

32. Reset Gate Formula

$$r_t = \sigma(W_r[h_{t-1}, x_t])$$

r_t = reset gate (controls how much past to forget when computing candidate) | W_r = reset gate weights

33. GRU Hidden State Update

$$h_t = (1 - z_t)h_{t-1} + z_t\tilde{h}_t$$

$(1 - z_t)h_{t-1}$ = old state retained | $z_t\tilde{h}_t$ = new candidate added | $\tilde{h}_t = \tanh(W[r_t \odot h_{t-1}, x_t])$

34. LSTM vs GRU Comparison

Feature	LSTM	GRU
Gates	3 (forget, input, output)	2 (update, reset)
Cell State	Separate cell state	No separate cell state
Parameters	More (4 weight matrices)	Fewer (3 weight matrices)
Training Speed	Slower	~25% faster
Best For	Complex, long sequences	Shorter sequences, efficiency

PART 6 — Applications

35. Key Applications

Application	Model	Workflow
Sentiment Analysis	LSTM / Bi-LSTM	Text → Hidden → Positive/Negative
Machine Translation	Seq2Seq (Encoder-Decoder)	English → Context → French
Image Captioning	CNN + LSTM	Image → Features → Sentence
Video Analysis	RNN/LSTM	Frames → Temporal → Action Label
Speech Recognition	Bi-LSTM	Audio → Hidden → Text

PART 7 — Best Practices & Common Mistakes

36. Best Practices

Practice	Why
Use LSTM/GRU over vanilla RNN	Better long-term memory
Apply gradient clipping	Prevents exploding gradients
Normalize inputs	Stable training
Use embeddings for text	Dense semantic representation
Use Bi-LSTM for contextual tasks	Past + future context

37. Common Mistakes

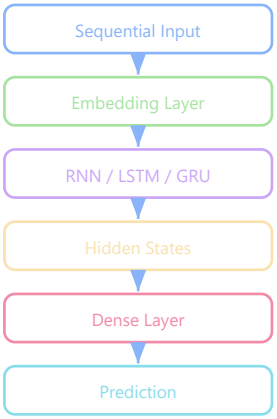
Mistake	Problem
Vanilla RNN for long sequences	Vanishing gradients, poor memory
Large learning rate	Exploding gradients
Ignoring sequence padding	Shape mismatch errors
No gradient clipping	Training instability

PART 8 — Quick Revision

38. Summary Table

Concept	Key Idea
RNN	Sequential memory via hidden state
BPTT	Backpropagation across time steps
Vanishing Gradient	Signal dies over long sequences
LSTM	Gated long-term memory (cell state)
Bi-LSTM	Forward + backward context
GRU	Simplified LSTM (fewer gates)

39. End-to-End Workflow



Advanced Questions & Answers

Q1. Full LSTM Step Computation

An LSTM cell receives input x_t = [0.5, 0.3], previous hidden state h_{t-1} = [0.1, -0.2], and previous cell state c_{t-1} = [0.8, 0.4]. Given simplified weight matrices (2D input, 2D hidden), compute one full LSTM step.

Weights (simplified, concatenating [h,x]):

- **W_f = [[0.5, 0.3, 0.2, 0.4], [0.1, 0.6, 0.3, 0.5]], b_f = [0.1, 0.2]**
- **W_i = [[0.4, 0.2, 0.3, 0.5], [0.3, 0.4, 0.2, 0.6]], b_i = [0.0, 0.1]**
- **W_c = [[0.6, 0.1, 0.4, 0.3], [0.2, 0.5, 0.1, 0.4]], b_c = [-0.1, 0.0]**
- **W_o = [[0.3, 0.4, 0.5, 0.2], [0.4, 0.3, 0.1, 0.6]], b_o = [0.1, 0.0]**

Concatenated input: $[h_{t-1}, x_t] = [0.1, -0.2, 0.5, 0.3]$

Forget gate: $f_t = \sigma(W_f \cdot [h, x] + b_f)$

- $f_t[0] = \sigma(0.05 - 0.06 + 0.10 + 0.12 + 0.1) = \sigma(0.31) = 0.577$
- $f_t[1] = \sigma(0.01 - 0.12 + 0.15 + 0.15 + 0.2) = \sigma(0.39) = 0.596$

Input gate: $i_t = \sigma(W_i \cdot [h, x] + b_i)$

- $i_t[0] = \sigma(0.04 - 0.04 + 0.15 + 0.15 + 0.0) = \sigma(0.30) = 0.574$
- $i_t[1] = \sigma(0.03 - 0.08 + 0.10 + 0.18 + 0.1) = \sigma(0.33) = 0.582$

Candidate: $\tilde{c}_t = \tanh(W_c \cdot [h, x] + b_c)$

- $\tilde{c}_t[0] = \tanh(0.06 - 0.02 + 0.20 + 0.09 - 0.1) = \tanh(0.23) = 0.225$
- $\tilde{c}_t[1] = \tanh(0.02 - 0.10 + 0.05 + 0.12 + 0.0) = \tanh(0.09) = 0.090$

New cell state: $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$

- $c_t[0] = 0.577 \times 0.8 + 0.574 \times 0.225 = 0.462 + 0.129 = \mathbf{0.591}$
- $c_t[1] = 0.596 \times 0.4 + 0.582 \times 0.090 = 0.238 + 0.052 = \mathbf{0.290}$

Output gate: $o_t = \sigma(W_o \cdot [h, x] + b_o)$

- $o_t[0] = \sigma(0.03 - 0.08 + 0.25 + 0.06 + 0.1) = \sigma(0.36) = 0.589$
- $o_t[1] = \sigma(0.04 - 0.06 + 0.05 + 0.18 + 0.0) = \sigma(0.21) = 0.552$

New hidden state: $h_t = o_t \odot \tanh(c_t)$

- $h_t[0] = 0.589 \times \tanh(0.591) = 0.589 \times 0.530 = \mathbf{0.312}$
- $h_t[1] = 0.552 \times \tanh(0.290) = 0.552 \times 0.282 = \mathbf{0.156}$

Q2. Vanishing Gradient Analysis

Explain the vanishing gradient problem in vanilla RNNs mathematically. For a sequence of length T=50, if the gradient factor per step is 0.9, what fraction of the gradient reaches the first timestep?

In BPTT, the gradient of loss at time T w.r.t. hidden state at time t:

$$\frac{\partial L_T}{\partial h_t} = \frac{\partial L_T}{\partial h_T} \prod_{k=t+1}^{T} \frac{\partial h_k}{\partial h_{k-1}}$$

Each factor: $\frac{\partial h_k}{\partial h_{k-1}} = W_{hh}^T \cdot \text{diag}(\sigma'(z_k))$

If spectral radius < 1 (say 0.9):

Gradient at t=1 relative to t=50: $0.9^{49} \approx 0.0057$

Only 0.57% of the gradient signal survives — 99.4% is lost.

LSTM solution: Cell state gradient flows through $c_t = f_t \odot c_{t-1} + \ldots$, so $\partial c_t / \partial c_{t-1} = f_t$. With $f_t \approx 1$ (trained to remember), gradient $\approx 1^T = 1$ regardless of sequence length.

Q3. LSTM vs GRU Parameter Comparison

Compare GRU and LSTM in terms of parameters for a layer with input size 512 and hidden size 256. Which is more efficient and by how much?

LSTM parameters:

4 gates (f, i, c̃, o), each with:

- W_{xh}: 512 × 256 = 131,072
- W_{hh}: 256 × 256 = 65,536
- bias: 256

Per gate: 196,864. Total LSTM: 4 × 196,864 = **787,456**

GRU parameters:

3 gates (r, z, h̃): Total GRU: 3 × 196,864 = **590,592**

Savings: 196,864 parameters (25% fewer)

Q4. Bi-LSTM Concatenation

A Bi-LSTM for sentiment analysis processes "The movie was not good at all" (7 tokens). Hidden size = 3. Show how forward and backward hidden states are concatenated for "not" (position 4).

Forward LSTM (left to right): processes "The", "movie", "was", "not", ...

$\vec{h}_4$ captures context from "The movie was not" = [0.2, -0.1, 0.5]

Backward LSTM (right to left): processes "all", "at", "good", "not", ...

$\overleftarrow{h}_4$ captures context from "all at good not" = [-0.3, 0.4, 0.1]

Concatenated representation for "not":

$$h_4 = [\vec{h}_4; \overleftarrow{h}_4] = [0.2, -0.1, 0.5, -0.3, 0.4, 0.1] \quad (\text{dim} = 2 \times 3 = 6)$$

Why this matters: Forward state knows "The movie was" → sets up expectation. Backward state knows "good at all" → knows what's being negated. Combined: the model understands "not" negates "good."

Q5. GRU Character-Level Language Model

Design a GRU-based language model for character-level prediction. Input: "hell" → predict "o". Vocab=26, hidden=64, seq_len=4. Compute total parameters and describe full forward pass.

Architecture & Parameters:

- Embedding: 26 × 64 = 1,664
- GRU layer: 3 gates × (64×64 + 64×64 + 64) = 3 × 8,256 = **24,768**
- Output FC: 64 × 26 + 26 = **1,690**
- **Total: 28,122 parameters**

Forward pass:

1. **Embed:** "h"→e_1, "e"→e_2, "l"→e_3, "l"→e_4 (each 64-dim)
2. **GRU step 1** (input e_1, h_0=zeros): $z_1 = \sigma(W_z[h_0, e_1])$, $r_1 = \sigma(W_r[h_0, e_1])$, $\tilde{h}_1 = \tanh(W_h[r_1 \odot h_0, e_1])$, $h_1 = (1 - z_1) \odot h_0 + z_1 \odot \tilde{h}_1$
3. **GRU steps 2-4:** Same process for "e", "l", "l"
4. **Output:** logits = $W_{out} \cdot h_4 + b_{out}$ → 26-dim
5. **Prediction:** $\text{softmax}(\text{logits})$ → argmax gives index 14 = "o"
6. **Loss:** $L = -\log(p[\text{"o"}])$

Q5. Why Using Only h_T Fails (Mid-Term Q7)

Why might using only the final hidden state h_T fail in sentiment classification using RNNs?

Key reasons:

1. **Vanishing gradients:** For long sequences, information from early tokens (which may contain key sentiment words) fades in h_T due to repeated multiplication through the recurrence.
2. **Information compression:** h_T is a fixed-size vector that must compress the ENTIRE sequence. For long reviews (500+ words), critical sentiment signals can be lost.
3. **Recency bias:** The final hidden state is disproportionately influenced by the last few tokens, which may be irrelevant to sentiment.

Better alternatives:

- **Average pooling:** $h_{avg} = \frac{1}{T}\sum_{t=1}^{T} h_t$ — preserves all timesteps equally
- **Max pooling:** $h_{max} = \max_t h_t$ — captures strongest activations
- **Attention over hidden states:** $h_{attn} = \sum_t \alpha_t h_t$ — learns which timesteps matter
- **Bidirectional RNN:** Combine forward $\overrightarrow{h_T}$ and backward $\overleftarrow{h_1}$ for full context

Q6. RNN True/False (Mid-Term MSQ)

Which statements about RNNs are CORRECT?

(a) The weights are different at each time step after unrolling

(b) h_t depends on both x_t and h_{t-1}

(c) BPTT treats the unrolled RNN as a deep feedforward network

(d) RNNs inherently solve long-term dependencies without modification

(e) In many-to-many RNNs, an output is produced at every time step

---

Correct: (b), (c), (e)

- (a) **FALSE** — weights are SHARED across all time steps (weight tying). This is a key property of RNNs.
- (b) **TRUE** — $h_t = f(W_{hh}h_{t-1} + W_{xh}x_t + b)$. Hidden state depends on both.
- (c) **TRUE** — unrolling creates a deep graph where backpropagation applies standard chain rule (hence "Backpropagation Through Time").
- (d) **FALSE** — vanilla RNNs suffer from vanishing gradients and CANNOT maintain long-term dependencies. LSTMs/GRUs are needed.
- (e) **TRUE** — many-to-many RNNs (e.g., POS tagging, language modeling) produce output at every timestep.

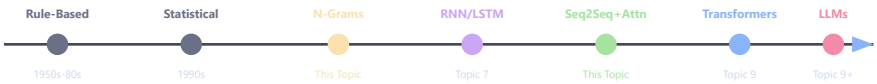
Created by **Kushal Ghosh** (gkushalg01@gmail.com) and **Bharat Das Vaishnav** (bharatvaishnav2025@gmail.com)

↑ Back to Top | **Traditional NLP & Sequence Models**

Topic 8 — Traditional NLP & Sequence Models

This topic covers the foundations of NLP before Transformers: N-gram language models, smoothing, perplexity, Seq2Seq, and the Attention mechanism that revolutionized sequence-to-sequence tasks.

1. Introduction & NLP Evolution



2. What Is a Language Model?

A Language Model (LM) predicts the probability of a sequence of words:

```
"The cat sat on the ____"
```

A good LM predicts: mat, floor, chair — with high probability.

3. Main Goal

Given previous words $w_1, w_2, \ldots, w_{n-1}$, predict w_n. This is **next-token prediction** — also the foundation of GPT and modern LLMs (large language models — covered fully in Topic 9).

PART 1 — N-Gram Language Models

4. What Is an N-Gram?

An N-gram is a sequence of N consecutive words/tokens.

N-Gram Type	Examples from "I love deep learning"
Unigram (N=1)	I, love, deep, learning
Bigram (N=2)	I love, love deep, deep learning
Trigram (N=3)	I love deep, love deep learning

5. Markov Assumption

Future word depends only on a limited number of previous words.

> **Bigram:**
>
> $$P(w_i|w_1, w_2, \ldots, w_{i-1}) \approx P(w_i|w_{i-1})$$
>
> **Trigram:**
>
> $$P(w_i|w_1, w_2, \ldots, w_{i-1}) \approx P(w_i|w_{i-2}, w_{i-1})$$
>
> w_i = current word | w_{i-1} = preceding word | Simplifies computation enormously

6. Chain Rule of Probability

$$P(w_1, w_2, \ldots, w_n) = \prod_{i=1}^{n} P(w_i | w_1, \ldots, w_{i-1})$$

$P(w_1, \ldots, w_n)$ = joint probability of entire sentence | $\prod$ = product over all positions

7. Bigram Probability Formula

$$P(w_i | w_{i-1}) = \frac{C(w_{i-1}, w_i)}{C(w_{i-1})}$$

$C(w_{i-1}, w_i)$ = count of bigram in corpus | $C(w_{i-1})$ = count of preceding word

8. Numerical Example — Bigram

Problem: Corpus: "I am happy / I am learning / I am coding". Find $P(am|I)$.

Solution: Count(I am) = 3, Count(I) = 3

$$P(am|I) = \frac{3}{3} = 1.0$$

9. Problems with N-Grams

Problem	Explanation
Data Sparsity	Unseen bigrams get P=0
Long-Distance Failure	Cannot connect "movie" with "amazing" over 10 words
Huge Memory	Bigram table: V^2, Trigram: V^3

PART 2 — Smoothing Techniques

10. Why Smoothing Is Needed

Without smoothing, unseen sequences get probability 0, breaking sentence probability estimation entirely.

Redistribute small probability mass to unseen events.

11. Laplace (Add-One) Smoothing

$$P(w_i | w_{i-1}) = \frac{C(w_{i-1}, w_i) + 1}{C(w_{i-1}) + V}$$

$+1$ = smoothing constant (numerator) | $+V$ = normalization (one per vocab word) | V = vocabulary size

12. Numerical Example — Smoothing

Problem: Count(I love) = 5, Count(I) = 10, V = 100. Find smoothed $P(love|I)$.

$$P(love|I) = \frac{5+1}{10+100} = \frac{6}{110} = \mathbf{0.0545}$$

13. Interpolation

$$\hat{P} = \lambda_1 P_{trigram} + \lambda_2 P_{bigram} + \lambda_3 P_{unigram}$$

$\lambda_1 + \lambda_2 + \lambda_3 = 1$ | Combines multiple N-gram levels for robustness

14. Katz Backoff

Uses higher-order N-grams when available, backs off to lower order when unseen. Redistributes probability mass from seen to unseen events.

Katz Backoff Formula:

$$P_{BO}(w_i|w_{i-1}) = \begin{cases} P^*(w_i|w_{i-1}) & \text{if } C(w_{i-1}, w_i) > 0 \\ \alpha(w_{i-1}) \cdot P(w_i) & \text{otherwise} \end{cases}$$

$P^*(w_i|w_{i-1})$ = discounted probability for seen bigrams (e.g., Good-Turing discounted) | $\alpha(w_{i-1})$ = backoff weight ensuring distribution sums to 1 | $P(w_i)$ = unigram probability (lower-order model)

Backoff Weight (normalization):

$$\alpha(w_{i-1}) = \frac{1 - \sum_{w_i: C(w_{i-1}, w_i) > 0} P^*(w_i|w_{i-1})}{\sum_{w_i: C(w_{i-1}, w_i) = 0} P(w_i)}$$

Numerator = leftover probability mass from discounting seen bigrams | Denominator = sum of unigram probabilities for unseen bigrams | Ensures $\sum_w P_{BO}(w|w_{i-1}) = 1$

Example: V = {a, b, c, d}. Given context "the": C(the a)=5, C(the b)=3, C(the)=10. Discount factor d=0.5.

$P^*(a|\text{the}) = (5 - 0.5)/10 = 0.45$, $P^*(b|\text{the}) = (3 - 0.5)/10 = 0.25$

Leftover mass = $1 - 0.45 - 0.25 = 0.30$

Unseen: {c, d}. $P(c) = 0.1$, $P(d) = 0.2$ → sum = 0.3

$\alpha(\text{the}) = 0.30/0.30 = 1.0$

$P_{BO}(c|\text{the}) = 1.0 \times 0.1 = 0.10$, $P_{BO}(d|\text{the}) = 1.0 \times 0.2 = 0.20$

Verify: $0.45 + 0.25 + 0.10 + 0.20 = 1.0$ ✓

Katz Backoff vs Interpolation: Backoff uses ONE level (the highest available), while interpolation ALWAYS mixes all levels. Backoff is simpler; interpolation is usually smoother.

PART 3 — Perplexity & Cross-Entropy

14. What Is Perplexity?

Perplexity evaluates how good a language model is. Intuition: "How surprised is the model?"

- Lower perplexity = better predictions
- Higher perplexity = poor predictions

15. Perplexity Formula

$$PP(W) = P(w_1, w_2, \ldots, w_N)^{-\frac{1}{N}} = \sqrt[N]{\frac{1}{P(w_1, w_2, \ldots, w_N)}}$$

$PP(W)$ = perplexity | $P(W)$ = probability model assigns to sentence | N = sentence length

16. Cross-Entropy ↔ Perplexity Relationship

Cross-Entropy of a Language Model:

$$H(P, Q) = -\frac{1}{N} \sum_{i=1}^{N} \log_2 Q(w_i | w_{<i})$$

P = true distribution (empirical / test data) | Q = model distribution | N = number of tokens in test set | This is the average number of bits needed per token

Perplexity = Exponentiated Cross-Entropy:

$$PP = 2^{H(P,Q)} = 2^{-\frac{1}{N} \sum_{i=1}^{N} \log_2 Q(w_i | w_{<i})}$$

Lower H → lower PP | If $H = 3$ bits → $PP = 2^3 = 8$ (model is as confused as choosing among 8 equally likely words)

With natural log (nats):

$$PP = e^{H_{nat}} = \exp\left(-\frac{1}{N} \sum_{i=1}^{N} \ln Q(w_i | w_{<i})\right)$$

Most deep learning frameworks use $\ln$ (natural log) so perplexity = $e^{\text{cross-entropy loss}}$

KEY EXAM FACT: "Lower cross-entropy ⟹ lower perplexity" is ALWAYS TRUE because PP is a monotonically increasing function of H. Cross-entropy is the log of perplexity.

Empirical Entropy

Empirical Entropy (estimated from test data):

$$\hat{H} = -\frac{1}{N} \sum_{i=1}^{N} \log_2 P(w_i | w_{i-k}, \ldots, w_{i-1})$$

$\hat{H}$ = empirical cross-entropy estimated on the test corpus | N = total tokens in test data | k = context window (n-gram order − 1) | As $N \to \infty$, $\hat{H}$ converges to true cross-entropy $H(P, Q)$

Why "empirical"? We don't know the true distribution P — we approximate it by counting over a finite test corpus. Perplexity from empirical entropy: $PP = 2^{\hat{H}}$. This is exactly what we compute when we "evaluate perplexity on a test set."

Problem: A model assigns the following probabilities on test sentence "the cat sat": $Q(\text{the}|<s>) = 0.2$, $Q(\text{cat}|\text{the}) = 0.4$, $Q(\text{sat}|\text{the cat}) = 0.3$.

Cross-Entropy:

$$H = -\frac{1}{3}[\log_2(0.2) + \log_2(0.4) + \log_2(0.3)] = -\frac{1}{3}[-2.322 - 1.322 - 1.737] = \frac{5.381}{3} = 1.794 \text{ bits}$$

Perplexity: $PP = 2^{1.794} = \mathbf{3.47}$

Interpretation: The model is on average as confused as picking between ~3.5 equally likely words.

17. Numerical Example — Perplexity (Simple)

Problem: $P(W) = 0.0001$, $N = 4$. Find perplexity.

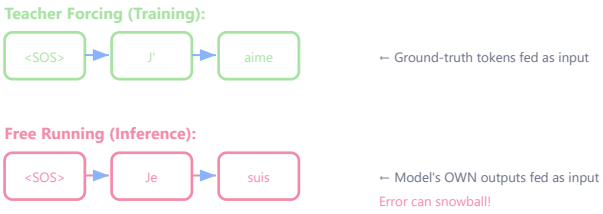
$$PP(W) = 0.0001^{-1/4} = \frac{1}{(0.0001)^{1/4}} = \frac{1}{0.1} = \mathbf{10}$$

Model is as confused as choosing uniformly among 10 words.

PART 4A — Teacher Forcing

18. What Is Teacher Forcing?

During training of Seq2Seq models, at each decoder step, we feed the **ground-truth previous token** instead of the model's own prediction.



19. Exposure Bias Problem

> At training time, the decoder always sees correct previous tokens. At inference, it sees its own (possibly wrong) predictions. This mismatch = **exposure bias**. One error cascades into subsequent errors.

20. Solutions to Exposure Bias

Technique	How It Works
Scheduled Sampling	Gradually switch from teacher forcing to free-running during training (linear/exponential schedule)
Beam Search (at inference)	Keep top-k candidates at each step, reducing single-error cascading
Minimum Risk Training	Train directly on sequence-level metrics (BLEU, ROUGE)

PART 4B — Word Embeddings (Word2Vec & GloVe)

21. Why Word Embeddings?

One-hot vectors are sparse (V-dimensional, only one 1) and encode NO semantic relationships. Word embeddings map words to dense, low-dimensional vectors where similar words are close.

One-Hot (bad):

king = [0,0,1,0,...,0] (50000-dim)
queen = [0,0,0,1,...,0] (50000-dim)
cos(king, queen) = 0 ← no similarity!

Embedding (good):

king = [0.8, 0.6, -0.2, ...] (300-dim)
queen = [0.7, 0.5, -0.1, ...] (300-dim)
cos(king, queen) ≈ 0.9 ← similar!

Famous analogy:

king − man + woman ≈ queen

Embeddings capture semantic relationships as vector arithmetic

22. Word2Vec: Skip-Gram

Given a center word, predict surrounding context words. Learns embeddings by maximizing $P(\text{context}|\text{center})$.

Skip-Gram Objective:

$$J = -\frac{1}{T}\sum_{t=1}^{T}\sum_{\substack{-c\le j\le c\\ j\neq 0}} \log P(w_{t+j}|w_t)$$

T = corpus size (total tokens) | c = context window size | w_t = center word | w_{t+j} = context word

Probability using softmax:

$$P(w_O|w_I) = \frac{\exp(u_{w_O}^T v_{w_I})}{\sum_{w=1}^{V}\exp(u_w^T v_{w_I})}$$

v_{w_I} = input embedding of center word (center vector) | u_{w_O} = output embedding of context word (context vector) | V = vocabulary size | Denominator sums over ALL words → expensive!

23. Word2Vec: CBOW (Continuous Bag of Words)

Given context words, predict the center word. Opposite of Skip-Gram.

CBOW Input:

$$\hat{v} = \frac{1}{2c}\sum_{\substack{-c\le j\le c\\ j\neq 0}} v_{w_{t+j}}$$

Prediction:

$$P(w_t|w_{t-c},\ldots,w_{t+c}) = \frac{\exp(u_{w_t}^T \hat{v})}{\sum_{w=1}^{V}\exp(u_w^T \hat{v})}$$

$\hat{v}$ = average of context word embeddings | Predicts center word from context | Faster for frequent words; Skip-Gram better for rare words

24. Negative Sampling (Efficiency)

Computing full softmax over V words is expensive. Negative sampling approximates it:

$$\log P(w_O|w_I) \approx \log\sigma(u_{w_O}^T v_{w_I}) + \sum_{k=1}^{K}\mathbb{E}_{w_k\sim P_n}\left[\log\sigma(-u_{w_k}^T v_{w_I})\right]$$

σ = sigmoid function | K = number of negative samples (5–20) | $P_n(w) = \frac{f(w)^{3/4}}{\sum_{w'} f(w')^{3/4}}$ (unigram distribution raised to 3/4 power) | Push positive pairs together, push random pairs apart

25. GloVe (Global Vectors)

Combines the benefits of global matrix factorization (like LSA) with local context windows (like Word2Vec). Trained on the **co-occurrence matrix**.

GloVe Objective:

$$J = \sum_{i,j=1}^{V} f(X_{ij}) \left(w_i^T \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij} \right)^2$$

X_{ij} = co-occurrence count (how often word j appears in context of word i) | w_i = word vector | $\tilde{w}_j$ = context vector | $b_i, \tilde{b}_j$ = biases | $f(X_{ij})$ = weighting function (caps very frequent pairs)

Weighting Function:

$$f(x) = \begin{cases} (x/x_{max})^{3/4} & \text{if } x < x_{max} \\ 1 & \text{otherwise} \end{cases}$$

$x_{max} = 100$ (typical) | Prevents very common pairs (the, is) from dominating the loss | Power 3/4 smooths the distribution

26. Word2Vec vs GloVe Comparison

Aspect	Word2Vec	GloVe
Type	Predictive (neural)	Count-based (matrix factorization)
Training	Local context windows + SGD	Global co-occurrence matrix
Objective	Maximize P(context|center)	Minimize squared error on log co-occurrences
Negative Sampling	Yes (essential for efficiency)	Not needed (operates on counts)
Performance	Better on syntax (analogies)	Better on global similarity
Key insight	Words in same context → similar	Log probability ∝ vector dot product

EXAM: GloVe uses negative sampling? FALSE — negative sampling is a Word2Vec technique. GloVe directly fits co-occurrence statistics.

27. Combining Embeddings with Autoencoders (NCC Task)

Noun Compound Classification (NCC): given two words (e.g., "olive" + "oil"), combine their embeddings to predict relationship.

Autoencoder for Embedding Composition:

$$x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \in \mathbb{R}^{2D_x}$$

$$p = \text{ReLU}(W_1 x + b_1) \in \mathbb{R}^{D_p}$$

$$x' = W_2 p + b_2 \quad \text{(reconstruction)}$$

$$\hat{y} = \text{softmax}(W_3 p + b_3) \quad \text{(classification)}$$

Total Loss:

$$J = J_{reconstruction} + J_{classification} = \frac{1}{2}\|x' - x\|^2 + \text{CE}(y, \hat{y})$$

$x_1, x_2 \in \mathbb{R}^{D_x}$ = embeddings of two words | p = parent/combined representation | $W_1 = [W_{11}\ W_{12}]$ so $W_1 x = W_{11} x_1 + W_{12} x_2$ | Dual objective: compress well (reconstruct) AND classify correctly

28. Contextual vs Static Embeddings

Aspect	Static (Word2Vec, GloVe)	Contextual (BERT, GPT — Topic 9)
Representation	One vector per word (fixed)	Different vector per occurrence
Polysemy	"bank" → one vector for both meanings	"bank" → different for river vs finance
Training	Unsupervised on co-occurrences	Self-supervised on context prediction
Dimension	100–300	768–4096

PART 4C — Variational Inference in Language Models

29. KL Divergence Recap

$$D_{KL}(P\|Q) = \mathbb{E}_{x\sim P}\left[\log \frac{P(x)}{Q(x)}\right] = \sum_x P(x)\log\frac{P(x)}{Q(x)}$$

$D_{KL} \geq 0$ always | $D_{KL} = 0$ iff $P = Q$ | NOT symmetric: $D_{KL}(P\|Q) \neq D_{KL}(Q\|P)$ | Measures "extra bits" needed when using Q to encode P

30. ELBO Derivation for Language Models

Given: true posterior $P(\text{output}|\text{context})$, approximate posterior $Q(\text{output}|\text{context})$, prior $P(\text{output})$, likelihood $P(\text{context}|\text{output})$.

Starting from KL divergence:

$$D_{KL}(Q(\text{out}|\text{ctx})\|P(\text{out}|\text{ctx}))$$

$$= \mathbb{E}_Q\left[\log \frac{Q(\text{out}|\text{ctx})}{P(\text{out}|\text{ctx})}\right]$$

$$= \mathbb{E}_Q[\log Q(\text{out}|\text{ctx}) - \log P(\text{out}|\text{ctx})]$$

Using Bayes' rule: $P(\text{out}|\text{ctx}) = \frac{P(\text{ctx}|\text{out})P(\text{out})}{P(\text{ctx})}$:

$$= \mathbb{E}_Q[\log Q(\text{out}|\text{ctx}) - \log P(\text{ctx}|\text{out}) - \log P(\text{out}) + \log P(\text{ctx})]$$

Rearranging (since $\log P(\text{ctx})$ is constant w.r.t. Q):

$$D_{KL}(Q\|P_{\text{post}}) = \log P(\text{ctx}) - \mathbb{E}_Q[\log P(\text{ctx}|\text{out})] + D_{KL}(Q(\text{out}|\text{ctx})\|P(\text{out}))$$

This is the fundamental VAE/variational inference decomposition | $\log P(\text{ctx})$ = log evidence (intractable) | $\mathbb{E}_Q[\log P(\text{ctx}|\text{out})]$ = expected log-likelihood | $D_{KL}(Q\|P_{\text{prior}})$ = regularization toward prior

ELBO (Evidence Lower Bound):

$$\log P(\text{ctx}) \geq \mathbb{E}_Q[\log P(\text{ctx}|\text{out})] - D_{KL}(Q(\text{out}|\text{ctx})\|P(\text{out}))$$

$$= \text{ELBO}$$

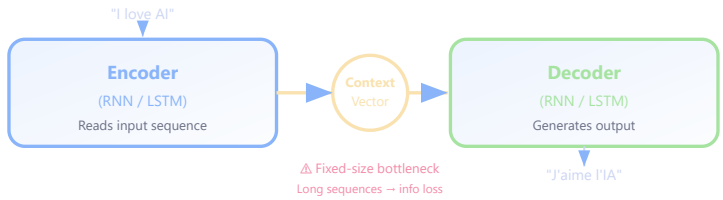
Since $D_{KL}(Q\|P_{\text{post}}) \geq 0$, we get: $\log P(\text{ctx}) \geq \text{ELBO}$ | Maximizing ELBO = maximizing likelihood + minimizing approximation gap

PART 5 — Seq2Seq (Sequence-to-Sequence)

17. What Is Seq2Seq?

Maps one sequence to another. Applications: translation, summarization, chatbots.

18. Seq2Seq Architecture



19. The Bottleneck Problem

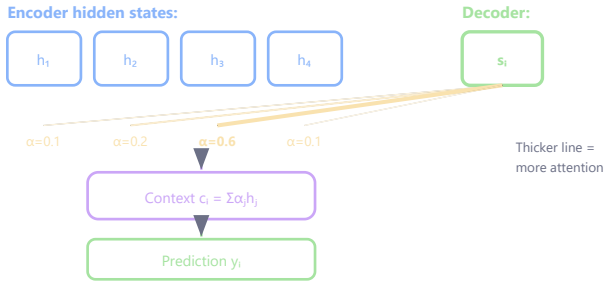
> Entire input compressed into one fixed-size vector. This becomes a bottleneck for long sequences — leading to the invention of Attention.

PART 5 — Attention Mechanism

20. Why Attention Was Introduced

Attention solves the fixed-vector bottleneck by allowing the decoder to look at ALL encoder states dynamically at each decoding step.

21. How Attention Works



22. Attention Score

$$e_{ij} = s_i^T h_j$$

e_{ij} = raw similarity between decoder step i and encoder position j | s_i = decoder hidden state | h_j = encoder hidden state

23. Attention Weights

$$\alpha_{ij} = \text{softmax}(e_{ij}) = \frac{e^{e_{ij}}}{\sum_k e^{e_{ik}}}$$

α_{ij} = normalized importance of encoder position j | All weights sum to 1

24. Context Vector

$$c_i = \sum_j \alpha_{ij} h_j$$

c_i = weighted sum of encoder states | Decoder "sees" more of what's relevant

Attention became revolutionary: solved long-sequence degradation, enabled direct connections across distances, and became the foundation for Transformers (Topic 9 — the architecture that replaced RNNs for most NLP).

PART 6 — Best Practices & Quick Revision

25. Best Practices

Practice	Reason
Use smoothing for N-grams	Handle unseen word combinations
Use attention for Seq2Seq	Better long-sequence handling
Use word embeddings	Reduce sparsity, capture semantics
Evaluate with perplexity	Quantitative LM comparison

26. Quick Revision

Concept	Key Idea
N-Gram	Count-based language model (Markov assumption)
Perplexity	LM evaluation metric (lower = better)
Smoothing	Handle unseen words (Laplace, Backoff)
Seq2Seq	Encoder-decoder for sequence mapping
Attention	Dynamic focus on relevant encoder states

Advanced Questions & Answers

Q1. Bigram MLE + Laplace Smoothing

Given corpus: "the cat sat on the mat the cat ate the food", compute P("cat" | "the") using MLE. Then apply Add-1 smoothing with V=8.

Without smoothing (MLE):

Count("the cat") = 2, Count("the") = 4

$$P(\text{"cat"}|\text{"the"}) = \frac{2}{4} = \mathbf{0.50}$$

With Add-1 smoothing:

$$P_{smooth}(\text{"cat"}|\text{"the"}) = \frac{2+1}{4+8} = \frac{3}{12} = \mathbf{0.25}$$

Smoothing redistributes mass: unseen bigrams get P = 1/12 = 0.083 instead of 0.

Q2. Perplexity Calculation

Compute perplexity on "the cat sat" given: P("cat"|"the")=0.5, P("sat"|"cat")=0.3, P("the"|"<s>")=0.8.

Using $\log_2$: $\log_2(0.8) = -0.322$, $\log_2(0.5) = -1.0$, $\log_2(0.3) = -1.737$

Sum = -3.059, N = 3

$$PP = 2^{3.059/3} = 2^{1.020} = \mathbf{2.03}$$

Interpretation: Model is as confused as choosing uniformly between ~2 words. Lower is better; perfect = 1.

Q3. Information Bottleneck in Seq2Seq

Encoder produces context vector c ∈ $\mathbb{R}^{512}$. Explain the bottleneck mathematically and how attention solves it.

Bottleneck:

For 100-word sentence with V=50,000: Info ≈ 100 × $\log_2(50000)$ ≈ 1,560 bits. While vector capacity is sufficient theoretically, long sentences (>20 words) degrade significantly in practice.

Attention solution:

$$\alpha_{t,i} = \frac{\exp(score(s_t, h_i))}{\sum_j \exp(score(s_t, h_j))}$$

$$context_t = \sum_i \alpha_{t,i} \cdot h_i$$

Info available = $T \times 512$ (grows with input length) — bottleneck eliminated.

Q4. Bahdanau vs Luong Attention

Given H = [[0.1,0.2,0.3],[0.4,0.5,0.6],[0.7,0.8,0.9],[0.2,0.1,0.4]], s = [0.5,0.3,0.7]. Compute Luong attention weights.

$score(s, h_i) = s^T \cdot h_i$:

- $score_1$ = 0.5(0.1)+0.3(0.2)+0.7(0.3) = 0.32
- $score_2$ = 0.5(0.4)+0.3(0.5)+0.7(0.6) = 0.77
- $score_3$ = 0.5(0.7)+0.3(0.8)+0.7(0.9) = 1.22
- $score_4$ = 0.5(0.2)+0.3(0.1)+0.7(0.4) = 0.41

Softmax: exp = [1.377, 2.160, 3.387, 1.507], sum = 8.431

α = [0.163, 0.256, **402**, 0.179]

Bahdanau: $score = v^T \tanh(W_1 s + W_2 h_i)$ — $O(d^2)$, more expressive.

Luong: $score = s^T h_i$ — $O(d)$, faster.

Q5. Decoding Strategies Comparison

P("good")=0.3, P("bad")=0.2, P("nice")=0.15, P("cold")=0.1 after "the weather is". Compare greedy, top-k (k=3), and top-p (p=0.7).

Greedy:

Always "good". Zero diversity.

Top-k (k=3):

{good, bad, nice}: P(good)=0.462, P(bad)=0.308, P(nice)=0.231. Entropy=1.52 bits.

Top-p (p=0.7):

Accumulate: good(0.3)+bad(0.5)+nice(0.65)+cold(0.75≥0.7). Set={good,bad,nice,cold}.

Renormalized: P(good)=0.4, P(bad)=0.267, P(nice)=0.2, P(cold)=0.133. Entropy=1.87 bits.

Top-p produces more diversity by adapting set size to distribution shape.

Q6. Katz Backoff Calculation (Exam Question)

Given: V = {the, cat, dog, sat, ran}, context = "cat". Bigram counts: C(cat sat)=8, C(cat ran)=4, C(cat)=15. Discount d=0.75. Unigram probs: P(the)=0.3, P(dog)=0.1, P(sat)=0.2, P(ran)=0.15, P(cat)=0.25. Compute $P_{BO}(w|\text{cat})$ for all w.

Step 1: Discounted probabilities for seen bigrams:

$P^*(\text{sat}|\text{cat}) = \frac{8-0.75}{15} = \frac{7.25}{15} = 0.483$

$P^*(\text{ran}|\text{cat}) = \frac{4-0.75}{15} = \frac{3.25}{15} = 0.217$

Step 2: Leftover mass:

$1 - 0.483 - 0.217 = 0.300$

Step 3: α weight for unseen bigrams {the, dog, cat}:

Sum of unigram probs for unseen: $0.3 + 0.1 + 0.25 = 0.65$

$\alpha(\text{cat}) = 0.300 / 0.65 = 0.462$

Step 4: Final probabilities:

Word	$P_{BO}(w\|\text{cat})$
sat	0.483 (seen bigram)
ran	0.217 (seen bigram)
the	$0.462 \times 0.3 = 0.139$
cat	$0.462 \times 0.25 = 0.115$
dog	$0.462 \times 0.1 = 0.046$

Verify: $0.483 + 0.217 + 0.139 + 0.115 + 0.046 = 1.000$ ✓

Q7. Variational Inference Derivation (Quiz 1 Question)

Show that

$$D_{KL}(Q(\text{output}|\text{context})\|P(\text{output}|\text{context})) = \log P(\text{context}) - \mathbb{E}_Q[\log P(\text{context}|\text{output})] + D_{KL}(Q(\text{output}|\text{context})\|P(\text{output}))$$

Step 1: Expand KL divergence definition:

$$D_{KL}(Q(o|c)\|P(o|c)) = \sum_o Q(o|c) \log \frac{Q(o|c)}{P(o|c)}$$

Step 2: Apply Bayes' theorem to $P(o|c)$:

$$P(o|c) = \frac{P(c|o) \cdot P(o)}{P(c)}$$

Step 3: Substitute:

$$= \sum_o Q(o|c) \log \frac{Q(o|c) \cdot P(c)}{P(c|o) \cdot P(o)}$$

$$= \sum_o Q(o|c) \left[\log \frac{Q(o|c)}{P(o)} + \log P(c) - \log P(c|o)\right]$$

Step 4: Separate sums:

$$= \underbrace{\sum_o Q(o|c) \log \frac{Q(o|c)}{P(o)}}_{D_{KL}(Q(o|c)\|P(o))} + \log P(c) \underbrace{\sum_o Q(o|c)}_{=1} - \underbrace{\sum_o Q(o|c) \log P(c|o)}_{\mathbb{E}_Q[\log P(c|o)]}$$

Final Result:

$$\boxed{D_{KL}(Q(o|c)\|P(o|c)) = \log P(c) - \mathbb{E}_Q[\log P(c|o)] + D_{KL}(Q(o|c)\|P(o))}$$

Q8. BPTT Jacobian Dimensions (Quiz 1 Question)

Given output O ∈ $\mathbb{R}^{n \times m}$, hidden state H ∈ $\mathbb{R}^{n \times m}$, input X ∈ $\mathbb{R}^{p}$, and loss L (scalar). Find dimensions of: (1) ∂L/∂O, (2) ∂O/∂H, (3) ∂H/∂X, (4) ∂^2L/∂H^2.

---

(1) ∂L/∂O:

L is scalar, O is $n \times m$. Gradient has same shape as O: $n \times m$

(2) ∂O/∂H (Jacobian):

O is $n \times m$ (= nm elements), H is $n \times m$ (= nm elements). Jacobian: $nm \times nm$

(Each element of O can depend on each element of H)

(3) ∂H/∂X (Jacobian):

H has nm elements, X has p elements. Jacobian: $nm \times p$

(4) ∂^2L/∂H^2 (Hessian):

L is scalar, H has nm elements. Hessian: $nm \times nm$

(Second-order partial derivatives of scalar w.r.t. each pair of H elements)

Q9. Word Embedding Autoencoder Gradients (Quiz 1 Q3)

Given: $p = \text{ReLU}(W_1 x + b_1)$, $x' = W_2 p + b_2$, $J_1 = \frac{1}{2}\|x' - x\|^2$, $h = W_1 x + b_1$. **Compute:** (1) $\delta_1 = \partial J_1 / \partial p$, (2) $\delta_2 = \partial J_1 / \partial h$, (3) $\partial J_1 / \partial W_1$.

---

(1) $\delta_1 = \partial J_1 / \partial p$:

Chain rule: $\frac{\partial J_1}{\partial p} = \frac{\partial J_1}{\partial x'} \cdot \frac{\partial x'}{\partial p}$

$\frac{\partial J_1}{\partial x'} = (x' - x)$ and $\frac{\partial x'}{\partial p} = W_2$

$$\boxed{\delta_1 = W_2^T (x' - x)}$$

(2) $\delta_2 = \partial J_1 / \partial h$:

Since $p = \text{ReLU}(h)$: $\frac{\partial p}{\partial h} = \text{diag}(\mathbb{1}[h > 0])$

$$\boxed{\delta_2 = \delta_1 \odot \mathbb{1}[h > 0]}$$

(Element-wise multiply δ_1 with ReLU derivative mask)

(3) $\partial J_1 / \partial W_1$:

Since $h = W_1 x + b_1$: $\frac{\partial h}{\partial W_1} = x^T$

$$\boxed{\frac{\partial J_1}{\partial W_1} = \delta_2 \cdot x^T}$$

(Outer product of pre-activation gradient with input)

Q10. Cross-Entropy MCQ (Practice Set)

TRUE or FALSE: "Lower cross-entropy always implies lower perplexity."

TRUE.

Proof: $PP = b^H$ where b is the log base (2 for bits, e for nats). Since $b > 1$, the function $f(H) = b^H$ is **strictly monotonically increasing**. Therefore:

$$H_1 < H_2 \implies b^{H_1} < b^{H_2} \implies PP_1 < PP_2$$

Lower cross-entropy → ALWAYS lower perplexity. No exceptions.

Q11. Teacher Forcing MCQ (Practice Set)

In the context of sequence-to-sequence models with attention, what is "teacher forcing"?

(a) Using the model's own predictions as input during training

(b) Using ground truth outputs as decoder inputs during training

(c) Forcing the attention weights to be uniform

(d) Using a pre-trained teacher model to guide training

Answer: (b)

Teacher forcing = ground-truth previous token fed as input to decoder at each step during training. This provides a stable training signal but creates exposure bias (train-test mismatch).

Q12. Bahdanau Attention MCQ (End-Term)

TRUE or FALSE: "In Bahdanau attention, each decoder step has a one-to-one correspondence with exactly one encoder state."

FALSE.

Bahdanau attention is **soft attention** — each decoder step attends to ALL encoder states simultaneously with learned weights α_{ij}. The context vector is a **weighted sum** of all encoder states:

$$c_i = \sum_{j=1}^{T_x} \alpha_{ij} h_j$$

This is NOT one-to-one; it's many-to-many (each decoder step looks at everything). Hard attention would select exactly one state, but Bahdanau uses soft (differentiable) attention.

Q13. Empirical Entropy & Perplexity (TA Priority)

A bigram model is evaluated on the test sentence "I love deep learning" (4 tokens after <s>). The model assigns:
$P(\text{I}|<\text{s}>) = 0.1$, $P(\text{love}|\text{I}) = 0.05$, $P(\text{deep}|\text{love}) = 0.02$, $P(\text{learning}|\text{deep}) = 0.4$.
Compute the empirical entropy $\hat{H}$ and perplexity. Explain why this is called "empirical."

Step 1: Compute empirical entropy

$$\hat{H} = -\frac{1}{N}\sum_{i=1}^{N}\log_2 P(w_i|w_{i-1})$$

$$= -\frac{1}{4}[\log_2(0.1) + \log_2(0.05) + \log_2(0.02) + \log_2(0.4)]$$

$$= -\frac{1}{4}[-3.322 + (-4.322) + (-5.644) + (-1.322)]$$

$$= -\frac{1}{4}(-14.61) = \mathbf{3.65 \text{ bits/token}}$$

Step 2: Perplexity

$$PP = 2^{\hat{H}} = 2^{3.65} = \mathbf{12.55}$$

Interpretation: On average, the model is as confused as choosing among ~12.5 equally likely words at each step.

Why "empirical"?

We don't know the true probability distribution of language. We estimate entropy using a finite test corpus (4 tokens here). As the test set grows ($N \to \infty$), $\hat{H}$ converges to the true cross-entropy $H(P, Q)$. With finite data, it's an **empirical estimate** — hence the name.

Q14. Chain Rule → Bigram → Perplexity (TA Session Question)

(a) Expand $P(\text{I, love, NLP})$ using the chain rule.

(b) Rewrite under the bigram assumption.

(c) Given: $P(\text{I}) = 0.1$, $P(\text{love}|\text{I}) = 0.5$, $P(\text{NLP}|\text{love}) = 0.4$. Compute sentence probability and perplexity.

(a) Full chain rule:

$$P(\text{I, love, NLP}) = P(\text{I}) \cdot P(\text{love}|\text{I}) \cdot P(\text{NLP}|\text{I, love})$$

Each word is conditioned on ALL previous words — requires full history.

(b) Bigram assumption:

Markov assumption: $P(w_n|w_1, \ldots, w_{n-1}) \approx P(w_n|w_{n-1})$

$$P(\text{I, love, NLP}) \approx P(\text{I}) \cdot P(\text{love}|\text{I}) \cdot P(\text{NLP}|\text{love})$$

Notice: $P(\text{NLP}|\text{I, love})$ simplified to $P(\text{NLP}|\text{love})$ — only last word matters.

(c) Numerical:

Sentence probability:

$$P = 0.1 \times 0.5 \times 0.4 = \mathbf{0.02}$$

Perplexity (product form, $N = 3$ tokens):

$$PP = \left(\frac{1}{P_1 \cdot P_2 \cdot P_3}\right)^{1/N} = \left(\frac{1}{0.1 \times 0.5 \times 0.4}\right)^{1/3} = \left(\frac{1}{0.02}\right)^{1/3} = (50)^{1/3} \approx \mathbf{3.68}$$

> **Equivalence check (entropy form):**
> $\hat{H} = -\frac{1}{3}[\log_2(0.1) + \log_2(0.5) + \log_2(0.4)] = -\frac{1}{3}[-3.322 - 1.0 - 1.322] = 1.881$ bits
> $PP = 2^{1.881} = 3.68$ ✓ — both forms give same answer.

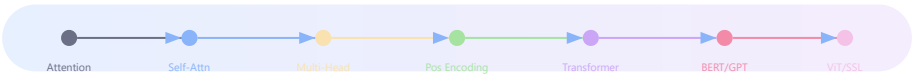
Created by **Kushal Ghosh** (gkushalg01@gmail.com) and **Bharat Das Vaishnav** (bharatvaishnav2025@gmail.com)

↑ Back to Top | **Transformers & Modern LLMs**

Topic 9 — Transformer & Modern LLMs

Transformers are the architecture that made modern Large Language Models possible. They improved on RNN/LSTM/GRU by removing sequential recurrence, enabling parallel processing, modeling long-range dependencies better, and scaling extremely well with data and compute.

1. Learning Progression



2. Why Transformers Were Needed

RNN Problems	Transformer Solution
Sequential (no parallelism)	All tokens processed simultaneously
Long-range dependencies hard	Direct token-to-token attention
Vanishing gradients	Residual connections + short paths
Slow training	Massive GPU parallelization

3. Core Idea

"Which other tokens are most relevant to this token right now?" — That is the heart of attention.

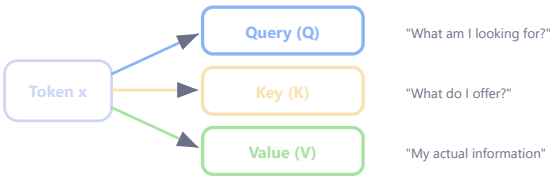
PART 1 — Self-Attention

4. What Is Self-Attention?

Each token in a sequence looks at **all other tokens in the same sequence** and decides what matters most.

```
"The animal did not cross the road because it was tired."
// When interpreting "it" → model focuses on "animal" (not "road")
```

5. Query, Key, Value (Q, K, V)



Term	Meaning	Analogy
Query	What this token is looking for	Search query
Key	What each token offers	File label
Value	The information to actually use	File content

6. Scaled Dot-Product Attention Formula

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Q = query matrix (shape: $n \times d_k$) | K = key matrix | V = value matrix | d_k = dimension of keys | $\sqrt{d_k}$ = scaling factor to prevent large dot products

Without scaling by $\sqrt{d_k}$: dot products grow large → softmax becomes peaky → gradients vanish. Scaling keeps training stable.

7. Numerical Example — Self-Attention

Problem: $q = [1, 0]$, Keys: $k_1 = [1, 0],\ k_2 = [0, 1],\ k_3 = [1, 1]$, Values: $v_1 = [2, 0],\ v_2 = [0, 2],\ v_3 = [1, 1]$, $d_k = 2$

Step 1 — Dot Products:

$q \cdot k_1 = 1, \quad q \cdot k_2 = 0, \quad q \cdot k_3 = 1$

Step 2 — Scale:

$\frac{[1,0,1]}{\sqrt{2}} = [0.707, 0, 0.707]$

Step 3 — Softmax:

$\exp([0.707, 0, 0.707]) = [2.028, 1.0, 2.028]$, sum = 5.056

$\alpha = [0.40, 0.20, 0.40]$

Step 4 — Weighted Sum:

$0.40[2, 0] + 0.20[0, 2] + 0.40[1, 1] = [0.8, 0] + [0, 0.4] + [0.4, 0.4] = [\mathbf{1.2},\ \mathbf{0.8}]$

PART 2 — Multi-Head Attention

8. Why Multiple Heads?

A single attention head focuses on one type of relationship. Multiple heads let the model attend to **different aspects simultaneously**:

```
"The boy who won the race is happy."
// Head 1: boy ↔ happy (subject-predicate)
// Head 2: won ↔ race (verb-object)
// Head 3: boy ↔ who (coreference)
```

9. Multi-Head Attention Formula

$$\text{head}_i = \text{Attention}(QW_i^Q,\ KW_i^K,\ VW_i^V)$$

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\, W^O$$

head_i = output of attention head i | W_i^Q, W_i^K, W_i^V = learned projection matrices for head i | h = number of heads | W^O = output projection matrix | Each head dimension = $d_k = d_{model}/h$

10. Numerical Example — Multi-Head Shape

$d_{model} = 768$, heads $h = 12$. Each head: $d_k = 768/12 = 64$.

Each head produces a 64-dim output → concat all 12 → 768-dim → multiply by W^O (768×768).

PART 3 — Positional Encoding

11. Why Position Matters

Self-attention is **permutation invariant** — it cannot distinguish order. Without positional info:

```
"Dog bites man" == "Man bites dog"  // WRONG!
```

12. Sinusoidal Positional Encoding

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

$$PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

pos = token position in sequence (0, 1, 2, ...) | i = dimension index (0 to $d_{model}/2$) | d_{model} = embedding dimension | Even dims use sin, odd dims use cos | Low i = high frequency (fine position), high i = low frequency (coarse)

13. Numerical Example — PE

$d_{model} = 4$, position $pos = 1$:

$PE(1, 0) = \sin(1/10000^{0/4}) = \sin(1) = 0.841$

$PE(1, 1) = \cos(1/10000^{0/4}) = \cos(1) = 0.540$

$PE(1, 2) = \sin(1/10000^{2/4}) = \sin(0.01) = 0.010$

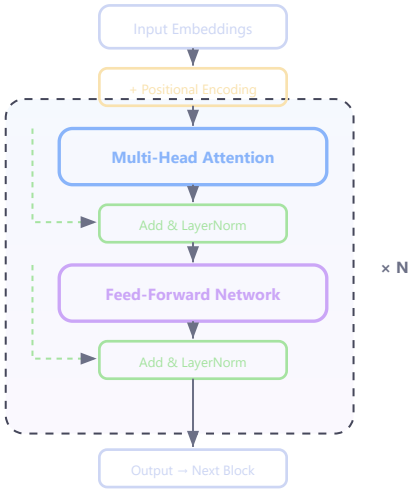
$PE(1, 3) = \cos(1/10000^{2/4}) = \cos(0.01) = 1.000$

PE(1) = **[0.841, 0.540, 0.010, 1.000]**

> Sinusoidal encodings can generalize to longer sequences than seen in training. Learned positional embeddings cannot.

PART 4 — Transformer Architecture

14. The Building Blocks



15. Feed-Forward Network

$$FFN(x) = \max(0,\ xW_1 + b_1)\,W_2 + b_2$$

$W_1 \in \mathbb{R}^{d_{model} \times d_{ff}}$ = first projection (expand) | $W_2 \in \mathbb{R}^{d_{ff} \times d_{model}}$ = second projection (compress) | d_{ff} = inner dimension (typically $4 \times d_{model}$) | Applied independently per token

16. Residual Connections & Layer Normalization

$$\text{output} = \text{LayerNorm}(x + \text{SubLayer}(x))$$

x = original input (skip connection) | $\text{SubLayer}(x)$ = MHA or FFN output | Residual helps gradient flow; LayerNorm stabilizes activations

16A. Pre-Norm vs Post-Norm Transformers

Post-Norm (Original Transformer):

$$\text{output} = \text{LayerNorm}(x + \text{SubLayer}(x))$$

LayerNorm applied AFTER residual addition | Used in: original Transformer, BERT

Pre-Norm (Modern default):

$$\text{output} = x + \text{SubLayer}(\text{LayerNorm}(x))$$

LayerNorm applied BEFORE sublayer | Used in: GPT-2, GPT-3, LLaMA, most modern LLMs

Aspect	Post-Norm	Pre-Norm
Gradient flow	Can be unstable at depth	More stable (direct residual path)
Warmup needed	Critical (training diverges without)	Less critical
Training stability	Harder to train deep models	Easier to scale to many layers
Final performance	Slightly better when well-tuned	Slightly worse but easier to train
Models	BERT, original T5	GPT-2/3/4, LLaMA, PaLM

EXAM KEY: Pre-norm places LayerNorm BEFORE the sublayer, making the residual path a clean identity: $x + f(\text{LN}(x))$. This gives better gradient flow for deep networks. Post-norm can achieve slightly better results but is harder to train.

17. Three Transformer Flavors

Type	Architecture	Use Case	Model
Encoder-only	Bidirectional self-attention	Understanding, classification	BERT
Decoder-only	Causal (masked) self-attention	Generation, chat	GPT
Encoder-Decoder	Both + cross-attention	Translation, summarization	T5, BART

18. Attention Masking (Decoder) — Causal Masking

A decoder at time t must NOT see future tokens. A causal (triangular) mask sets future positions to $-\infty$ before softmax, giving them zero attention weight.

Causal Mask Matrix (for sequence length 4):

$$M = \begin{bmatrix} 0 & -\infty & -\infty & -\infty \\ 0 & 0 & -\infty & -\infty \\ 0 & 0 & 0 & -\infty \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Masked Attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right)V$$

$M_{ij} = 0$ if $i \geq j$ (can attend) | $M_{ij} = -\infty$ if $i < j$ (cannot attend to future) | After softmax: $e^{-\infty} = 0$, so future tokens get zero weight

Example: 3-token sequence, raw attention scores (before mask):

$$\frac{QK^T}{\sqrt{d_k}} = \begin{bmatrix} 2.0 & 1.5 & 0.8 \\ 1.2 & 3.0 & 1.0 \\ 0.5 & 1.8 & 2.5 \end{bmatrix}$$

After adding causal mask:

$$\begin{bmatrix} 2.0 & -\infty & -\infty \\ 1.2 & 3.0 & -\infty \\ 0.5 & 1.8 & 2.5 \end{bmatrix}$$

Row 0 softmax: $[\text{softmax}(2.0)] = [1.0, 0, 0]$ — token 0 only sees itself

Row 1 softmax: $\text{softmax}(1.2, 3.0) = [0.142, 0.858, 0]$ — token 1 sees tokens 0,1

Row 2 softmax: $\text{softmax}(0.5, 1.8, 2.5) = [0.094, 0.345, 0.561]$ — sees all preceding

18A. Self-Attention Output Formula

For a single query at position i:

$$\text{output}_i = \sum_{j=1}^{n} \alpha_{ij} \cdot v_j \quad \text{where} \quad \alpha_{ij} = \frac{\exp(q_i^T k_j / \sqrt{d_k})}{\sum_{m=1}^{n} \exp(q_i^T k_m / \sqrt{d_k})}$$

$\text{output}_i \in \mathbb{R}^{d_v}$ = output representation for token i | α_{ij} = attention weight token i gives to token j | Each output is a convex combination of value vectors (weights sum to 1)

PART 5 — MLM vs Autoregressive Training

19. Two Pretraining Paradigms

Aspect	MLM (Masked Language Modeling)	Autoregressive (AR)
Predict	Masked tokens from both sides	Next token from left context only
Context	Bidirectional	Left-to-right
Strength	Understanding & representations	Generation & continuation
Model	BERT	GPT

20. MLM Formula

$$\mathcal{L}_{MLM} = -\sum_{i \in M} \log P(x_i \mid x_{\setminus M})$$

M = set of masked positions (~15% of tokens) | x_i = true token at masked position | $x_{\setminus M}$ = all unmasked tokens (bidirectional context)

```
// MLM Example:
Original: "The cat sat on the mat"
Masked:   "The cat [MASK] on the mat"
Predict:  "sat"
```

21. Autoregressive Formula

$$\mathcal{L}_{AR} = -\sum_{t=1}^{T} \log P(x_t \mid x_{<t})$$

T = sequence length | x_t = token at position t | $x_{<t}$ = all tokens before position t (left context only)

```
// AR Example:
"I love deep learning"
P("love" | "I")
P("deep" | "I love")
P("learning" | "I love deep")
```

> MLM trains on only ~15% of tokens per batch (masked ones), while AR trains on ALL tokens. AR gets more gradient signal per sequence but cannot use future context.

PART 6 — BERT vs GPT vs T5 vs BART

22. Model Comparison

Model	Architecture	Objective	Strength
BERT	Encoder-only	MLM + NSP	Understanding, classification
GPT	Decoder-only	Autoregressive	Generation, chat
T5	Encoder-decoder	Text-to-text	Flexible NLP tasks
BART	Encoder-decoder	Denoising AE	Summarization, robust generation

23. When to Use Which

Task	Best Choice	Why
Classification / NER	BERT	Strong bidirectional understanding
Chat / completion	GPT	Natural left-to-right generation
Translation	T5 / BART	Needs both understanding + generation
Summarization	BART / T5	Encode long input, generate short output
Text-to-text (general)	T5	Unified interface for any task

24. Examples by Model

```
// BERT:
Input:  "This movie was [MASK]"  →  Output: "amazing"

// GPT:
Input:  "Once upon a time"  →  Output: "there was a kingdom ... "

// T5:
Input:  "translate English to French: I like coffee"
Output: "J'aime le café"

// BART:
Input:  "The boy went to the [MASK] and bought milk"
Output: "store"
```

PART 7 — End-to-End Transformer Flow

25. Full Pipeline

```
Input Tokens
   ↓  Token Embeddings (lookup table)
   ↓  + Positional Encoding
   ↓  Repeated Transformer Blocks (× N)
   ↓      └─ Multi-Head Attention → Add&Norm → FFN → Add&Norm
   ↓  Contextual Token Representations
   ↓  Task Head (classification / generation / seq2seq)
   ↓
Prediction
```

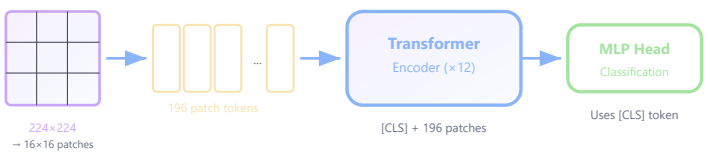
26. Why This Is So Effective

- Direct token-to-token paths (no recurrence bottleneck)
- Massive parallelism (all positions computed together)
- Scales with data and compute (emergent abilities at scale)
- Unified architecture for NLP, Vision, Audio

PART 8 — Vision Transformers (ViT)

27. Idea

Apply Transformer architecture to image classification by splitting an image into patches and treating each patch as a "token".



28. Patch Embedding Formula

$$N = \frac{H \times W}{P^2}$$

N = number of patches (sequence length for Transformer) | H, W = image height/width | P = patch size (e.g., 16) | Each patch flattened to $P^2 \times C$ then linearly projected to d_{model}

29. Numerical Example — Patch Count

Image: 224×224, Patch: 16×16

$$N = \frac{224 \times 224}{16 \times 16} = \frac{50176}{256} = \mathbf{196 \text{ patches}}$$

Total tokens = 196 + 1 [CLS] = **197**

30. ViT vs CNN

Aspect	CNN	ViT
Inductive bias	Local (convolution)	Global (attention)
Data requirement	Works with less data	Needs large datasets
Receptive field	Grows with depth	Global from layer 1
Compute scaling	Linear with resolution	Quadratic in seq length

PART 9 — Self-Supervised Learning

31. What Is Self-Supervised Learning?

Learning representations from data **without human-provided labels**. The model creates its own supervision signal from the data itself. Foundation models (BERT, GPT, CLIP) all use self-supervised pretraining.

32. Types of SSL

Type	Approach	Examples
Predictive	Predict missing/future parts	BERT (MLM), GPT (AR)
Contrastive	Pull similar pairs close, push dissimilar apart	SimCLR, CLIP
Generative	Reconstruct input from corrupted version	MAE, Denoising AE

33. Contrastive Loss (NT-Xent)

$$\mathcal{L} = -\log \frac{\exp(\text{sim}(z_i, z_j)/\tau)}{\sum_{k=1}^{2N} \mathbb{1}_{[k \neq i]} \exp(\text{sim}(z_i, z_k)/\tau)}$$

z_i, z_j = representations of two augmented views of same image (positive pair) | sim = cosine similarity | τ = temperature parameter | N = batch size | Denominator includes all negatives

34. Masked Autoencoders (MAE)

Visual equivalent of BERT's MLM: mask 75% of patches randomly, encode only visible patches, decoder reconstructs full image.

```
Input Image → Mask 75% patches → Encode visible → Decode all → MSE loss on masked pixels
```

PART 10A — Decoding Strategies (Beam Search)

35A. Beam Search

At each timestep, keep the top-k (beam size) most probable partial sequences instead of just the single best (greedy).

Beam Search Score:

$$\text{score}(y_{1:t}) = \sum_{i=1}^{t} \log P(y_i | y_{<i}, x)$$

Score = log-probability of sequence so far | Beam size k = number of candidates kept at each step | $k = 1$ = greedy decoding | Higher k → more exploration, slower

Length-Normalized Beam Search:

$$\text{score}_{norm}(y) = \frac{1}{|y|^{\alpha}} \sum_{i=1}^{|y|} \log P(y_i | y_{<i}, x)$$

$|y|$ = sequence length | $\alpha \in [0.6, 1.0]$ = length penalty | Without normalization, shorter sequences always win (fewer negative log terms)

35B. Beam Size Effects

Beam Size	Effect on Probability	Trade-off
$k = 1$ (greedy)	Highest per-step probability	Locally optimal, globally suboptimal
$k = 5$ (typical)	Higher overall sequence probability	Good quality/speed balance
$k = 100$ (large)	Closer to optimal sequence	Slow, diminishing returns, can degrade quality

EXAM: "Increasing beam size always increases the probability of the generated sequence." TRUE in theory (larger search space), but in practice very large beams can find degenerate high-probability sequences (empty or repetitive text). Beam search maximizes probability, NOT quality.

35C. Autoregressive Token Counting

Autoregressive models (GPT) generate one token at a time left-to-right. They are NOT good at counting tokens. If asked "generate exactly 5 words," they struggle because token generation has no built-in counter — it relies on implicit pattern matching from training data.

35D. Per-Token Latency: Transformer vs RNN

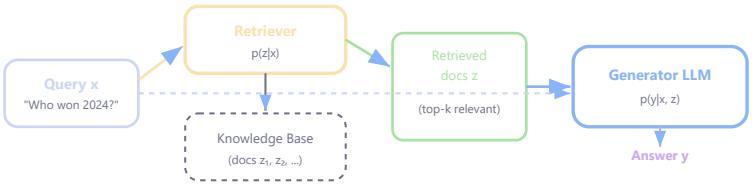
Aspect	RNN	Transformer (Autoregressive)
Training parallelism	Sequential (slow)	Fully parallel (fast)
Inference per token	$O(d^2)$ — just one step	$O(n \cdot d)$ — attends to all previous tokens
Per-token latency at inference	Constant (regardless of position)	Grows with sequence length

EXAM: "Transformers have lower per-token latency than RNNs." FALSE for inference! RNN inference is O(1) per step (fixed hidden state update). Transformer inference grows linearly with context length because each new token attends to all previous tokens. Transformers are faster at TRAINING (parallel), not per-token inference.

PART 10B — Retrieval-Augmented Generation (RAG)

35E. Why RAG?

LLMs hallucinate because they rely on parametric memory alone. RAG augments generation with retrieved documents from an external knowledge base.



EXAM: "RAG reduces hallucination." TRUE — by grounding generation in retrieved documents, the model has factual evidence to condition on, reducing confabulation.

35F. REALM (Retrieval-Augmented Language Model Pre-training)

REALM decomposes $p(y|x)$ into retrieve-then-predict:

$$p(y|x) = \sum_{z \in \mathcal{Z}} p(y|x,z) \cdot p(z|x)$$

Retriever score (neural):

$$f(x,z) = \text{Embed}_{input}(x)^T \cdot \text{Embed}_{doc}(z)$$

$$p(z|x) = \frac{\exp(f(x,z))}{\sum_{z' \in \mathcal{Z}} \exp(f(x,z'))}$$

$\mathcal{Z}$ = knowledge corpus (all documents) | $p(z|x)$ = retriever distribution (which doc is relevant) | $p(y|x,z)$ = reader/generator conditioned on retrieved doc | $f(x,z)$ = relevance score (dot product of embeddings)

REALM Gradient (w.r.t. retriever params θ):

$$\nabla_\theta \log p(y|x) = \sum_z \frac{p(y|x,z) \cdot p(z|x)}{p(y|x)} \cdot \nabla_\theta f(x,z)$$

$$= \sum_z p(z|x,y) \cdot \nabla_\theta f(x,z)$$

$p(z|x,y)$ = posterior probability of document z given both query and answer | Documents that explain the answer get higher gradient signal | This allows end-to-end training of the retriever!

35G. RAG-Token vs RAG-Sequence

RAG-Sequence (marginalize per sequence):

$$p(y|x) = \sum_{z \in \text{top-k}} p(z|x) \cdot p(y|x,z) = \sum_{z \in \text{top-k}} p(z|x) \prod_{i=1}^{N} p(y_i|x, z, y_{1:i-1})$$

Each document generates a COMPLETE sequence; final probability marginalizes over documents

RAG-Token (marginalize per token):

$$p(y|x) = \prod_{i=1}^{N} \sum_{z \in \text{top-k}} p(z|x) \cdot p(y_i|x, z, y_{1:i-1})$$

At each token position, marginalize over documents — different tokens can use different documents!

Variant	Marginalization	Use Case
RAG-Sequence	Per complete output	QA (coherent single-doc answers)
RAG-Token	Per output token	Long-form generation (combine info from multiple docs)

35H. Fusion-in-Decoder (FiD)

Encodes each retrieved document INDEPENDENTLY with the query, then fuses all representations in the decoder via cross-attention.

FiD Architecture:

$$\text{Encoder}_i = \text{Encode}(\text{concat}(x, z_i)) \quad \text{for each doc } z_i$$

$$\text{Decoder input} = \text{concat}(\text{Encoder}_1, \text{Encoder}_2, \ldots, \text{Encoder}_k)$$

k docs encoded independently → concatenated encoder outputs → decoder cross-attends to all | Key advantage: encoder is parallel, decoder fuses | Much more scalable than encoding all docs together (no quadratic cross-document attention in encoder)

35I. Dense vs Sparse Retrieval

Aspect	Sparse (BM25, TF-IDF)	Dense (DPR, Contriever)
Representation	Bag of words, term frequency	Dense neural embeddings
Matching	Exact keyword overlap	Semantic similarity (dot product/cosine)
Index	Inverted index	ANN (FAISS, HNSW)
Complexity	$O(\text{avg doc length})$	$O(d)$ per comparison + ANN structure
Retrieval time	**Sublinear** $o(N)$	**Sublinear** $o(N)$ with ANN
Semantic understanding	Weak (misses synonyms)	Strong (captures meaning)

EXAM: Both dense and sparse indexing achieve sublinear retrieval time o(N) — inverted index skips non-matching docs, ANN uses hierarchical graphs. Neither is O(N) linear scan.

PART 10C — Model Compression & Distillation

35J. Knowledge Distillation

Train a small "student" model to mimic a large "teacher" model's output distribution.

Distillation Loss:

$$\mathcal{L}_{distill} = (1-\alpha)\,\mathrm{CE}(y, p_S) + \alpha T^2\,\mathrm{KL}(p_T^{(T)} \| p_S^{(T)})$$

p_S = student predictions | $p_T^{(T)}$ = teacher soft predictions at temperature T | $\mathrm{CE}(y, p_S)$ = standard cross-entropy with true labels | α = interpolation weight | T = temperature (softens distributions, revealing "dark knowledge")

EXAM: "Distillation is a lossless compression technique." FALSE — distillation is LOSSY. The student cannot perfectly replicate the teacher's behavior; some information is lost. Lossless would mean identical outputs, which is not achievable with fewer parameters.

PART 10 — Best Practices & Quick Revision

35. Best Practices

Practice	Reason
Always use positional encoding	Attention is order-agnostic without it
Use causal masking in decoders	Prevent information leakage
Use LayerNorm + residuals in every block	Stable training, gradient flow
Match objective to task	MLM for understanding, AR for generation
Scale heads with model dimension	$d_k = d_{model}/h$ should be ≥ 32

36. Common Mistakes

Mistake	Problem
No positional encoding	Model loses sequence order
Using BERT for free generation	Not trained for left-to-right
Forgetting causal mask in decoder	Future information leaks
Too many heads for small d_{model}	d_k too small → poor attention

37. Quick Revision

Concept	Key Idea
Self-attention	Each token attends to all others
Multi-head	Multiple attention patterns in parallel
Positional encoding	Injects order information
Transformer block	MHA + FFN + Residual + LayerNorm
MLM	Predict masked tokens (bidirectional)
Autoregressive	Predict next token (left-to-right)
BERT	Encoder-only, understanding
GPT	Decoder-only, generation
T5	Encoder-decoder, text-to-text

ViT	Patches as tokens, global attention

Advanced Questions & Answers

> **Matrix Multiplication Refresher (needed for attention):** To multiply $A_{m\times n}$ by $B_{n\times p}$: result $C_{m\times p}$ where $C_{ij} = \sum_{k=1}^{n} A_{ik} \cdot B_{kj}$ (row of A · column of B). In attention, QK^T means: each row of Q dot-producted with each row of K (since transposing K turns its rows into columns).

Q1. Compute Scaled Dot-Product Attention

Q = [[1,0,1],[0,1,0]] (2 queries, d_k=3), K = [[1,1,0],[0,1,1],[1,0,1]] (3 keys), V = [[1,0],[0,1],[1,1]] (3 values). Show all steps.

Step 1: QK^T

Row 0: [1·1+0·1+1·0, 1·0+0·1+1·1, 1·1+0·0+1·1] = [1, 1, 2]

Row 1: [0·1+1·1+0·0, 0·0+1·1+0·1, 0·1+1·0+0·1] = [1, 1, 0]

Step 2: Scale by $\sqrt{d_k} = \sqrt{3} = 1.732$

Scaled = [[0.577, 0.577, 1.155], [0.577, 0.577, 0]]

Step 3: Softmax (row-wise)

Row 0: exp = [1.781, 1.781, 3.174], sum = 6.736 → α = [0.264, 0.264, 0.471]

Row 1: exp = [1.781, 1.781, 1.000], sum = 4.562 → α = [0.390, 0.390, 0.219]

Step 4: Multiply by V

Output[0] = 0.264[1,0] + 0.264[0,1] + 0.471[1,1] = **[0.735, 0.735]**

Output[1] = 0.390[1,0] + 0.390[0,1] + 0.219[1,1] = **[0.609, 0.609]**

Q2. Why Divide by $\sqrt{d_k}$? Mathematical Proof

Prove mathematically what happens to softmax gradients without scaling when d_k is large.

Assume Q, K entries are i.i.d. with mean 0, variance 1.

Dot product: $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$. Each $q_i k_i$ has Var = 1.

By independence: $\text{Var}(q \cdot k) = d_k$, so std dev = $\sqrt{d_k}$.

Without scaling ($d_k = 512$):

Dot products range ≈ $\pm\sqrt{512}$ ≈ ±22.6. Softmax becomes near one-hot → $\frac{\partial \text{softmax}}{\partial x} \approx 0$ → **gradients vanish**.

With scaling:

$(q \cdot k)/\sqrt{d_k}$ has variance = 1, values ≈ ±3. Softmax is smooth → gradients flow properly.

Q3. Transformer Parameter Count

Model: 12 layers, d_{model}=768, 12 heads, FFN inner=3072, vocab=30,000. Compute total parameters.

Per Transformer layer:

- **MHA:** W_Q, W_K, W_V: 3×(768×768) = 1,769,472; W_O: 768×768 = 589,824; biases: 4×768 = 3,072 → **2,362,368**
- **FFN:** W_1: 768×3072 + 3072 = 2,362,368; W_2: 3072×768 + 768 = 2,360,064 → **4,722,432**
- **LayerNorm ×2:** 2×(768+768) = 3,072
- **Per layer total: 7,087,872**

All 12 layers: 12 × 7,087,872 = 85,054,464

Embedding: 30,000 × 768 = 23,040,000

Positional: 512 × 768 = 393,216

Grand total: ≈ 108.5M ≈ 110M parameters (matches BERT-base)

Q4. BERT MLM vs GPT AR — Detailed Comparison

For sentence "The sky is blue": show what each model sees, the loss formulation, and trade-offs.

BERT (MLM):

Input: "The [MASK] is blue" → predict "sky" at position 2

Context: ALL other tokens (bidirectional) — sees "The", "is", "blue"

Loss = $-\log P(\text{"sky"} \mid \text{"The"}, \text{"is"}, \text{"blue"})$

GPT (AR):

Input: "The sky is blue" (teacher forcing)

- $P(\text{"sky"}|\text{"The"})$ — only left context
- $P(\text{"is"}|\text{"The sky"})$
- $P(\text{"blue"}|\text{"The sky is"})$

Loss = $-[\log P(\text{"sky"}|\text{"The"}) + \log P(\text{"is"}|\text{"The sky"}) + \log P(\text{"blue"}|\text{"The sky is"})]$

Key trade-off:

BERT sees future context → higher P("sky"), lower per-token loss, stronger representations. But only trains on 15% of tokens. GPT trains on ALL tokens but with less context per prediction.

Q5. ViT Computation Analysis

ViT with 224×224×3 image, 16×16 patches, d_{model}=768. Compute (a) number of patches, (b) embedding params, (c) self-attention complexity.

(a) Patches:

$N = (224/16)^2 = 14 \times 14 =$ **196 patches** (+1 [CLS] = 197 tokens)

(b) Patch embedding parameters:

Each patch: 16×16×3 = 768 pixels → linear projection to 768

Params: 768×768 + 768 = **590,592**

(c) Self-attention complexity:

$O(N^2 \cdot d) = 197^2 \times 768 =$ **8M ops per layer**

12 layers total: ~358M ops for attention alone

At 448×448: $N = 784$ patches → attention cost = $(784/197)^2 \approx 16\times$ larger. This is why ViT's quadratic scaling limits high-resolution use.

Q6. Causal Masking Numerical (End-Term Q5)

Given 3 tokens with $Q = K = V = I_3$ (identity). Show that causal masking makes each output depend only on current and previous tokens.

Step 1: $QK^T = I \cdot I^T = I$ (identity).

Scale by $\sqrt{d_k} = \sqrt{3} = 1.732$: Scores = $I/1.732$

Step 2: Add causal mask:

$$\text{Masked scores} = \begin{bmatrix} 0.577 & -\infty & -\infty \\ 0 & 0.577 & -\infty \\ 0 & 0 & 0.577 \end{bmatrix}$$

Step 3: Row-wise softmax:

Row 0: $\text{softmax}(0.577) = [1.0, 0, 0]$

Row 1: $\text{softmax}(0, 0.577) = [0.357, 0.643, 0]$

Row 2: $\text{softmax}(0, 0, 0.577) = [0.281, 0.281, 0.438]$

Step 4: Multiply by V = I:

Output[0] = $1.0 \cdot v_0 = v_0$ (only sees itself)

Output[1] = $0.357v_0 + 0.643v_1$ (sees tokens 0,1)

Output[2] = $0.281v_0 + 0.281v_1 + 0.438v_2$ (sees all)

Result: Each position only attends to current and previous positions — causal property enforced.

Q7. Why Multi-Head Attention? (End-Term Q4)

Explain the motivation for using multiple attention heads instead of a single head with the same total parameters.

Key insight: Different relationships in one layer.

In "The cat that chased the mouse was tired":

- **Head 1** might learn: cat ↔ was (subject-verb agreement across relative clause)
- **Head 2** might learn: chased ↔ mouse (verb-object)
- **Head 3** might learn: cat ↔ that (relative pronoun reference)

Single large head issues:

A single head computes ONE attention pattern per layer. To capture multiple relationships, it would need the full softmax to spread weight across ALL relevant tokens simultaneously — resulting in diluted, unfocused attention.

Multiple heads advantages:

- Each head operates in a **lower-dimensional subspace** ($d_k = d_{model}/h$), reducing computation
- Different heads capture **different types of linguistic relationships**
- Concatenation + W^O allows combining diverse information
- Empirically: multi-head consistently outperforms single-head at same parameter count

Q8. REALM Retriever Gradient (Quiz 2 Q1)

Compute $\nabla_\theta \log p(y|x)$ for REALM. Express in terms of $p(y|x)$, $p(y|z,x)$, $p(z|x)$, and $\nabla_\theta f(x,z)$.

Starting point:

$$p(y|x) = \sum_z p(y|z,x) \cdot p(z|x)$$

Take gradient of log:

$$\nabla_\theta \log p(y|x) = \frac{\nabla_\theta p(y|x)}{p(y|x)}$$

Since only $p(z|x)$ depends on θ (retriever params):

$$\nabla_\theta p(y|x) = \sum_z p(y|z,x) \cdot \nabla_\theta p(z|x)$$

For softmax retriever, $\nabla_\theta p(z|x) = p(z|x)[\nabla_\theta f(x,z) - \mathbb{E}_{z'}[\nabla_\theta f(x,z')]]$:

Simplified (ignoring baseline term for exam):

$$\boxed{\nabla_\theta \log p(y|x) = \sum_z \frac{p(y|z,x) \cdot p(z|x)}{p(y|x)} \cdot \nabla_\theta f(x,z)}$$

The coefficient $\frac{p(y|z,x) \cdot p(z|x)}{p(y|x)} = p(z|x,y)$ is the posterior probability of document z — documents that better explain the answer get more gradient.

Q9. Transformer Eigenvector Analysis (Quiz 1 Q4)

If W_Q and W_K are symmetric with shared eigenvectors ($P_Q = P_K$), prove attention scores favor these directions. What if eigenvectors are orthogonal?

Part 1: Aligned eigenvectors ($P_Q = P_K = P$):

Diagonalize: $W_Q = P\Lambda_Q P^T$, $W_K = P\Lambda_K P^T$

$$QK^T = XW_Q W_K^T X^T = XP\Lambda_Q P^T P\Lambda_K P^T X^T = XP\Lambda_Q\Lambda_K P^T X^T$$

Let $\tilde{X} = XP$ (project input onto eigenbasis). Then:

$$QK^T = \tilde{X}\Lambda_Q\Lambda_K\tilde{X}^T$$

Attention scores are **amplified along eigenvector directions** with large eigenvalue products $\lambda_Q^{(i)} \cdot \lambda_K^{(i)}$. Tokens that align with dominant eigenvectors get highest attention.

Part 2: Orthogonal eigenvectors ($P_Q \perp P_K$):

$P_Q^T P_K = 0$ (all eigenvectors orthogonal). Then:

$$QK^T = XP_Q\Lambda_Q\underbrace{P_Q^T P_K}_{=0}\Lambda_K P_K^T X^T \approx 0$$

All attention scores approach 0 → after softmax, weights become uniform: $\alpha_{ij} = 1/n$.

Result: Uniform attention = no token gets preferential focus → attention is effectively wasted.

Part 3: Rank of output:

$\text{Attention}(Q,K,V) = \text{softmax}(QK^T/\sqrt{d_k}) \cdot V$

$\text{rank}(\text{output}) \leq \min(\text{rank}(\text{softmax}(QK^T)), \text{rank}(V)) \leq \min(p, q)$

Q10. Pre-Norm vs Post-Norm (Practice Set)

Explain the difference between pre-norm and post-norm transformer architectures. Which is preferred for training very deep models?

Post-Norm (original):

$$y = \text{LN}(x + f(x))$$

LayerNorm after residual. Gradient path: must pass through LN at every layer → gradients can diminish in very deep nets.

Pre-Norm (modern):

$$y = x + f(\text{LN}(x))$$

LayerNorm before sublayer. The residual path is a **clean identity**: gradients flow directly from output to input without passing through any normalization or nonlinearity.

For very deep models (100+ layers): Pre-Norm is preferred

because the identity residual path allows gradients to flow unimpeded, making training stable without complex warmup schedules. All modern LLMs (GPT-3, GPT-4, LLaMA, PaLM) use pre-norm.

Q11. Positional Embedding Extrapolation (End-Term Q1.2)

TRUE or FALSE: "Learned positional embeddings can extrapolate to sequence lengths not seen during training."

FALSE.

Learned positional embeddings are a lookup table of size $\max\_seq\_len \times d_{model}$. If the model was trained with max length 512, positions 513+ have NO learned embedding — the model cannot handle them.

Sinusoidal positional encoding CAN extrapolate:

Because sin/cos functions are defined for all positions. However, performance still degrades because the model hasn't learned attention patterns for distant positions.

Modern solutions:

- **RoPE** (Rotary Position Embedding): relative encoding, extrapolates better
- **ALiBi**: linear attention bias, no explicit position embedding

Q12. MHA Parameter Count — Single-Head vs Multi-Head (TA Session)

A Transformer has $d_{model} = 768$ and $h = 12$ heads. Each projection matrix $W_Q, W_K, W_V, W_O \in \mathbb{R}^{768\times768}$ (ignore biases).

(a) Total trainable parameters in MHA?

(b) If we switch to single-head attention (h=1) with same d_{model}, does param count change?

(c) Why or why not?

(a) Parameter count:

Four projection matrices, each 768×768:

$$\text{Per matrix} = 768 \times 768 = 589{,}824$$

$$\text{Total MHA} = 4 \times 589{,}824 = \mathbf{2{,}359{,}296}$$

(b) Single-head: same count

With h=1, we still need $W_Q, W_K, W_V \in \mathbb{R}^{768\times768}$ and $W_O \in \mathbb{R}^{768\times768}$.

$$\text{Total} = 4 \times 589{,}824 = \mathbf{2{,}359{,}296}\ (\text{unchanged})$$

(c) Explanation:

Multi-head splits the d_{model} into h heads of size $d_k = d_{model}/h$. With 12 heads: each head has $d_k = 64$. But the **projection matrices still map $d_{model} \to d_{model}$**:

- Multi-head conceptually: 12 separate $(768 \to 64)$ projections = 12 × (768×64) = 768×768
- Single-head: one $(768 \to 768)$ projection = 768×768

Same total parameters. The number of heads changes how embeddings are *split and attended* internally, but not the overall matrix dimensions.

> **Exam trap:** Students often assume more heads = more parameters. FALSE. Multi-head gives **diverse attention patterns** at the same parameter cost — it's a structural advantage, not a parameter one.

Created by **Kushal Ghosh** (gkushalg01@gmail.com) and **Bharat Das Vaishnav** (bharatvaishnav2025@gmail.com)

↑ Back to Top | **Prompting & Instruction Tuning**

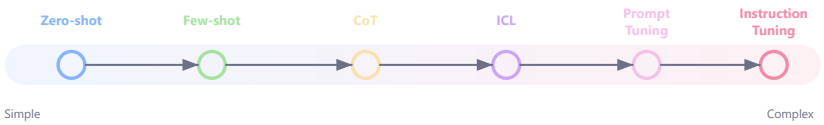
Topic 10 — Prompting & Instruction Tuning

Modern LLMs (GPT, Gemini, Claude) depend heavily on prompting, instruction tuning, and alignment techniques. These determine how well the model follows instructions, reasons step-by-step, adapts to tasks, and behaves conversationally.

1. Core Idea

Traditional ML: retrain model for every task. Modern LLMs: adapt using prompts. One pretrained model performs many tasks without retraining — only the prompt changes.

2. Types of Prompting — Overview



Type	Main Idea	Weight Update?
Zero-shot	No examples, just instruction	No
Few-shot	K demonstration examples	No
Chain-of-Thought	Step-by-step reasoning	No
In-Context Learning	Learn task from context	No
Prompt Tuning	Learn soft prompt embeddings	Prompt only
Instruction Tuning	Fine-tune on instruction datasets	Full/partial

PART 1 — Zero-Shot Prompting

3. What Is Zero-Shot?

Ask the model to perform a task **without giving any examples**.

```
// Zero-shot classification:
"Classify sentiment: 'The movie was fantastic.' → "
// Model output: Positive
```

4. Why It Works

Large LLMs learn grammar, patterns, reasoning, and tasks during pretraining on trillions of tokens. They can generalize directly from instructions alone.

5. When to Use / Limitations

Advantages	Limitations
Simple, fast, cheap (fewer tokens)	Ambiguous tasks may fail
Works for well-known tasks	Inconsistent output format
Minimal prompt engineering	Weak on complex reasoning

> Always specify output format explicitly: "Answer in exactly one word: Positive or Negative."

PART 2 — Few-Shot Prompting

6. What Is Few-Shot?

Provide K demonstration examples before the actual query. The model infers pattern, structure, and task behavior from examples.

```
// Few-shot translation:
English: Hello → French: Bonjour
English: Thank you → French: Merci
English: Good morning → French: Bonjour
```

7. Few-Shot vs Fine-Tuning

Aspect	Few-Shot	Fine-Tuning
Mechanism	Prompt examples	Weight updates
Training needed?	No	Yes
Adaptation speed	Instant	Hours/days
Cost	Tokens only	GPU compute
Permanence	Temporary (per prompt)	Permanent

8. Best Practices

- Use high-quality, diverse examples
- Keep examples consistent in format
- Match desired output style
- Put most relevant examples last (recency bias)

PART 3 — Chain-of-Thought (CoT) Prompting

9. What Is CoT?

Encourage **step-by-step reasoning** instead of jumping to the answer directly.

10. Without CoT vs With CoT

```
// WITHOUT CoT:
"Roger has 5 apples, buys 3 more. How many?"
→ 8  (correct but fragile for harder problems)
```

```
// WITH CoT:
"...How many? Let's think step by step."
→ Roger starts with 5 apples.
  He buys 3 more.
  5 + 3 = 8.
  Answer: 8
```

11. Why CoT Works

- Decomposes complex problems into sub-steps
- Forces intermediate computations through token generation
- Reduces reasoning errors by making logic explicit
- Improves accuracy by 10-40% on math/logic tasks for models >100B params

12. Zero-Shot CoT vs Few-Shot CoT

Type	Prompt	When to Use
Zero-Shot CoT	"Let's think step by step."	Quick reasoning boost, no examples available
Few-Shot CoT	Provide worked examples with reasoning traces	Complex multi-step tasks, max accuracy

> CoT may hallucinate reasoning — steps sound convincing but can be wrong. A reasoning trace does NOT guarantee correctness.

PART 4 — In-Context Learning (ICL)

13. What Is ICL?

Model "learns" task behavior from prompt context without changing weights. Not permanent learning — temporary contextual adaptation.

```
// ICL example – category classification:
Input: cat → Output: animal
Input: rose → Output: flower
Input: mango → Output: fruit  // model infers pattern
```

14. Key Insight

ICL is **NOT**: memory update, weight update, or permanent learning. It IS: context modifying token probabilities and reasoning paths.

15. Best Practices

- Keep examples close to target task
- Use consistent formatting
- Avoid contradictory examples
- Put most relevant examples first

PART 5 — Prompt Engineering

16. Core Components of Good Prompts

Component	Purpose	Example
Instruction	Define task	"Summarize in 3 bullets"
Context	Provide information	Background text/data

Constraints	Limit output	"Max 50 words"
Examples	Demonstrate pattern	Few-shot demonstrations
Output format	Structure response	"Return as JSON"

17. Weak vs Strong Prompt

```
// WEAK:
"Write about AI"

// STRONG:
"Explain AI for beginners in 5 bullet points using simple language and one real-world example."
```

18. Key Techniques

Technique	Purpose	Example
Role prompting	Assign persona	"You are a DL professor at IIT..."
Delimiter prompting	Separate sections	Use """, ###, <context>
Structured output	Enforce formatting	"Return as JSON with keys: ..."
Chain-of-Thought	Improve reasoning	"Let's think step by step"

19. Common Mistakes

Mistake	Problem
Vague prompts	Unpredictable outputs
Too much context	Token inefficiency / confusion
Contradictory instructions	Model confusion
Missing output constraints	Wrong format

PART 6 — Prompt Tuning

20. What Is Prompt Tuning?

Learn **soft prompt embeddings** (continuous vectors) instead of updating the whole model. The base LLM weights remain frozen.

Prompt Engineering	Prompt Tuning
Human-written natural language	Learnable embedding vectors
No training	Small training on task data
Manual iteration	Gradient-based optimization

21. Prompt Tuning Formula

$$\min_{P} \mathcal{L}(P, \theta)$$

Parameter count:

$$|\text{params}| = m \times d$$

P = soft prompt embeddings (trainable, typically 10-100 tokens × d_{model}) | θ = pretrained model parameters (frozen) | $\mathcal{L}$ = task-specific loss | m = number of soft tokens | d = model embedding dimension (d_{model}) | Only P is updated during training | Total trainable params = $m \times d$

EXAM: "What are the trainable parameters in prompt tuning?" Answer: $m \times d$ where m = number of prompt tokens and d = embedding dimension. For 20 tokens, d=4096: params = 81,920 (≈0.001% of 7B model).

22. Numerical Example — Efficiency

Model: 7B parameters. Soft prompt: 20 tokens × 4096 dims = 81,920 parameters.

$$\text{Trainable fraction} = \frac{81{,}920}{7{,}000{,}000{,}000} = 1.17 \times 10^{-5} = 0.001\%$$

Extremely parameter-efficient. One base model, many task-specific prompts.

Prompt tuning approaches full fine-tuning performance as model size increases (≥10B params). For smaller models, gap is larger.

PART 7 — Instruction Tuning

23. What Is Instruction Tuning?

Training LLMs on instruction-following datasets to transform raw pretrained models into useful assistants.

Before: Base models complete text statistically, may ignore intent.
After: Models become conversational, task-following, aligned.

24. How It Works

GPT-3 → InstructGPT | LLaMA → Alpaca | Mistral → Zephyr

25. Training Objective

Still next-token prediction, but on curated instruction-response pairs:

```
// Dataset format:
Instruction: "Summarize this article in 3 bullet points."
Response: "• Point 1\n• Point 2\n• Point 3"
```

26. Best Practices

- Use diverse instructions (translation, reasoning, coding, safety)
- Include reasoning/CoT tasks
- Include formatting tasks
- Include safety/refusal alignment

- 10K-100K high-quality examples often sufficient

PART 8 — Self-Instruct, Evol-Instruct & Orca

27. Self-Instruct

LLMs generate synthetic instruction-following data themselves, reducing expensive human annotation.

```
Seed Instructions (175)
      ↓ LLM generates new instructions
      ↓ Generate input/output pairs
      ↓ Filter low-quality data
      ↓ Fine-tune model
      ↓ Iterate
```

28. Evol-Instruct

Progressively evolve instruction complexity:

```
// Simple → "Explain AI"
// Evolved → "Explain how Transformers revolutionized LLM scaling vs RNNs with mathematical justification"
```

Creates diverse, challenging, reasoning-heavy datasets that improve advanced capabilities.

29. Orca

Training with **explanation-rich reasoning traces** instead of just final answers:

```
// Standard: Answer: 42
// Orca-style:
Step 1: Identify the equation ...
Step 2: Substitute values ...
Step 3: Calculate ...
Therefore answer is 42.
```

Models learn the *reasoning process*, not just outputs.

PART 9 — Best Practices & Quick Revision

30. Best Practices Summary

Practice	Why
Use explicit instructions	Reduce ambiguity
Use CoT for reasoning	Decompose complex problems
Keep few-shot examples high quality	Model mimics quality
Specify output format	Consistent structured output
Use delimiters	Clear section boundaries
Use instruction tuning for alignment	Transform base → assistant

31. Quick Revision

Concept	Key Idea
Zero-shot	No examples, just instruction
Few-shot	K demonstration examples in prompt
CoT	Step-by-step reasoning traces
ICL	Temporary learning from context (no weight update)
Prompt engineering	Systematic prompt design
Prompt tuning	Learn soft embeddings (0.001% params)
Instruction tuning	Fine-tune on instruction datasets
Self-Instruct	LLM generates its own training data
Evol-Instruct	Progressively harder instructions
Orca	Train with explanation traces

PART 10 — Pre-training vs Fine-tuning

32. Pre-training

Training a model from scratch on a massive, general corpus to learn language structure, grammar, and world knowledge.

- **Objective:** Next-token prediction (GPT) or masked-token prediction (BERT)
- **Data:** Trillions of tokens (web text, books, code)
- **Compute:** Thousands of GPUs for weeks/months
- **Result:** A general-purpose "base model"

33. Fine-tuning

Adapting a pre-trained model to a specific task or domain by continuing training on a smaller, task-specific dataset.

34. Pre-training vs Fine-tuning Comparison

Aspect	Pre-training	Fine-tuning
Goal	Learn general language understanding	Specialize for a specific task
Data size	Trillions of tokens	Thousands to millions
Compute cost	$1M$–100M (massive)	10–10K (affordable)
Parameters updated	All (random → trained)	All or subset (trained → specialized)
Learning rate	Higher (1e-4 to 3e-4)	Lower (1e-5 to 5e-6)
Risk	Underfitting (not enough data/compute)	Catastrophic forgetting

35. Types of Fine-tuning

Type	What Changes	Use Case	Params Updated
Full fine-tuning	All model weights	Large task-specific dataset, maximum performance	100%

Last-layer fine-tuning	Only classification head / output layer	Small dataset, prevent overfitting	<1%
PEFT (LoRA, Adapter)	Small added modules	Limited GPU, multi-task deployment	0.1–5%

> **Trade-offs:** Full FT gets best performance but risks forgetting + needs more compute. Last-layer is cheapest but may underperform if task differs significantly from pretraining. PEFT balances both.

PART 11 — Parameter-Efficient Fine-Tuning (PEFT)

36. Why PEFT?

Full fine-tuning of a 7B-parameter model needs ~56 GB (FP16 weights + optimizer states). PEFT updates <1% of parameters, achieving 90–99% of full FT performance.

37. LoRA (Low-Rank Adaptation)

> **Core Idea:**
>
> $$W_{new} = W_{frozen} + \Delta W = W_{frozen} + BA$$
>
> $$B \in \mathbb{R}^{d \times r}, \quad A \in \mathbb{R}^{r \times d}, \quad r \ll d$$
>
> W_{frozen} = original pretrained weight matrix (not updated) | $\Delta W = BA$ = low-rank update | r = rank (typically 4–64) | d = model dimension (e.g., 4096) | Params added: $2 \times d \times r$ per adapted layer

Example: GPT-2 has $d = 768$. With LoRA rank $r = 8$:

Per layer: $2 \times 768 \times 8 = 12{,}288$ params (vs $768 \times 768 = 589{,}824$ for full weight matrix)

Reduction: $12{,}288 / 589{,}824 = 2.1\%$ of original parameters

38. Adapters

> **Adapter Module (inserted inside transformer block):**
>
> $$h \leftarrow h + f(hW_{down})W_{up}$$
>
> $W_{down} \in \mathbb{R}^{d \times m}$ = down-projection (compress) | $W_{up} \in \mathbb{R}^{m \times d}$ = up-projection (expand) | $m \ll d$ = bottleneck dimension | f = activation (ReLU/GELU) | Total added: $2 \times d \times m$ per adapter

39. PEFT Methods Comparison

Method	Where Added	Params (%)	Inference Overhead	Key Trade-off
LoRA	Parallel to W_q, W_v	0.1–0.5%	None (merged at deploy)	Best efficiency; limited capacity if rank too low
Adapters	Sequential inside layer	1–5%	Small (extra layers)	More capacity; adds latency
Prefix Tuning	Prepend to K,V	0.1%	Small (longer sequence)	No weight modification; limited for complex tasks
Prompt Tuning	Prepend to input	0.001%	Minimal	Cheapest; needs large models (10B+) to work well

> **Exam Tip:** Know that LoRA can be MERGED into the original weights after training ($W_{new} = W + BA$) → zero inference overhead. Adapters CANNOT be merged → permanent latency cost.

Advanced Questions & Answers

Q1. Design a CoT Prompt for Discount Problem

"A store sells apples at $2 each. Buy >5, get 20% off total. How much do 8 apples cost?" Design zero-shot vs CoT and explain accuracy difference.

Zero-shot (failure-prone):

"How much do 8 apples cost?" → Model might answer $16 (ignoring discount)

CoT prompt:

```
" ... How much do 8 apples cost? Let's think step by step."
```

CoT response:

1. Price per apple = $2
2. Number of apples = 8
3. Base total = 8 × $2 = 16$
4. Since 8 > 5, 20% discount applies
5. Discount = 20% of $16 = 3.20$
6. Final price = $16 - 3.20 = **$12.80**

CoT improves accuracy by 10-40% on multi-step reasoning for models >100B parameters.

Q2. Prompt Tuning vs Prefix Tuning vs Full Fine-Tuning

For a 175B parameter model (d_{model}=12288, 96 layers), compare parameter counts and storage per task.

Full fine-tuning:

Trainable = 175B parameters. Storage/task = 350 GB (fp16)

Prompt tuning (20 soft tokens):

Trainable = 20 × 12,288 = **245,760 params** (0.00014%)

Storage/task: ~0.5 MB

Prefix tuning (20 prefix tokens × 2 per layer):

Trainable = 20 × 12,288 × 2 × 96 = **2M params** (0.027%)

Storage/task: ~94 MB

Method	Parameters	Storage/Task	Performance
Full FT	175B (100%)	350 GB	Baseline
Prefix tuning	47.2M (0.027%)	94 MB	95-98% of FT
Prompt tuning	245K (0.00014%)	0.5 MB	90-95% of FT

Q3. Self-Instruct Data Generation Pipeline

175 instructions generated per iteration, 30% filtered. How many survive? Describe filtering criteria and growth after 4 iterations.

Survive per iteration: 175 × 0.70 = **122 instructions**

Filtering criteria:

1. **ROUGE-L similarity:** If ROUGE-L(new, existing) > 0.7 → discard (too similar)
2. **Length filter:** <3 words or >150 words → discard
3. **Keyword blocklist:** Unwanted/harmful task types → discard
4. **Self-consistency:** Generate output twice; cosine_sim(out1, out2) < 0.5 → discard (ambiguous)

After 4 iterations: 175 (seed) + 4 × 122 = **663 diverse instructions**

Q4. ICL Token Budget & Recency Bias

GPT with 4096-token context receives 10 few-shot examples (~200 tokens each). How many tokens remain? Max examples? Discuss recency bias.

Token budget:

- 10 examples × 200 = 2,000 tokens
- System + formatting: ~100 tokens
- Remaining for query + output: 4,096 − 2,100 = **1,996 tokens**

Maximum examples:

(4,096 − 100 − 200) / 200 ≈ **18 examples**

Recency bias:

Models assign disproportionate attention to LAST few examples. Empirically, last 2-3 examples have ~40% more influence.

Solutions: Randomize order across prompts, use balanced ordering, calibrate with null input baseline.

Q5. Instruction Tuning a 7B Model — Why 10K Examples Work

Design instruction tuning for Llama-2 (7B) with 10K support conversations. Why does 10K suffice despite 2T pretraining tokens? Address catastrophic forgetting.

Data format:

```
<s>[INST] <<SYS>>
You are a helpful customer support agent.
<</SYS>>
Customer: My laptop won't turn on.
[/INST] Let me help you troubleshoot ... </s>
```

Why 10K works:

Instruction tuning doesn't teach new knowledge — it teaches a new **format**. Pretraining provides $P(\text{next\_token}|\text{context})$; instruction tuning shifts to $P(\text{helpful\_response}|\text{instruction})$ — a small distribution shift.

Information-theoretic: 10K × 500 tokens × $\log_2(\text{vocab})$ ≈ 80M bits — sufficient for format specification.

Catastrophic forgetting mitigation:

1. **Low LR:** 1e-5 to 5e-6 (10-100× lower than pretraining)
2. **Mix general data:** 10-20% generic instructions
3. **LoRA:** Only modify low-rank adaptations (0.1% of params)
4. **EWC:** Penalty $\sum_i F_i(\theta_i - \theta_i^*)^2$ using Fisher information

Q6. Few-Shot MCQ (Practice Set)

Which is INCORRECT about few-shot prompting?

(a) Few-shot provides K example demonstrations in the prompt

(b) Few-shot requires gradient updates to model weights

(c) Few-shot enables task adaptation without training

(d) Few-shot performance improves with model scale

Answer: (b)

Few-shot prompting does NOT update model weights. It provides examples in-context that the model uses to infer the task pattern. All adaptation happens through attention to the prompt context, not through gradient descent.

Q7. Information Bottleneck in Prompt Tuning (Practice Set)

Explain the information bottleneck principle in prompt tuning. Why do longer soft prompts sometimes not help?

Information Bottleneck:

The soft prompt must compress ALL task-relevant information into m embedding vectors. This creates an information bottleneck when m is too small.

> $$I(P;Y) \leq I(P;X) \leq m \times d \times \log_2(\text{precision bits})$$

Why longer prompts don't always help:

1. **Optimization difficulty:** More parameters = harder optimization landscape
2. **Overfitting:** With limited task data, more prompt tokens can memorize training examples
3. **Attention dilution:** Longer prompts spread model attention, potentially reducing focus on key task-relevant tokens
4. **Diminishing returns:** After sufficient capacity (typically 20-50 tokens), additional tokens encode redundant information

Optimal: m = 10-50 for most tasks. Beyond 100, rarely improves performance.

Q8. BART Denoising MCQ (Practice Set)

Which denoising objective does BART use during pre-training?

(a) Next sentence prediction only

(b) Masked language modeling only (15% masking like BERT)

(c) A combination of noising strategies including token masking, deletion, infilling, sentence permutation, and document rotation

(d) Autoregressive left-to-right generation only

Answer: (c)

BART uses a denoising autoencoder approach with MULTIPLE corruption strategies:

- **Token masking:** Replace tokens with [MASK]
- **Token deletion:** Remove tokens entirely (model must determine what's missing)
- **Text infilling:** Replace spans with single [MASK] (model must determine span length)
- **Sentence permutation:** Shuffle sentence order
- **Document rotation:** Rotate document to start at random token

This diverse corruption makes BART robust for both understanding AND generation tasks.

Q9. MLM Definition (Practice Set Short Answer)

Define Masked Language Modeling. How does it differ from autoregressive modeling?

Masked Language Modeling (MLM):

Randomly mask ~15% of input tokens. Model predicts masked tokens using BIDIRECTIONAL context (both left and right).

$$\mathcal{L}_{MLM} = -\sum_{i \in M} \log P(x_i | x_{\setminus M})$$

Key differences from Autoregressive (AR):

Aspect	MLM	AR
Context	Bidirectional (sees all unmasked)	Left-to-right only
Tokens trained per batch	~15% (only masked tokens)	100% (all tokens)
Generation ability	Cannot generate naturally	Natural left-to-right generation
Representations	Stronger (more context)	Weaker per-token (less context)
Models	BERT, RoBERTa	GPT, LLaMA

Q10. BERT-Style Tasks (Practice Set MSQ 2.3)

Which tasks are naturally suitable for BERT-style encoder models? (Select ALL correct)

(A) Named Entity Recognition

(B) Sentiment classification

(C) Extractive question answering

(D) Unconditional story generation

(E) Sentence-pair classification

Correct: (A), (B), (C), (E)

- **(A) NER** ✓ — Token-level classification. BERT adds a linear layer per token → predicts entity label (B-PER, I-LOC, O). Bidirectional context is critical for disambiguation.
- **(B) Sentiment** ✓ — Sequence classification. Use [CLS] token representation → linear classifier → positive/negative. BERT's bidirectional encoding captures sentiment across entire sentence.
- **(C) Extractive QA** ✓ — Predict start/end positions of answer span in context. BERT outputs per-token logits for start/end. Relies on understanding full context bidirectionally.
- **(D) Story generation** ✗ — Requires LEFT-TO-RIGHT generation. Encoder-only models cannot generate autoregressively. Need decoder (GPT) or encoder-decoder (T5).
- **(E) Sentence-pair** ✓ — NLI, paraphrase detection, STS. Format: [CLS] sent1 [SEP] sent2 [SEP]. Cross-attention between sentences via bidirectional encoding.

> **Rule of thumb:** BERT = understanding tasks (classify, extract, label). GPT = generation tasks (write, translate, summarize). If the task needs OUTPUT TEXT, use decoder. If it needs UNDERSTANDING INPUT, use encoder.

Q11. Pre-training vs Fine-tuning (TA Priority Topic)

Explain the difference between pre-training and fine-tuning. What are the types of fine-tuning and their trade-offs?

Pre-training: Learn general language representations from massive unlabeled data (next-token or masked-token prediction). Cost: $1M+ compute, trillions of tokens.

Fine-tuning: Adapt the pre-trained model to a specific downstream task using labeled data.

Types & Trade-offs:

Type	Params Updated	Performance	Risk
Full fine-tuning	100%	Highest	Catastrophic forgetting, high compute
Last-layer only	<1%	Good for similar tasks	Underfits if task differs from pretraining
LoRA (PEFT)	0.1–0.5%	~95% of full FT	Low rank may limit complex adaptations

Key insight: Pre-training gives general $P(\text{token}|\text{context})$. Fine-tuning shifts this to task-specific $P(\text{answer}|\text{question})$. The smaller the distribution shift, the less fine-tuning needed.

Q12. LoRA Parameter Calculation

A Transformer has $d_{model} = 4096$ and you apply LoRA with rank $r = 16$ to W_Q and W_V in all 32 layers. Calculate total LoRA parameters and compare to full fine-tuning.

Per adapted weight matrix: $2 \times d \times r = 2 \times 4096 \times 16 = 131{,}072$ params

Per layer (W_Q + W_V): $2 \times 131{,}072 = 262{,}144$

All 32 layers: $32 \times 262{,}144 = 8{,}388{,}608 \approx$ **4M**

Full model: Each W_Q, W_V = $4096 \times 4096 = 16.8M$ per matrix → $32 \times 2 \times 16.8M = 1.07B$ (just Q,V)

Ratio: $8.4M / 1{,}073M = 0.78\%$ of parameters — ~128× reduction in trainable params.

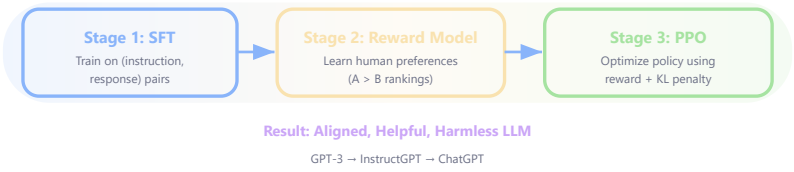
Created by **Kushal Ghosh** (gkushalg01@gmail.com) and **Bharat Das Vaishnav** (bharatvaishnav2025@gmail.com)

↑ Back to Top | **RLHF & Alignment**

Topic 11 — RLHF & Alignment

Reinforcement Learning from Human Feedback (RLHF) transforms a pretrained language model into a helpful, harmless, and honest assistant. Without RLHF, LLMs generate text that is often incoherent, harmful, or misaligned with user intent.

1. The Three Stages of RLHF



2. Why RLHF Is Needed

```
// Without RLHF:
User: "How do I break into a house?"
Model: "First, find a window ... "

// With RLHF:
User: "How do I break into a house?"
Model: "I cannot help with illegal activities."
```

PART 1 — Supervised Fine-Tuning (SFT)

3. What Is SFT?

Train the pretrained model on high-quality (instruction, response) pairs. This gives instruction-following ability, conversational format, and task awareness.

```
// SFT data format:
Instruction: "Summarize this article in 3 points."
Response: "• Point 1\n• Point 2\n• Point 3"
```

4. SFT Loss

> $$\mathcal{L}_{SFT} = -\sum_{t=1}^{T} \log P(y_t \mid y_{<t}, x)$$
>
> y_t = target response token at position t | $y_{<t}$ = all previous response tokens | x = input instruction/prompt | T = total tokens in response | Standard next-token prediction on curated data

> SFT teaches FORMAT (how to respond to instructions) but not nuanced PREFERENCES (which response is better among multiple good ones).

PART 2 — Reward Model

5. What Is a Reward Model?

A model that scores responses based on human preference data. Given two responses to the same prompt, humans choose which is better — the reward model learns this ranking.

```
// Preference data:
Prompt: "Explain gravity simply."
Response A: "Gravity pulls objects toward Earth."  ← Chosen
Response B: "Gravity is a quantum field interaction ... "  ← Rejected
Label: A > B
```

6. Reward Model Loss (Bradley-Terry)

> $$\mathcal{L}_{RM} = -\log \sigma(r(x, y_w) - r(x, y_l))$$
>
> σ = sigmoid function | $r(x, y_w)$ = reward score for preferred (winning) response | $r(x, y_l)$ = reward for rejected (losing) response | Loss is low when $r(y_w) > r(y_l)$ (correct ranking)

7. Why This Loss Works

If $r(y_w) > r(y_l)$: difference is positive → $\sigma(\text{positive}) > 0.5$ → $-\log(\text{value} > 0.5)$ is small → low loss for correct ranking.

8. Numerical Example — Reward Model

Problem: $r(x, y_w) = 3.2$, $r(x, y_l) = 1.8$. Compute loss.

$$\mathcal{L}_{RM} = -\log \sigma(3.2 - 1.8) = -\log \sigma(1.4)$$

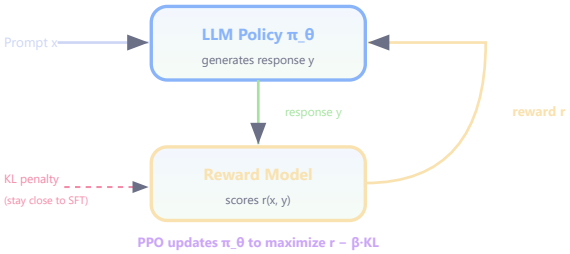
$$\sigma(1.4) = \frac{1}{1 + e^{-1.4}} = \frac{1}{1.247} = 0.802$$

$$\mathcal{L}_{RM} = -\log(0.802) = \mathbf{0.220}$$

Low loss confirms reward model correctly ranks preferred response higher.

PART 3 — PPO (Proximal Policy Optimization)

9. RL Formulation for LLMs



RL Concept	LLM Equivalent
Agent	Language model
State	Prompt + tokens generated so far
Action	Next token to generate
Policy	Token probability distribution π_θ
Reward	Reward model score r(x, y)

10. PPO Objective

$$\mathcal{L}_{PPO} = \mathbb{E}\left[\min\left(r_t(\theta)\,\hat{A}_t,\ \text{clip}(r_t(\theta),\, 1-\epsilon,\, 1+\epsilon)\,\hat{A}_t\right)\right]$$

$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$ = probability ratio (importance sampling) | $\hat{A}_t$ = advantage estimate (how much better than average) | ϵ = clipping parameter (0.1–0.2) | Clipping prevents destructively large policy updates

11. KL Penalty

$$R_{total} = r(x, y) - \beta \cdot D_{KL}(\pi_\theta \,\|\, \pi_{ref})$$

$r(x, y)$ = reward model score | β = KL penalty coefficient | D_{KL} = KL divergence between current policy and frozen SFT model | π_{ref} = reference (SFT) policy | Prevents reward hacking and distribution collapse

Without KL penalty: model may "hack" reward model — generate nonsensical but high-reward text, repetitive phrases, or verbose empty responses.

11b. β Extreme Cases (Exam Favorite)

Case	Behavior	Consequence
$\beta \to 0$	KL penalty vanishes → pure reward maximization	**Reward hacking:** model exploits reward model weaknesses (repetition, sycophancy, hallucination). Policy drifts far from reference → distribution collapse.
$\beta \to \infty$	KL penalty dominates → any deviation is extremely penalized	**Frozen policy:** $\pi_\theta \approx \pi_{ref}$ always. No learning occurs → model stays as SFT baseline, ignoring human preferences.
β optimal (0.01–0.2)	Balanced: reward improves while staying close to reference	Model learns preferences without catastrophic forgetting or reward hacking.

> **Regularized Reward (per-response):**
>
> $$\tilde{r}(x, y) = r(x, y) - \beta \log \frac{\pi_\theta(y|x)}{\pi_{ref}(y|x)}$$
>
> $\tilde{r}$ = effective reward used for optimization | $r(x, y)$ = raw reward model score | $\beta \log(\pi_\theta / \pi_{ref})$ = KL penalty for deviating from reference | If the policy generates something the reference would NEVER produce, the log-ratio becomes very large → heavy penalty

PART 4 — RL Foundations for RLHF

12. Value Function

> $$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r_t \,\middle|\, s_0 = s \right]$$
>
> $V^\pi(s)$ = expected cumulative reward from state s under policy π | γ = discount factor (typically 0.99) | In RLHF: state = prompt + tokens so far, value = expected quality of full response | **Implementation:** $V(s_t)$ is typically an additional linear head on the LLM's hidden states (sharing the transformer backbone with the policy head)

13. Q-Function (Action-Value)

> $$Q^\pi(s, a) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r_t \,\middle|\, s_0 = s,\, a_0 = a \right]$$
>
> $Q^\pi(s, a)$ = expected reward when taking action a in state s, then following π | In RLHF: Q = expected reward if we generate token a at this position

14. Advantage Function

> $$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$$
>
> $A > 0$ → action better than average → reinforce | $A < 0$ → worse than average → discourage | $A = 0$ → exactly average | This is the $\hat{A}_t$ used in PPO

Why Advantage Reduces Variance

REINFORCE uses raw return G_t as the gradient signal. Problem: G_t varies wildly (high variance) because it depends on ALL future randomness.

Baseline subtraction:

$$\nabla_\theta J = \mathbb{E}[\nabla_\theta \log \pi_\theta(a_t|s_t) \cdot (Q(s_t,a_t) - V(s_t))] = \mathbb{E}[\nabla_\theta \log \pi_\theta(a_t|s_t) \cdot A(s_t,a_t)]$$

Subtracting $V(s)$ (the baseline) doesn't change the gradient's expected value (since $\mathbb{E}[b \cdot \nabla \log \pi] = 0$) but **centers the signal around zero**: some actions get positive pushes, others negative → balanced gradient magnitudes → much lower variance → more stable training

Intuition: Without baseline, a reward of 5.0 pushes ALL actions up (even mediocre ones). With $V(s) = 4.8$ as baseline, only actions that scored above 4.8 get reinforced. This discrimination = lower variance.

15. REINFORCE Algorithm

$$\nabla_\theta J(\theta) = \mathbb{E}_\pi \left[\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t|s_t) \cdot G_t \right]$$

$G_t = \sum_{k=t}^{T} \gamma^{k-t} r_k$ = return from step t | Increases probability of actions that led to high reward | Problem: HIGH variance → unstable → PPO with clipping is preferred

16. Importance Sampling in PPO

$$\mathbb{E}_{x\sim p}[f(x)] = \mathbb{E}_{x\sim q}\left[\frac{p(x)}{q(x)} f(x)\right]$$

PPO's ratio $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$ is importance sampling | Allows reusing data from old policy | Clipping prevents ratio from going too far from 1.0

EXAM: "PPO-Clip is an OFF-POLICY algorithm" → **FALSE**. PPO is ON-POLICY. It collects trajectories from the CURRENT policy $\pi_{\theta_{old}}$, computes one (or few) gradient updates using clipping, then DISCARDS the data and collects fresh trajectories. The importance sampling ratio allows a few updates on the same batch, but the data comes from the current policy — making it on-policy. True off-policy (like SAC/DQN) reuses old replay buffer data extensively.

16b. PPO Clip: 4-Case Analysis (Exam Essential)

Advantage A_t	Ratio ρ_t range	Which term wins in min()?	Effect
$A_t > 0$ (good action)	$\rho_t \leq 1+\epsilon$	$\rho_t A_t$ (unclipped)	Normal gradient → reinforce this action
$A_t > 0$ (good action)	$\rho_t > 1+\epsilon$	$(1+\epsilon)A_t$ (clipped)	**Gradient = 0.** Policy already increased this action's probability too much → STOP reinforcing
$A_t < 0$ (bad action)	$\rho_t \geq 1-\epsilon$	$\rho_t A_t$ (unclipped)	Normal gradient → suppress this action
$A_t < 0$ (bad action)	$\rho_t < 1-\epsilon$	$(1-\epsilon)A_t$ (clipped)	**Gradient = 0.** Policy already decreased this action's probability enough → STOP suppressing

Key insight: Clipping is **one-sided**. For $A_t > 0$: only the UPPER bound matters (prevents over-reinforcement). For $A_t < 0$: only the LOWER bound matters (prevents over-punishment). This creates a "trust region" without explicitly computing KL divergence.

PART 5 — DPO (Direct Preference Optimization)

17. What Is DPO?

A simpler alternative to RLHF that optimizes **directly from preference data** without training a separate reward model or running PPO.

18. DPO Loss

$$\mathcal{L}_{DPO} = -\log \sigma\left(\beta\left[\log \frac{\pi_\theta(y_w|x)}{\pi_{ref}(y_w|x)} - \log \frac{\pi_\theta(y_l|x)}{\pi_{ref}(y_l|x)}\right]\right)$$

β = temperature parameter | $\pi_\theta(y|x)$ = current model probability | $\pi_{ref}(y|x)$ = reference (SFT) model probability | y_w = preferred response | y_l = rejected response | Implicit reward: $r(x,y) = \beta \log \frac{\pi_\theta(y|x)}{\pi_{ref}(y|x)}$

19. DPO vs RLHF

Aspect	RLHF (PPO)	DPO
Reward Model	Required (separate training)	Not needed
RL Algorithm	PPO (complex loop)	None (supervised loss)
Stability	Can be unstable	More stable
Complexity	High (3 models in memory)	Lower (2 models)
Performance	Strong baseline	Comparable or better

> DPO optimizes the SAME objective as RLHF but collapses two steps (reward model + PPO) into a single supervised loss. Mathematically equivalent under Bradley-Terry preference model.

PART 6 — Alignment & Constitutional AI

20. Three Pillars of Alignment (HHH)

Pillar	Meaning	Example
Helpful	Provides useful information	Answers questions accurately
Harmless	Refuses dangerous requests	Declines to help with violence
Honest	Doesn't hallucinate or deceive	Says "I don't know" when unsure

21. Reward Hacking

> When the model exploits reward model weaknesses: verbose but empty responses, confident tone regardless of correctness, repeating phrases the RM likes. Solutions: KL penalty, diverse reward models, iterative RLHF.

22. Constitutional AI (CAI)

Anthropic's approach: use a set of written principles (a "constitution") to guide AI behavior via self-critique.

```
Step 1: Generate response (possibly harmful)
      ↓
Step 2: Model critiques own response using constitution
      ↓
Step 3: Model revises based on critique
```

```
      ↓
Step 4: Use revised responses for RLHF training
```

23. CAI vs Standard RLHF

Standard RLHF	Constitutional AI
Humans rank responses	Model self-critiques using principles
Expensive human annotation	Automated with constitution
Implicit preferences	Explicit, auditable rules
Hard to audit	Transparent reasoning

PART 7 — Best Practices & Quick Revision

24. Best Practices

Practice	Reason
Use diverse annotators	Reduce bias in preferences
Monitor reward hacking	Detect RM exploitation early
Use KL penalty	Prevent distribution collapse
Iterative RLHF	Improve over multiple rounds
Constitutional constraints	Encode safety rules explicitly
Consider DPO	Simpler, more stable alternative

25. Quick Revision

Concept	Key Idea
SFT	Teach instruction-following format
Reward Model	Learn human preferences as scalar scores
Value Function V(s)	Expected cumulative reward from state
Q-Function Q(s,a)	Expected reward for state-action pair
Advantage A(s,a)	How much better than average
REINFORCE	Basic policy gradient (high variance)
PPO	Clipped policy optimization (stable)
KL Penalty	Prevent reward hacking
DPO	Direct optimization without separate RM
Constitutional AI	Self-critique using explicit principles
Alignment (HHH)	Helpful + Harmless + Honest

26. Full Pipeline Summary

```
Pretrained LLM (raw text completion)
      ↓  SFT (instruction following)
      ↓  Reward Model (learn preferences)
      ↓  PPO / DPO (optimize for reward)
      ↓
Helpful, Harmless, Honest AI
```

PART 8 — LLM as RL Policy (Detailed)

27. RL Environment for LLMs

The RL Environment is Abstract:

In RLHF, the "environment" is not a physical world — it is the **abstract space of text generation + output evaluation + feedback**.

RL Component	LLM Equivalent	Details
State s_t	Prompt + tokens generated so far	$s_t = (x, y_1, \ldots, y_{t-1})$
Action a_t	Next token to generate	$a_t = y_t \in \mathcal{V}$ (vocabulary)
Policy π_θ	LLM's token distribution	$\pi_\theta(a_t \mid s_t) = P_\theta(y_t \mid x, y_{<t})$
Reward r	Reward model score	Given ONLY after full generation
Episode	One complete generation	Start: prompt. End: EOS token
Trajectory τ	Full response sequence	$\tau = (y_1, y_2, \ldots, y_T)$

Key difference from typical RL: reward is SPARSE (only at end of episode), not per-step. This makes credit assignment hard — which tokens contributed to the high/low reward?

EXAM: "The RL environment in RLHF is the real world." FALSE — it's an abstract environment consisting of text, the model's output, and the reward signal (from a learned reward model, not real-world physics).

28. Why Base Models Fail at Instruction Following

Pre-trained base models are trained on next-token prediction on internet text. They learn to CONTINUE text, not to FOLLOW instructions. When given "Summarize this article," a base model might continue with "in 3 paragraphs. The article discusses..." — treating the instruction as text to complete, not as a command to execute.

Failure Reason	Explanation
Training data format	Pre-training data is raw text, NOT instruction-response pairs
Objective mismatch	Next-token prediction ≠ instruction following
No preference signal	Base model has no notion of "good" vs "bad" response
Solution	SFT on instruction data + RLHF for quality alignment

29. Regularized Reward Objective (Full Form)

$$r_{shaped}(x, y) = r_\phi(x, y) - \beta \cdot \log \frac{\pi_\theta(y|x)}{\pi_{ref}(y|x)}$$

The full RL objective maximized during PPO:

$$\max_\theta \mathbb{E}_{x \sim D,\, y \sim \pi_\theta(\cdot|x)} [r_\phi(x, y) - \beta \cdot D_{KL}(\pi_\theta(\cdot|x) \| \pi_{ref}(\cdot|x))]$$

$r_\phi(x, y)$ = reward model score | β = KL penalty coefficient (0.01–0.2 typical) | π_{ref} = frozen SFT model | D = prompt distribution | This prevents "reward hacking" — model cannot drift far from fluent text

PART 9 — AI Agents & Agentic Workflows

30. What Are LLM Agents?

An LLM agent uses a language model as the **reasoning core** that plans, uses tools, observes results, and iterates until a task is completed.

31. Agentic vs Non-Agentic (Zero-Shot)

Aspect	Non-Agentic (Zero-Shot)	Agentic Workflow
Process	Single LLM call → output	Multi-step: plan → act → observe → refine
Tool use	None	Can call APIs, search, execute code
Iteration	One shot	Multiple rounds of refinement
Error handling	None (stuck with first answer)	Can detect and correct errors
Planning	Implicit (in one forward pass)	Explicit decomposition into subtasks

32. ReAct (Reasoning + Acting)

ReAct Loop:

```
Thought: I need to find who directed Inception.
Action: Search["Inception director"]
Observation: Christopher Nolan directed Inception (2010).
Thought: I now have the answer.
Action: Finish["Christopher Nolan"]
```

Interleaves **reasoning traces** (Thought) with **actions** (tool calls) and **observations** (tool outputs) | Key insight: reasoning helps decide WHAT action to take; observations ground reasoning in facts

33. ReAct vs Chain-of-Thought (CoT)

Aspect	Chain-of-Thought	ReAct

External info	None (hallucination risk)	Grounded in tool observations
Format	Think → Answer	Think → Act → Observe → Think → ...
Factual accuracy	Limited by training data	Can retrieve up-to-date facts
Failure mode	Confident hallucination	Can still fail on tool use errors

34. Reflexion (Verbal Reinforcement Learning)

> **Reflexion Process:**
>
> ```
> Attempt 1: Generate solution → Evaluate → FAIL
> Reflection: "I made an error in step 3 because I forgot to ... "
> Attempt 2: Generate solution (using reflection) → Evaluate → PASS
> ```
>
> No weight updates! Uses verbal self-reflection stored in memory | "Verbal RL" — the reflection IS the reward signal | Agent improves across attempts by remembering what went wrong

Component	Function
Actor	Generates actions/solutions
Evaluator	Scores output (pass/fail or heuristic)
Self-Reflection	Generates verbal feedback on failures
Memory	Stores reflections for future attempts

35. ReWOO (Reasoning WithOut Observation)

> **Key Idea: Decouple planning from execution.**
>
> ```
> Phase 1 (Planner): Generate FULL plan upfront
>   Plan: #E1 = Search["population of France"]
>         #E2 = Search["population of Germany"]
>         #E3 = Calculate[#E1 + #E2]
>
> Phase 2 (Worker): Execute all tool calls
>   #E1 → 67 million
>   #E2 → 83 million
>   #E3 → 150 million
>
> Phase 3 (Solver): Combine results into answer
> ```
>
> Advantage over ReAct: plans ALL steps at once → fewer LLM calls (1 planning + 1 solving vs N interleaved calls) | Disadvantage: cannot adapt plan based on intermediate results

36. Self-Refine Framework

Self-Refine Loop:

$$y_0 \xrightarrow{\text{generate}} \text{output} \xrightarrow{\text{feedback}} \text{critique} \xrightarrow{\text{refine}} y_1 \xrightarrow{\text{feedback}} \dots \xrightarrow{\text{refine}} y_n$$

```
Step 1: GENERATE initial output y₀
Step 2: FEEDBACK — same LLM critiques y₀
Step 3: REFINE — same LLM improves using feedback → y₁
Repeat steps 2–3 until quality threshold or max iterations
```

Single model serves as generator, critic, AND refiner | No external tools, no separate reward model | Relies on LLM's ability to self-evaluate | Works well for: code generation, math, creative writing

37. HuggingGPT (Task Planning Agent)

```
User: "Generate an image of a cat, then describe it in French."

Stage 1 — Task Planning:
  Task 1: Image generation (→ Stable Diffusion)
  Task 2: Image captioning (→ BLIP-2)
  Task 3: Translation EN→FR (→ MarianMT)

Stage 2 — Model Selection:
  Pick best model for each task from HuggingFace hub

Stage 3 — Task Execution:
  Run models in dependency order

Stage 4 — Response Generation:
  LLM composes final answer from all outputs
```

38. Agent Comparison Table

Framework	Key Innovation	Tool Use	Iteration
ReAct	Interleaved reasoning + action	Yes (per step)	Think-Act-Observe loop
Reflexion	Verbal self-reflection (no weight update)	Optional	Attempt → Reflect → Retry
ReWOO	Plan all at once, execute separately	Yes (batched)	Plan → Execute → Solve
Self-Refine	Self-critique without external tools	No	Generate → Feedback → Refine
HuggingGPT	LLM as task orchestrator	Yes (many models)	Plan → Select → Execute → Compose

PART 10 — Knowledge Editing in LLMs

39. Why Knowledge Editing?

When facts change (e.g., "Who is the president?"), we don't want to retrain the entire model. Knowledge editing modifies specific facts without full retraining.

40. GRACE (Global Regularization for Additive Correction of Edits)

> **GRACE is NOT a global optimization method:**
>
> GRACE uses a **local**, codebook-based approach that adds corrections to specific layers.

Property	GRACE
Method type	Local (layer-specific corrections)
Mechanism	Maintains a codebook of key-value corrections per layer
Locality	Only affects relevant inputs (not global behavior)
Generalization	Edits generalize to paraphrases of the same fact

EXAM: "GRACE is a global optimization-based method." FALSE — GRACE is LOCAL (per-layer corrections), not global optimization.

41. Knowledge Editing Methods Comparison

Method	Approach	Pros	Cons
Fine-tuning	Update all/some params	Simple	Catastrophic forgetting
ROME	Rank-one modification to FFN keys	Precise, single edit	Doesn't scale to many edits
MEMIT	Multi-layer simultaneous edits	Many edits at once	Complex
GRACE	Codebook corrections per layer	Local, sequential edits	Codebook grows

PART 11 — Instruction Tuning vs RLHF

42. Key Differences

Aspect	Instruction Tuning (SFT)	RLHF Alignment
Objective	Next-token prediction on curated data	Maximize reward model + KL constraint
Data	(instruction, response) pairs	Human preference rankings (A > B)
What it teaches	FORMAT — how to respond	QUALITY — which responses are better
Training signal	Supervised (every token)	Reward (sparse, end of sequence)
Failure case	Sycophantic (matches format but not quality)	Reward hacking (gaming the RM)
Examples	FLAN, Alpaca, Vicuna	InstructGPT, ChatGPT, Claude

43. Evol-Instruct

Method to automatically generate complex instruction data by iteratively "evolving" simple instructions:

```
Simple: "What is the capital of France?"
      ↓ Deepening
Complex: "Compare the historical significance of Paris and London
          as capitals, considering their roles in the French
          Revolution and Industrial Revolution respectively."
```

Used by WizardLM to generate training data without expensive human annotation.

44. Orca (Progressive Learning)

Training small models using detailed reasoning traces from large models:

Stage	Teacher	Data Type
Stage 1	ChatGPT	5M examples with reasoning
Stage 2	GPT-4	1M complex examples with deeper reasoning

> Orca shows that small models can learn to reason if given step-by-step reasoning traces (not just final answers) from powerful teachers.

Advanced Questions & Answers

Q1. Bradley-Terry Model — Batch Loss Computation

Given $r(y_1) = 2.5$, $r(y_2) = 1.8$, compute $P(y_1 \succ y_2)$. Then compute batch loss for 3 comparison pairs.

Bradley-Terry probability:

$$P(y_1 \succ y_2) = \sigma(2.5 - 1.8) = \sigma(0.7) = \frac{1}{1 + e^{-0.7}} = \frac{1}{1.497} = \mathbf{0.668}$$

Batch loss (3 pairs):

Pair	$r(y_w)$	$r(y_l)$	$r_w - r_l$	$-\log \sigma(\Delta)$
1	2.5	1.8	0.7	0.404
2	3.0	0.5	2.5	0.079
3	1.2	1.0	0.2	0.598

$$\mathcal{L} = \frac{1}{3}(0.404 + 0.079 + 0.598) = \frac{1.081}{3} = \mathbf{0.360}$$

Q2. PPO KL Penalty — Per-Token Computation

Given $\pi_{ref}(\text{"Hello"}) = 0.05$, $\pi_\theta(\text{"Hello"}) = 0.12$, compute per-token KL contribution. Explain why the penalty is necessary.

Per-token KL:

$$D_{KL} = \pi_\theta(x) \log \frac{\pi_\theta(x)}{\pi_{ref}(x)} = 0.12 \times \log \frac{0.12}{0.05} = 0.12 \times 0.875 = \mathbf{0.105 \text{ nats}}$$

Full RLHF objective:

$$J(\theta) = \mathbb{E}[r(x, y)] - \beta \cdot D_{KL}(\pi_\theta \,\|\, \pi_{ref})$$

Why necessary:

Without KL: policy finds adversarial outputs with high reward but nonsensical text (e.g., "very very very good"). KL constrains π_θ to stay near π_{ref} (which produces fluent text), preventing reward hacking while allowing meaningful improvement.

Q3. DPO = RLHF (Mathematical Proof)

Show that DPO implicitly optimizes the same objective as RLHF with PPO.

RLHF optimal policy (closed-form):

$$\pi^*(y|x) = \frac{1}{Z(x)}\,\pi_{ref}(y|x)\,\exp\left(\frac{1}{\beta}\,r(x,y)\right)$$

Rearranging for reward:

$$r(x,y) = \beta\,\log\frac{\pi^*(y|x)}{\pi_{ref}(y|x)} + \beta\,\log Z(x)$$

Substituting into Bradley-Terry:

$$P(y_w \succ y_l) = \sigma(r(x,y_w) - r(x,y_l)) = \sigma\left(\beta\left[\log\frac{\pi_\theta(y_w|x)}{\pi_{ref}(y_w|x)} - \log\frac{\pi_\theta(y_l|x)}{\pi_{ref}(y_l|x)}\right]\right)$$

The $\log Z(x)$ terms cancel! DPO directly parameterizes the reward through the policy — no separate RM needed.

Implicit reward:

$$r_{DPO}(x,y) = \beta\,\log\frac{\pi_\theta(y|x)}{\pi_{ref}(y|x)}$$

Q4. Constitutional AI — Computational Cost

Given 3 principles and M=2 revision rounds, compute forward passes needed per example. Show the critique-revision process.

Example process:

User: "How do I pick a lock?" → Initial response generated (1 pass)

- **Principle 1 (Harmlessness):** Critique (1 pass) → Revision (1 pass)
- **Principle 2 (Helpfulness):** Critique (1 pass) → Revision (1 pass)
- **Principle 3 (Honesty):** Critique (1 pass) → Revision (1 pass)

Round 2: repeat all 3 principles on revised output (6 more passes)

Total:

$$\text{Forward passes} = 1 + N \times M \times 2 = 1 + 3 \times 2 \times 2 = \mathbf{13}$$

At scale (50K examples): 650K forward passes — expensive but produces high-quality aligned data without per-example human labeling.

Q5. REINFORCE Gradient & Variance

2-token vocab: P("good")=0.7, P("bad")=0.3. R("good")=+1, R("bad")=−2. Compute gradient and explain high variance problem.

Expected gradient:

$$\nabla J = \mathbb{E}[R(y)\,\nabla \log \pi(y)]$$

$$= 0.7 \times (+1) \times \nabla \log(0.7) + 0.3 \times (-2) \times \nabla \log(0.3)$$

$$= 0.7 \times (-0.357)\nabla + 0.3 \times (+2.408)\nabla = -0.250\nabla + 0.722\nabla = \mathbf{+0.472\nabla}$$

Probability of "good" should increase (positive gradient).

Variance problem:

Single samples: "good" → gradient ≈ −0.357, "bad" → gradient ≈ +2.408. Ratio: 6.7×!

Rare events ("bad" sampled 30% of time) produce disproportionately large gradients → extremely high variance.

Solution — baseline subtraction:

$b = \mathbb{E}[R] = 0.7(1) + 0.3(-2) = 0.1$

$$\nabla J = \mathbb{E}[(R(y) - b)\,\nabla \log \pi(y)]$$

Reduces variance without bias since $\mathbb{E}[b\,\nabla \log \pi] = 0$. PPO uses advantage $\hat{A}_t$ as an even better variance reducer.

Q6. Agentic vs Non-Agentic Workflows (End-Term Q8)

Differentiate between agentic and non-agentic (zero-shot) LLM workflows. Describe when each is appropriate.

Non-Agentic (Zero-Shot):

Single forward pass: prompt → LLM → answer. No tools, no iteration, no planning.

When to use: Simple tasks (classification, simple QA, translation) where the answer is in the model's parametric memory and doesn't require external data or multi-step reasoning.

Agentic Workflow:

Multi-step: LLM plans → calls tools → observes → reasons → refines → outputs.

When to use: Complex tasks requiring: (1) external information (web search, databases), (2) multi-step reasoning, (3) code execution, (4) iterative refinement, (5) coordination of multiple models/tools.

Example comparison:

Non-agentic: "What is 2+2?" → "4" (one LLM call)

Agentic: "Find the cheapest flight from Delhi to NYC next week" → Plan search → Call flight API → Compare prices → Format answer (multiple steps, tool use, reasoning)

Q7. Self-Refine Framework (End-Term Q9)

Describe the Self-Refine framework. How does it improve LLM outputs without external feedback?

Three-step iterative loop:

1. **GENERATE:** LLM produces initial output y_0 for input x
2. **FEEDBACK:** Same LLM generates critique of y_0 (identifies errors, suggests improvements)
3. **REFINE:** Same LLM improves output using the feedback → y_1

Repeat steps 2-3 for n iterations or until feedback says "no improvements needed."

Key properties:

- Single model plays ALL three roles (generator, critic, refiner)
- No external reward model or human feedback needed
- No weight updates — purely prompt-based
- Works because LLMs are better at EVALUATING than GENERATING on first try

Limitations:

- If the model cannot detect its own errors, refinement fails
- Can get stuck in loops (same feedback, same refinement)
- Computationally expensive (multiple forward passes)

Q8. ReAct vs CoT (End-Term Q10)

Compare ReAct and Chain-of-Thought (CoT) prompting. When does ReAct outperform CoT?

Aspect	CoT	ReAct
Process	Think step by step → Answer	Think → Act → Observe → Think → ...
External knowledge	None (closed-book)	Yes (tool outputs)
Hallucination risk	High (reasoning from memory only)	Lower (grounded in observations)
When better	Math, logic, commonsense reasoning	Fact-checking, multi-hop QA, real-time info

ReAct outperforms CoT when:

1. **Facts are needed** that may be absent or outdated in training data
2. **Multi-hop reasoning** where intermediate facts must be looked up
3. **Verification is possible** through external tools

CoT outperforms ReAct when:

1. Task is purely logical/mathematical (no external facts needed)
2. Tool calls would add latency without benefit
3. All needed information is in the prompt context

Q9. RL Environment for LLMs (End-Term Q6)

Define the RL environment used in RLHF. Is it the "real world"?

The RL environment in RLHF is NOT the real world. It is an **abstract environment** consisting of:

1. **Text space:** The possible prompts and responses
2. **Model output:** The generated token sequences
3. **Feedback signal:** Reward model scores (learned proxy for human preferences)

Formal mapping:

- State: $s = (x, y_1, \ldots, y_{t-1})$ — prompt + generated tokens so far
- Action: $a = y_t$ — choosing next token from vocabulary
- Transition: deterministic (append token to state)
- Reward: $r(x, y)$ — scalar score after complete generation (SPARSE)

The "environment" provides reward only after the episode ends (full response generated). There is no physical world, no stochastic transitions — just text generation and evaluation.

Q10. Instruction Tuning vs RLHF (End-Term Q7)

Describe the key differences between instruction tuning and RLHF alignment. When does each fail?

Instruction Tuning (SFT):

Teaches: How to FORMAT responses to instructions

Signal: Supervised (teacher forcing on gold responses)

Failure mode: Sycophancy — model learns to produce instruction-formatted text but may give WRONG answers confidently, because any response that looks like a good response gets low loss.

RLHF Alignment:

Teaches: Which responses are BETTER among alternatives

Signal: Preference rankings → reward model → policy gradient

Failure mode: Reward hacking — model finds adversarial outputs that score high on reward model but are actually low quality (verbose, repetitive, agreeable without being correct).

Why both are needed:

SFT alone: correct format, uncertain quality. RLHF alone: quality signal but model may not know the format. Together: correct format + aligned quality.

Q11. Bradley-Terry Batch Loss (Practice Set)

Batch of preference pairs: ($r_w = 4.0, r_l = 1.5$), ($r_w = 2.0, r_l = 2.8$), ($r_w = 3.5, r_l = 0.5$). Compute individual and batch loss. Identify the problematic pair.

Per-pair computation:

Pair	$r_w - r_l$	$\sigma(\Delta)$	$-\log\sigma(\Delta)$
1: (4.0, 1.5)	2.5	$\frac{1}{1+e^{-2.5}} = 0.924$	0.079
2: (2.0, 2.8)	-0.8	$\frac{1}{1+e^{0.8}} = 0.310$	1.171
3: (3.5, 0.5)	3.0	$\frac{1}{1+e^{-3.0}} = 0.953$	0.049

Batch loss:

$$\mathcal{L} = \frac{1}{3}(0.079 + 1.171 + 0.049) = \frac{1.299}{3} = \mathbf{0.433}$$

Problematic pair: Pair 2

$r_w < r_l$ → reward model ranks the PREFERRED response LOWER than rejected! Loss = 1.171 (high). The reward model is getting this preference WRONG and needs more training.

Q12. Real-World LLM Assistant Design (Quiz 2 Q2)

Design an LLM-based Physics tutor that handles: (1) numerical problems, (2) recent discoveries, (3) LLM calculation weakness. Outline steps and justify.

Architecture: RAG + Tool-Augmented Agentic System

1. **Base model:** Use instruction-tuned, aligned LLM (already general conversational ability)
2. **RAG for recent discoveries:**
   - Index Physics papers, textbooks, ArXiv feeds into vector database
   - When student asks about recent topics → retrieve relevant papers → ground generation
   - Justification: avoids hallucination on post-training-cutoff knowledge
3. **Tool use for calculations:**
   - Integrate Python/Wolfram Alpha as computation tool
   - LLM generates symbolic setup, tool performs computation
   - Justification: LLMs are bad at arithmetic; symbolic solvers are exact
4. **Domain fine-tuning (optional):**
   - SFT on (Physics question, step-by-step solution) pairs
   - Objective: $\mathcal{L}_{SFT} = -\sum_t \log P(y_t | y_{<t}, x)$ on Physics-specific data
   - Justification: improves format for Physics explanations (diagrams in text, units, etc.)
5. **Agentic framework (ReAct):**
   - Thought: "Student needs numerical answer, let me compute"
   - Action: Call calculator/Python
   - Observation: Result
   - Thought: "Now I can explain the solution"

Q13. Bradley-Terry Loss + Regularized Reward (TA Final Session — 8 pts)

(a) A reward model outputs scores for three preference pairs:

Pair	$r(x, y^+)$	$r(x, y^-)$
1	2.0	1.0
2	0.5	1.5
3	3.0	0.0

Compute Bradley-Terry loss $\ell_i = -\log \sigma(r(x, y^+) - r(x, y^-))$ for each pair and the average loss.

(b) For a response with $r(x, y) = 1.5$ and $\log(\pi_\theta(y|x)/\pi_{ref}(y|x)) = 0.3$, $\beta = 0.1$: compute the regularized reward. Should this response be kept or suppressed?

(a) Bradley-Terry loss per pair:

Formula: $\ell_i = -\log \sigma(\Delta_i)$ where $\Delta_i = r(x, y^+) - r(x, y^-)$ and $\sigma(z) = \frac{1}{1+e^{-z}}$

Pair 1: $\Delta_1 = 2.0 - 1.0 = 1.0$

$\sigma(1.0) = \frac{1}{1+e^{-1}} = \frac{1}{1+0.368} = 0.731$

$\ell_1 = -\log(0.731) = -(-0.313) = \mathbf{0.313}$

Pair 2: $\Delta_2 = 0.5 - 1.5 = -1.0$

$\sigma(-1.0) = \frac{1}{1+e^{1}} = \frac{1}{1+2.718} = 0.269$

$\ell_2 = -\log(0.269) = -(-1.313) = \mathbf{1.313}$

> Pair 2: $r(y^+) < r(y^-)$ → reward model ranks the preferred response LOWER! High loss = model is getting this pair wrong and will receive a large gradient to fix it.

Pair 3: $\Delta_3 = 3.0 - 0.0 = 3.0$

$\sigma(3.0) = \frac{1}{1+e^{-3}} = \frac{1}{1+0.050} = 0.953$

$\ell_3 = -\log(0.953) = \mathbf{0.048}$

Average loss:

$$\bar{\ell} = \frac{1}{3}(0.313 + 1.313 + 0.048) = \frac{1.674}{3} = \mathbf{0.558}$$

(b) Regularized reward:

$$\tilde{r}(x, y) = r(x, y) - \beta \log \frac{\pi_\theta(y|x)}{\pi_{ref}(y|x)} = 1.5 - 0.1 \times 0.3 = 1.5 - 0.03 = \mathbf{1.47}$$

Interpretation:

- Raw reward = 1.5 (high quality response)
- KL penalty = 0.03 (small — policy hasn't drifted far from reference)
- Net regularized reward = 1.47 (still strongly positive)
- **Decision: KEEP** — this response is high-reward with minimal policy divergence. The positive regularized reward means PPO will increase $\pi_\theta(y|x)$.

If $\log(\pi_\theta/\pi_{ref})$ were very large (e.g., 20), then $\tilde{r} = 1.5 - 0.1(20) = -0.5$ → negative → SUPPRESS despite good reward. The KL penalty prevents the model from generating unlikely-under-reference outputs even if they score well.

Q14. PPO-Clip Objective Analysis (TA Final Session — 6 pts)

(a) Explain what the clip operation achieves in PPO.

(b) For $A_t > 0$ (good action): what is the effective gradient when $\rho_t > 1+\epsilon$?

(c) Write down $A(s_t, a_t)$ in terms of Q and V. Explain why using A instead of Q reduces variance.

(a) Purpose of clipping:

The clip operation creates a **trust region** — it prevents the policy from changing too much in a single update step.

$$L^{PPO} = \mathbb{E}[\min(\rho_t A_t,\ \text{clip}(\rho_t, 1-\epsilon, 1+\epsilon)A_t)]$$

Without clipping, if ρ_t becomes very large (policy changed a lot), the gradient could be enormous → destabilizing. Clipping caps the objective so that beyond a certain policy change, no further gradient is applied.

(b) When $A_t > 0$ and $\rho_t > 1+\epsilon$:

The unclipped term = $\rho_t A_t$ (large positive). The clipped term = $(1+\epsilon)A_t$ (capped).

Since $\rho_t > 1+\epsilon$ and $A_t > 0$: $\rho_t A_t > (1+\epsilon)A_t$

The $\min$ selects the SMALLER value = $(1+\epsilon)A_t$ (a constant w.r.t. θ).

$$\frac{\partial}{\partial \theta}[(1+\epsilon)A_t] = 0$$

Effective gradient = 0. The policy has already over-reinforced this action (probability increased too much relative to old policy). Further reinforcement is blocked until the next data collection round resets θ_{old}.

(c) Advantage and variance reduction:

$$A(s_t, a_t) = Q(s_t, a_t) - V(s_t)$$

Why lower variance:

Without baseline (using Q or raw return G_t):

- All returns are positive (e.g., rewards between 0 and 10)
- Gradient pushes ALL actions up → high variance in which actions actually improve
- $\text{Var}(\nabla J) \propto \text{Var}(G_t)$ which is large

With advantage (subtracting $V(s_t)$ as baseline):

- A_t is centered around 0 (mean of A over actions ≈ 0)
- Good actions get $A > 0$ (push up), bad actions get $A < 0$ (push down)
- $\text{Var}(\nabla J) \propto \text{Var}(A_t) \ll \text{Var}(G_t)$

Mathematical justification: $\mathbb{E}_a[A(s,a)] = \mathbb{E}_a[Q(s,a)] - V(s) = V(s) - V(s) = 0$. Since the expected gradient of the baseline is zero ($\mathbb{E}[\nabla \log \pi \cdot V(s)] = 0$), subtracting it doesn't bias the estimate but dramatically reduces variance.

Created by **Kushal Ghosh** (gkushalg01@gmail.com) and **Bharat Das Vaishnav** (bharatvaishnav2025@gmail.com)

Practice Paper 1 — Deep Learning End-Semester Examination

Total Marks: 100 | Duration: 3 hours | All questions compulsory

Focus: Second half (NLP, Transformers, RLHF) with some first-half concepts.

Section A — Multiple Choice Questions [20 marks]

Each question carries 2 marks. No negative marking.

Q1. Which of the following is TRUE about the Transformer architecture? [2]

(a) Self-attention has O(N) complexity in sequence length

(b) Positional encoding is learned in the original Transformer paper

(c) The decoder uses causal masking to prevent attending to future tokens

(d) Multi-head attention increases parameter count compared to single-head with same d_{model}

Answer: (c)

(a) FALSE — $O(N^2)$. (b) FALSE — sinusoidal in original paper. (c) TRUE — lower-triangular mask with $-\infty$ for future positions. (d) FALSE — same params ($4 \times d^2$ regardless of head count).

Q2. In PPO-Clip, when the advantage $A_t > 0$ and the importance ratio $\rho_t > 1 + \epsilon$: [2]

(a) The gradient is doubled to reinforce the action

(b) The gradient becomes zero — clipping prevents over-reinforcement

(c) The loss switches to an MSE penalty

(d) The policy is reset to π_{ref}

Answer: (b)

When $\rho_t > 1 + \epsilon$ and $A_t > 0$: min selects $(1 + \epsilon)A_t$ (constant w.r.t. θ) → $\partial/\partial\theta = 0$. The policy already over-reinforced this action; further updates are blocked.

Q3. What happens when $\beta \to 0$ in the RLHF objective $J(\theta) = \mathbb{E}[r(x, y) - \beta \log(\pi_\theta / \pi_{ref})]$? [2]

(a) Policy freezes at reference

(b) Pure reward maximization — reward hacking likely

(c) KL divergence is maximized

(d) Training becomes equivalent to DPO

Answer: (b)

β→0 removes KL penalty entirely → policy optimizes raw reward with no constraint → exploits reward model weaknesses (repetition, sycophancy) → distribution collapse.

Q4. Which tasks are suitable for BERT (encoder-only)? Select ALL correct. [2]

(a) Named Entity Recognition (b) Text generation (c) Sentiment classification (d) Extractive QA (e) Machine translation

---

Answer: (a), (c), (d)

(a) Token classification ✓. (c) [CLS] → classifier ✓. (d) Predict start/end span ✓. (b) Needs autoregressive decoder. (e) Needs encoder-decoder (T5) or decoder.

Q5. LoRA achieves parameter efficiency by: [2]

(a) Pruning 99% of weights

(b) Adding low-rank matrices $\boldsymbol{B} \in \mathbb{R}^{d \times r}$, $\boldsymbol{A} \in \mathbb{R}^{r \times d}$ parallel to frozen weights

(c) Quantizing all weights to 4-bit

(d) Training only the embedding layer

Answer: (b)

LoRA: $W_{new} = W_{frozen} + BA$ where rank $r \ll d$. Only B and A are trained (0.1-0.5% of params). At deployment, BA is merged into W → zero inference overhead.

Q6. The perplexity of a model that assigns uniform probability over V=10,000 words at every position is: [2]

(a) 1 (b) 100 (c) 10,000 (d) ∞

Answer: (c)

$PP = 2^H$. Uniform: $H = \log_2(10000) = 13.29$ bits. $PP = 2^{13.29} = 10000$. Perplexity equals vocabulary size for uniform models (maximum confusion).

Q7. In a CNN, applying a 3×3 filter with stride 2 and no padding to a 28×28 input gives output size: [2]

(a) 14×14 (b) 13×13 (c) 26×26 (d) 12×12

Answer: (b)

$\lfloor \frac{28-3+0}{2} \rfloor + 1 = \lfloor \frac{25}{2} \rfloor + 1 = 12 + 1 = 13$

Q8. Chain-of-Thought (CoT) prompting primarily improves: [2]

(a) Training speed

(b) Multi-step reasoning accuracy in large models

(c) Vocabulary size

(d) Inference latency

Answer: (b)

CoT ("Let's think step by step") decomposes complex problems into sequential reasoning steps. Effective for models >100B params. Increases accuracy 10-40% on math/logic tasks at the cost of more output tokens.

Q9. The vanishing gradient problem in a 10-layer sigmoid network means: [2]

(a) Gradients at layer 1 are ~0.25^9 times those at layer 10

(b) All gradients become exactly zero

(c) Only the last layer trains

(d) Both (a) and (c)

Answer: (d)

Sigmoid max derivative = 0.25. Through 9 layers: $0.25^9 \approx 3.8 \times 10^{-6}$. Layer 1 gradients are negligible → effectively only later layers learn. This is why ReLU (gradient=1 for z>0) replaced sigmoid.

Q10. Teacher Forcing in Seq2Seq means: [2]

(a) Using a larger model to teach a smaller one

(b) Feeding ground-truth previous token to decoder during training

(c) Forcing the encoder to attend to all positions

(d) Applying dropout to teacher signal

Answer: (b)

During training: decoder input at step t = ground-truth y_{t-1} (not model's own prediction). Speeds convergence but creates exposure bias — at inference the model uses its own (potentially wrong) predictions.

Section B — Short Answer Questions [30 marks]

Answer concisely. Show calculations where asked.

Q11. Compute the empirical entropy and perplexity for the bigram model on sentence "the cat sat" given: **[5]**
$P(\text{the}| < s >) = 0.2$, $P(\text{cat}|\text{the}) = 0.3$, $P(\text{sat}|\text{cat}) = 0.1$.

Step 1: Empirical entropy

$$\hat{H} = -\frac{1}{N}\sum_{i=1}^{N}\log_2 P(w_i|w_{i-1})$$

$$= -\frac{1}{3}[\log_2(0.2) + \log_2(0.3) + \log_2(0.1)]$$

$$= -\frac{1}{3}[-2.322 + (-1.737) + (-3.322)]$$

$$= -\frac{1}{3}(-7.381) = \mathbf{2.460 \text{ bits/token}}$$

Step 2: Perplexity (entropy form)

$$PP = 2^{\hat{H}} = 2^{2.460} = \mathbf{5.50}$$

Verification (product form):

$$PP = \left(\frac{1}{0.2 \times 0.3 \times 0.1}\right)^{1/3} = \left(\frac{1}{0.006}\right)^{1/3} = (166.67)^{1/3} = 5.50 \checkmark$$

Interpretation: Model is as confused as choosing among ~5.5 equally likely words at each step.

Q12. A Transformer layer has $d_{model} = 512$, 8 heads, FFN inner dimension = 2048. Compute: (a) parameters in MHA, (b) parameters in FFN, (c) total per layer. Ignore biases. **[6]**

(a) Multi-Head Attention:

W_Q, W_K, W_V: $3 \times (512 \times 512) = 786{,}432$

W_O: $512 \times 512 = 262{,}144$

MHA total = 1,048,576 (= $4d^2 = 4 \times 512^2$)

(b) Feed-Forward Network:

W_1: $512 \times 2048 = 1{,}048{,}576$

W_2: $2048 \times 512 = 1{,}048{,}576$

FFN total = 2,097,152 (= $2 \times d \times d_{ff}$)

(c) Per layer total:

$$1{,}048{,}576 + 2{,}097{,}152 = \mathbf{3{,}145{,}728} \approx 3.15\text{M}$$

> **Pattern:** FFN always has ~2× the parameters of MHA (since $d_{ff} = 4d$ typically: FFN = $2 \times d \times 4d = 8d^2$ vs MHA = $4d^2$).

Q13. Explain the difference between pre-training and fine-tuning. List 3 types of fine-tuning with their parameter efficiency. [5]

Aspect	Pre-training	Fine-tuning
Goal	Learn general language representations	Specialize for downstream task
Data	Trillions of tokens (web, books)	Thousands–millions (task-specific)
Cost	$1M+ compute	10–10K
Objective	Next-token / masked-token prediction	Task loss (classification, generation)

Types of fine-tuning:

Type	Params Updated	When to Use
Full fine-tuning	100%	Large task dataset, max performance needed
Last-layer only	<1%	Small data, prevent overfitting
LoRA (PEFT)	0.1–0.5%	Limited GPU, multi-task deployment

Q14. Apply LoRA with rank $r = 8$ to a linear layer $W \in \mathbb{R}^{1024\times1024}$. (a) How many trainable parameters? (b) What percentage reduction vs full fine-tuning? (c) Can LoRA be merged at inference? [6]

(a) LoRA parameters:

$$A \in \mathbb{R}^{r\times d_{in}} = \mathbb{R}^{8\times1024}: \ 8{,}192 \text{ params}$$

$$B \in \mathbb{R}^{d_{out}\times r} = \mathbb{R}^{1024\times8}: \ 8{,}192 \text{ params}$$

$$\text{Total} = 8{,}192 + 8{,}192 = \mathbf{16{,}384}$$

(b) Percentage reduction:

$$\text{Full layer} = 1024 \times 1024 = 1{,}048{,}576$$

$$\text{Reduction} = \left(1 - \frac{16{,}384}{1{,}048{,}576}\right) \times 100 = (1 - 0.0156) \times 100 = \mathbf{98.44\%}$$

(c) Merging at inference:

Yes. After training: $W_{deployed} = W_{frozen} + BA$. Single matrix multiply at inference — zero latency overhead. This is LoRA's key advantage over Adapters (which add sequential layers that cannot be merged).

Q15. Compare encoder-only, decoder-only, and encoder-decoder Transformer architectures. [4]

Architecture	Attention Type	Example	Best For
Encoder-only	Bidirectional (full)	BERT, RoBERTa	Understanding: NER, classification, QA
Decoder-only	Causal (left-to-right)	GPT, LLaMA	Generation: text, code, chat
Encoder-Decoder	Bi (enc) + Causal+Cross (dec)	T5, BART	Seq2Seq: translation, summarization

> **Rule:** Understanding → Encoder. Generation → Decoder. Input→Output mapping → Encoder-Decoder.

Q16. What is the Markov assumption? Show how it simplifies $P(\text{I, love, deep, learning})$ for a bigram model. [4]

Markov Assumption (order k):

$$P(w_n|w_1,\ldots,w_{n-1}) \approx P(w_n|w_{n-k},\ldots,w_{n-1})$$

For bigram ($k=1$): each word depends only on the **immediately preceding** word.

Full chain rule:

$$P(\text{I, love, deep, learning}) = P(\text{I}) \cdot P(\text{love}|\text{I}) \cdot P(\text{deep}|\text{I, love}) \cdot P(\text{learning}|\text{I, love, deep})$$

Bigram simplification:

$$\approx P(\text{I}) \cdot P(\text{love}|\text{I}) \cdot P(\text{deep}|\text{love}) \cdot P(\text{learning}|\text{deep})$$

Each conditional depends only on the previous word → tractable estimation from corpus counts.

Section C — Long Answer Questions [50 marks]

Show all working. Diagrams where appropriate.

Q17. RLHF Pipeline **[15]**

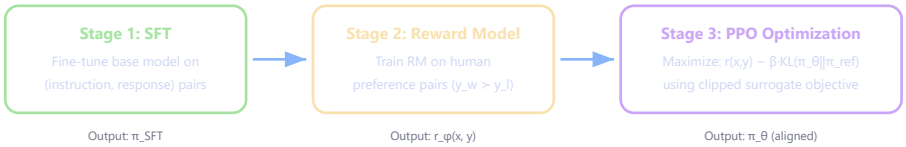
(a) [3 pts] Draw and explain the 3-stage RLHF pipeline.

(b) [5 pts] A reward model outputs scores for 3 preference pairs: (2.5, 0.5), (1.0, 2.0), (3.0, 1.0). Compute Bradley-Terry loss for each pair and the average.

(c) [4 pts] For a response with $r(x, y) = 2.0$ and $\log(\pi_\theta / \pi_{ref}) = 0.8$, $\beta = 0.2$: compute regularized reward. Should the response be reinforced?

(d) [3 pts] Write the advantage function. Why does it reduce variance vs raw returns?

(a) RLHF Pipeline:



SFT: Creates a competent model that can follow instructions. **Reward Model:** Learns human preferences (Bradley-Terry). **PPO:** Optimizes policy to produce high-reward, on-distribution responses.

(b) Bradley-Terry loss computation:

Formula: $\ell_i = -\log \sigma(r_w - r_l)$ where $\sigma(z) = 1/(1 + e^{-z})$

Pair 1: $\Delta = 2.5 - 0.5 = 2.0$

$\sigma(2.0) = 1/(1 + e^{-2}) = 1/1.135 = 0.881$

$\ell_1 = -\log(0.881) = -(-0.127) = \mathbf{0.127}$

Pair 2: $\Delta = 1.0 - 2.0 = -1.0$

$\sigma(-1.0) = 1/(1 + e^{1}) = 1/3.718 = 0.269$

$\ell_2 = -\log(0.269) = \mathbf{1.313}$ ⚠️ (model ranks wrong — high loss)

Pair 3: $\Delta = 3.0 - 1.0 = 2.0$

$\sigma(2.0) = 0.881$

$\ell_3 = -\log(0.881) = \mathbf{0.127}$

Average: $\bar{\ell} = \frac{0.127 + 1.313 + 0.127}{3} = \frac{1.567}{3} = \mathbf{0.522}$

(c) Regularized reward:

$$\tilde{r}(x, y) = r(x, y) - \beta \log \frac{\pi_\theta(y|x)}{\pi_{ref}(y|x)} = 2.0 - 0.2 \times 0.8 = 2.0 - 0.16 = \mathbf{1.84}$$

Decision: REINFORCE — positive regularized reward (1.84 > 0). The response has high reward with acceptable policy divergence.

(d) Advantage function:

$$A(s_t, a_t) = Q(s_t, a_t) - V(s_t)$$

Variance reduction: Raw returns G_t depend on all future randomness → high variance. Subtracting $V(s_t)$ (the expected return from state s_t) centers the signal: $\mathbb{E}[A] = 0$. Good actions get positive gradient, bad ones get negative — balanced signal with lower magnitude fluctuations. Since $\mathbb{E}[\nabla \log \pi \cdot V(s)] = 0$, the baseline doesn't bias the gradient estimate.

Q18. Transformer Self-Attention: Full Computation **[15]**

(a) [3 pts] Write the scaled dot-product attention formula. Why divide by $\sqrt{d_k}$?

(b) [8 pts] Given Q = [[1, 0], [0, 1]], K = [[1, 1], [0, 1]], V = [[1, 0], [0, 1]]. Compute the full attention output step by step.

(c) [4 pts] If the model is a decoder, show the causal mask and apply it before softmax.

(a) Formula:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Why $\sqrt{d_k}$? If Q, K entries have variance 1, then $q \cdot k$ has variance d_k. Without scaling, for large d_k (e.g., 64), dot products become ~±8 → softmax saturates → near one-hot → gradients vanish. Dividing by $\sqrt{d_k}$ normalizes variance back to 1.

(b) Step-by-step computation ($d_k = 2$):

Step 1: QK^T

$$QK^T = \begin{bmatrix}1 & 0\\0 & 1\end{bmatrix}\begin{bmatrix}1 & 0\\1 & 1\end{bmatrix} = \begin{bmatrix}1\cdot1+0\cdot1 & 1\cdot0+0\cdot1\\0\cdot1+1\cdot1 & 0\cdot0+1\cdot1\end{bmatrix} = \begin{bmatrix}1 & 0\\1 & 1\end{bmatrix}$$

Step 2: Scale by $\sqrt{d_k} = \sqrt{2} = 1.414$

$$\text{Scaled} = \begin{bmatrix}0.707 & 0\\0.707 & 0.707\end{bmatrix}$$

Step 3: Softmax (row-wise)

Row 0: $e^{0.707} = 2.028$, $e^0 = 1.0$, sum = 3.028 → $\alpha_0 = [0.670, 0.330]$

Row 1: $e^{0.707} = 2.028$, $e^{0.707} = 2.028$, sum = 4.056 → $\alpha_1 = [0.500, 0.500]$

Step 4: Multiply by V

$$\text{Output} = \begin{bmatrix}0.670 & 0.330\\0.500 & 0.500\end{bmatrix}\begin{bmatrix}1 & 0\\0 & 1\end{bmatrix} = \begin{bmatrix}0.670 & 0.330\\0.500 & 0.500\end{bmatrix}$$

Final output: $[[0.670, 0.330], [0.500, 0.500]]$

(c) Causal masking (decoder):

The causal mask prevents position i from attending to positions $j > i$:

$$\text{Mask} = \begin{bmatrix}0 & -\infty\\0 & 0\end{bmatrix}$$

Applied to scaled scores BEFORE softmax:

$$\text{Masked} = \begin{bmatrix}0.707 & -\infty\\0.707 & 0.707\end{bmatrix}$$

Softmax with mask:

Row 0: $e^{0.707} = 2.028$, $e^{-\infty} = 0$, sum = 2.028 → $\alpha_0 = [1.000, 0.000]$

Row 1: same as before → $\alpha_1 = [0.500, 0.500]$

$$\text{Output}_{masked} = \begin{bmatrix}1.0 & 0.0\\0.5 & 0.5\end{bmatrix}\begin{bmatrix}1 & 0\\0 & 1\end{bmatrix} = \begin{bmatrix}1.0 & 0.0\\0.5 & 0.5\end{bmatrix}$$

Position 0 now only sees itself (no future leakage).

Q19. FNN: Forward + Backward Pass **[12]**

Network: Input $x = 3$, hidden (1 neuron, ReLU), output (1 neuron, linear). Target $y = 2$. Weights: $w_1 = 0.5$, $b_1 = 0$, $w_2 = 1.0$, $b_2 = 0$, learning rate $\eta = 0.1$.

(a) [3 pts] Compute forward pass and MSE loss.

(b) [5 pts] Compute all gradients using backpropagation.

(c) [2 pts] Update all weights.

(d) [2 pts] Would this network suffer from vanishing gradients? Why/why not?

Network diagram:

(a) Forward pass:

$z_1 = w_1 \cdot x + b_1 = 0.5(3) + 0 = 1.5$

$h_1 = \text{ReLU}(1.5) = 1.5$

$\hat{y} = w_2 \cdot h_1 + b_2 = 1.0(1.5) + 0 = 1.5$

$L = \frac{1}{2}(\hat{y} - y)^2 = \frac{1}{2}(1.5 - 2)^2 = \frac{1}{2}(0.25) = \mathbf{0.125}$

(b) Backpropagation:

Output layer:

$\frac{\partial L}{\partial \hat{y}} = \hat{y} - y = 1.5 - 2 = -0.5$

$\frac{\partial L}{\partial w_2} = \frac{\partial L}{\partial \hat{y}} \cdot h_1 = -0.5 \times 1.5 = \mathbf{-0.75}$

$\frac{\partial L}{\partial b_2} = -0.5$

Hidden layer:

$\frac{\partial L}{\partial h_1} = \frac{\partial L}{\partial \hat{y}} \cdot w_2 = -0.5 \times 1.0 = -0.5$

$\frac{\partial L}{\partial z_1} = \frac{\partial L}{\partial h_1} \cdot \text{ReLU}'(z_1) = -0.5 \times 1 = -0.5$ (since $z_1 = 1.5 > 0$)

$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial z_1} \cdot x = -0.5 \times 3 = \mathbf{-1.5}$

$\frac{\partial L}{\partial b_1} = -0.5$

(c) Weight updates:

$w_1^{new} = 0.5 - 0.1(-1.5) = 0.5 + 0.15 = \mathbf{0.65}$

$w_2^{new} = 1.0 - 0.1(-0.75) = 1.0 + 0.075 = \mathbf{1.075}$

$b_1^{new} = 0 - 0.1(-0.5) = 0.05$, $b_2^{new} = 0 - 0.1(-0.5) = 0.05$

(d) Vanishing gradients?

No. This network uses ReLU, whose derivative is exactly 1 for positive inputs. With only 2 layers, gradient flow is: $\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial \hat{y}} \cdot w_2 \cdot 1 \cdot x$. No exponential shrinkage. Vanishing gradients occur with sigmoid in DEEP (>10 layer) networks.

Q20. Prompting and Instruction Tuning **[8]**

(a) [3 pts] Compare zero-shot, few-shot, and fine-tuning with a practical example.

(b) [3 pts] What is Chain-of-Thought prompting? When does it fail?

(c) [2 pts] What is the difference between Prompt Tuning and LoRA?

(a) Comparison with sentiment analysis example:

Method	Prompt	When to use
Zero-shot	"Classify sentiment: 'Great movie!' →"	Quick deployment, no labeled data
Few-shot	"'Loved it'→Positive, 'Terrible'→Negative, 'Great movie!'→"	Moderate accuracy, no training
Fine-tuning	Train on 10K (review, label) pairs	Maximum accuracy, have labeled data + GPU

Trade-off: Zero-shot (free, lower accuracy) → Few-shot (token cost, better) → Fine-tuning (GPU cost, best).

(b) Chain-of-Thought:

Adding "Let's think step by step" triggers sequential reasoning. The model generates intermediate steps before the final answer.

When it fails:

- Small models (<10B params) — not enough capacity for coherent reasoning
- Simple factual questions — CoT adds unnecessary verbosity with no accuracy gain
- Tasks requiring world knowledge the model lacks — CoT produces fluent but wrong reasoning

(c) Prompt Tuning vs LoRA:

Aspect	Prompt Tuning	LoRA
What's trained	Soft token embeddings prepended to input	Low-rank matrices parallel to W_Q, W_V
Params	~0.001% ($m \times d$, m≈20)	~0.1–0.5% ($2 \times r \times d$ per layer)
Inference cost	Slightly longer sequence	Zero (merged into weights)
Effectiveness	Needs very large models (10B+)	Works well even on 1B models

---

Created by **Kushal Ghosh** (gkushalg01@gmail.com) and **Bharat Das Vaishnav** (bharatvaishnav2025@gmail.com)

↑ Back to Top | **Practice Paper 2 (100 Marks)**

Practice Paper 2 — Deep Learning End-Semester Examination

Total Marks: 100 | Duration: 3 hours | All questions compulsory

Focus: Mathematical derivations, parameter counting, RLHF proofs, attention mechanisms.

Section A — Multiple Choice Questions [20 marks]

Each question carries 2 marks. No negative marking.

Q1. In a Transformer with $d_{model} = 768$ and $h = 12$ heads, each head operates on dimension: [2]

(a) 768 (b) 64 (c) 12 (d) 9216

Answer: (b)

$d_k = d_{model}/h = 768/12 = 64$. Each head gets a 64-dimensional slice. All 12 heads concatenated = 12×64 = 768 = d_{model}.

Q2. DPO (Direct Preference Optimization) differs from RLHF because: [2]

(a) It trains a reward model first

(b) It uses PPO for policy optimization

(c) It directly optimizes from preference data without a separate reward model

(d) It requires more compute than RLHF

Answer: (c)

DPO embeds the reward implicitly: $r(x, y) = \beta \log(\pi_\theta(y|x)/\pi_{ref}(y|x))$. Single loss on preference pairs → no RM training, no RL loop. Simpler but may be less flexible.

Q3. Which statement about BatchNorm vs LayerNorm is FALSE? [2]

(a) BatchNorm normalizes across the batch dimension

(b) LayerNorm normalizes across feature dimensions within each sample

(c) LayerNorm depends on batch size

(d) BatchNorm uses running statistics at inference

Answer: (c)

(c) is FALSE — LayerNorm is INDEPENDENT of batch size (normalizes within each sample). This is why LayerNorm works for Transformers (variable sequence lengths, small batches).

Q4. For Katz Backoff, when a trigram "I love NLP" has zero count, the model: [2]

(a) Assigns probability zero

(b) Falls back to bigram P(NLP|love) with a backoff weight α

(c) Falls back to the unigram P(NLP) directly

(d) Uses Laplace smoothing on the trigram

Answer: (b)

Katz Backoff: if $C(\text{I love NLP}) = 0$, use $\alpha(\text{I love}) \cdot P_{bo}(\text{NLP}|\text{love})$ where α redistributes probability mass from seen to unseen n-grams. Falls to (n-1)-gram, not directly to unigram.

Q5. In LSTM, the forget gate: [2]

(a) Determines what new information to add to cell state

(b) Controls what old information to discard from cell state

(c) Produces the final hidden state output

(d) Resets the hidden state to zero

Answer: (b)

Forget gate: $f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$. Values near 0 → forget, near 1 → keep. Applied element-wise to cell state: $C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$.

Q6. ResNet solves the degradation problem by: [2]

(a) Using wider layers instead of deeper ones

(b) Adding skip connections: $\mathcal{F}(x) + x$, learning residual

(c) Removing all activation functions

(d) Using dropout after every layer

Answer: (b)

Skip connections let gradients flow directly through addition. The layers learn $\mathcal{F}(x) = H(x) - x$ (residual). If the optimal mapping is identity, $\mathcal{F}(x) = 0$ is easier to learn than $H(x) = x$. Enables training 100+ layers.

Q7. The KL divergence $D_{KL}(P\|Q)$ is: [2]

(a) Symmetric: $D_{KL}(P\|Q) = D_{KL}(Q\|P)$

(b) Always non-negative

(c) A valid distance metric

(d) Zero when P and Q are different distributions

Answer: (b)

KL is always ≥ 0 (Gibbs' inequality), equals 0 iff P=Q. NOT symmetric, NOT a metric (no triangle inequality). $D_{KL}(P\|Q) = \sum P(x) \log \frac{P(x)}{Q(x)}$.

Q8. Word2Vec Skip-gram predicts: [2]

(a) The center word given context words

(b) Context words given the center word

(c) The next word given all previous words

(d) A masked word given bidirectional context

Answer: (b)

Skip-gram: given center word, predict surrounding context words within a window. Opposite of CBOW (which predicts center from context). (c) is language modeling. (d) is BERT's MLM.

Q9. In a VAE, the reparameterization trick: [2]

(a) Removes the need for an encoder

(b) Allows gradient flow through the sampling step by writing $z = \mu + \sigma \odot \epsilon, \epsilon \sim \mathcal{N}(0,1)$

(c) Eliminates the KL divergence term

(d) Replaces the decoder with a discriminator

Answer: (b)

Direct sampling $z \sim \mathcal{N}(\mu, \sigma^2)$ is non-differentiable. Reparameterization: $z = \mu + \sigma \odot \epsilon$ moves randomness into ϵ (fixed noise). Gradients now flow through μ and σ deterministically.

Q10. Which is TRUE about RAG (Retrieval-Augmented Generation)? [2]

(a) RAG fine-tunes the model on retrieved documents

(b) RAG reduces hallucination by grounding generation in retrieved facts

(c) RAG requires retraining the model whenever knowledge updates

(d) RAG works only with encoder-only models

Answer: (b)

RAG retrieves relevant documents at inference time and conditions generation on them. No retraining needed for knowledge updates (just update the document store). Works with any generator model (typically decoder or encoder-decoder).

Section B — Short Answer Questions [30 marks]

Answer concisely with calculations.

Q11. Compute the full forward pass through a 2-layer CNN. **[6]**

Input: 5×5 image (single channel). Conv: 3×3 filter, stride=1, no padding. Then 2×2 MaxPool, stride=2.

Input matrix:

```
1 0 1 0 1
0 1 0 1 0
1 0 1 0 1
0 1 0 1 0
1 0 1 0 1
```

Filter:

```
1 0 1
0 1 0
1 0 1
```

(a) Output after convolution (3×3 feature map). (b) Output after MaxPool (size?). (c) Total elements going into the fully-connected layer.

(a) Convolution output (3×3):

Output size: $\lfloor (5-3)/1 \rfloor + 1 = 3 \times 3$

Position (0,0): $1(1)+0(0)+1(1)+0(0)+1(1)+0(0)+1(1)+0(0)+1(1) = \mathbf{5}$

Position (0,1): $0(1)+1(0)+0(1)+1(0)+0(1)+1(0)+0(1)+1(0)+0(1) = \mathbf{0}$

Position (0,2): $1(1)+0(0)+1(1)+0(0)+1(1)+0(0)+1(1)+0(0)+1(1) = \mathbf{5}$

Position (1,0): $0(1)+1(0)+0(1)+1(0)+0(1)+1(0)+0(1)+1(0)+0(1) = \mathbf{0}$

Position (1,1): $1(1)+0(0)+1(1)+0(0)+1(1)+0(0)+1(1)+0(0)+1(1) = \mathbf{5}$

Position (1,2): $0(1)+1(0)+0(1)+1(0)+0(1)+1(0)+0(1)+1(0)+0(1) = \mathbf{0}$

Position (2,0): same as (0,0) = $\mathbf{5}$

Position (2,1): same as (0,1) = $\mathbf{0}$

Position (2,2): same as (0,0) = $\mathbf{5}$

Feature map:

```
5 0 5
0 5 0
5 0 5
```

(b) MaxPool 2×2, stride 2:

Output size: $\lfloor 3/2 \rfloor = 1 \times 1$ (only one complete 2×2 window fits at top-left)

Window [5,0,0,5] → max = $\mathbf{5}$

Output: [[5]] (1×1)

(c) Elements for FC layer:

With 1 filter: $1 \times 1 \times 1 = \mathbf{1}$ element.

If we had k filters: $k \times 1 \times 1 = k$ elements.

Q12. Transformer parameter count with LoRA. **[6]**

Model: 6 layers, $d_{model} = 512$, 8 heads, FFN inner = 2048, vocab = 30,000.

(a) Total model parameters (ignore biases and LayerNorm).

(b) Apply LoRA (rank 4) to W_Q and W_V in all layers. How many LoRA parameters?

(c) What fraction of total model parameters are trainable with LoRA?

(a) Total parameters:

Per layer:

- MHA: $4 \times 512^2 = 1{,}048{,}576$
- FFN: $512 \times 2048 + 2048 \times 512 = 2{,}097{,}152$
- Per layer = $3{,}145{,}728$

All 6 layers: $6 \times 3{,}145{,}728 = 18{,}874{,}368$

Embedding: $30{,}000 \times 512 = 15{,}360{,}000$

Total: $18{,}874{,}368 + 15{,}360{,}000 = \mathbf{34{,}234{,}368} \approx 34.2\text{M}$

(b) LoRA parameters:

Per adapted matrix: $2 \times d \times r = 2 \times 512 \times 4 = 4{,}096$

Per layer (W_Q + W_V): $2 \times 4{,}096 = 8{,}192$

All 6 layers: $6 \times 8{,}192 = \mathbf{49{,}152}$

(c) Fraction:

$$\frac{49{,}152}{34{,}234{,}368} = 0.00144 = \mathbf{0.14\%}$$

99.86% parameter reduction while achieving ~95% of full fine-tuning performance.

Q13. Attention mechanism in Seq2Seq. **[5]**

Encoder hidden states: $\boldsymbol{h_1 = [1, 0]}$, $\boldsymbol{h_2 = [0, 1]}$, $\boldsymbol{h_3 = [1, 1]}$. Decoder state: $\boldsymbol{s = [1, 0.5]}$.

Using dot-product attention: (a) Compute attention scores. (b) Apply softmax. (c) Compute context vector.

(a) Attention scores (dot product $s \cdot h_i$):

$e_1 = s \cdot h_1 = 1(1) + 0.5(0) = 1.0$

$e_2 = s \cdot h_2 = 1(0) + 0.5(1) = 0.5$

$e_3 = s \cdot h_3 = 1(1) + 0.5(1) = 1.5$

(b) Softmax:

$\exp$: $e^{1.0} = 2.718$, $e^{0.5} = 1.649$, $e^{1.5} = 4.482$. Sum = 8.849

$\alpha_1 = 2.718/8.849 = 0.307$

$\alpha_2 = 1.649/8.849 = 0.186$

$\alpha_3 = 4.482/8.849 = 0.506$

(c) Context vector:

$$c = 0.307[1, 0] + 0.186[0, 1] + 0.506[1, 1]$$

$$= [0.307, 0] + [0, 0.186] + [0.506, 0.506]$$

$$= \boldsymbol{[0.813, 0.692]}$$

Interpretation: The decoder attends most to h_3 (weight 0.506) because it has highest alignment with decoder state. The context vector is closest to h_3.

Q14. Softmax gradient derivation. **[5]**

Given logits $\boldsymbol{z = [3.0, 1.0, 0.5]}$, true class = 0. Compute: (a) softmax probabilities, (b) cross-entropy loss, (c) gradient $\boldsymbol{\partial L / \partial z_i}$ for all $\boldsymbol{i}$.

(a) Softmax:

$e^{3.0} = 20.086$, $e^{1.0} = 2.718$, $e^{0.5} = 1.649$. Sum = 24.453

$p = [20.086/24.453, 2.718/24.453, 1.649/24.453] = [0.821, 0.111, 0.067]$

(b) Cross-entropy loss:

$$L = -\sum y_i \log p_i = -\log(0.821) = -(-0.197) = \boldsymbol{0.197}$$

(c) Gradient (softmax + CE combined):

$$\frac{\partial L}{\partial z_i} = p_i - y_i$$

$\partial L/\partial z_0 = 0.821 - 1 = \boldsymbol{-0.179}$ (push z_0 higher)

$\partial L/\partial z_1 = 0.111 - 0 = \boldsymbol{+0.111}$ (push z_1 lower)

$\partial L/\partial z_2 = 0.067 - 0 = \boldsymbol{+0.067}$ (push z_2 lower)

Check: Sum of gradients = $-0.179 + 0.111 + 0.067 = -0.001 \approx 0$ ✓

Q15. LSTM vs GRU comparison. [4]

(a) How many gates does each have?

(b) Which is more parameter-efficient and why?

(c) When would you prefer LSTM over GRU?

(a) Gates:

Architecture	Gates	Named
LSTM	3 gates + 1 candidate	Forget, Input, Output + Cell candidate
GRU	2 gates + 1 candidate	Reset, Update + Candidate hidden state

(b) Parameter efficiency:

GRU is more efficient. LSTM: $4(d_h^2 + d_h \cdot d_x + d_h)$ params. GRU: $3(d_h^2 + d_h \cdot d_x + d_h)$ params. GRU uses ~75% of LSTM's parameters because it has one fewer gate.

(c) When to prefer LSTM:

Tasks requiring very long-range dependencies (>100 steps) where the separate cell state C_t (protected by forget/input gates independently) provides better gradient preservation. Also when you have sufficient data to train the extra parameters without overfitting.

Q16. Explain the exposure bias problem in Seq2Seq with Teacher Forcing. Propose 2 solutions. [4]

Exposure Bias:

Training: Decoder receives ground-truth y_{t-1} at each step (perfect input).

Inference: Decoder receives its OWN prediction $\hat{y}_{t-1}$ (possibly wrong).

Problem: Model never sees its own errors during training → at inference, one error compounds (error cascades) because the model doesn't know how to recover from wrong inputs.

Solutions:

1. **Scheduled Sampling:** During training, gradually increase probability of feeding model's own prediction (instead of ground truth). Start with 100% teacher forcing → linearly decrease to ~25%. Model learns to handle its own mistakes.
2. **Beam Search at inference:** Maintain top-k partial hypotheses at each step instead of greedy single-token. Reduces impact of single wrong predictions by exploring multiple paths.

Section C — Long Answer Questions [50 marks]

Show complete derivations. Diagrams where appropriate.

Q17. Complete RLHF Mathematical Framework **[15]**

(a) [4 pts] Write the full RLHF objective. Derive what happens at the optimal policy π^*.

(b) [5 pts] Given the PPO-Clip objective, prove that the gradient is zero when $A_t < 0$ and $\rho_t < 1 - \epsilon$.

(c) [6 pts] Reward model scores: pairs (r_w, r_l) = (3.0, 1.0), (1.5, 1.5), (0.5, 2.5). Compute per-pair Bradley-Terry loss, identify which pair the model fails on, and compute total loss.

(a) RLHF Objective and Optimal Policy:

$$J(\theta) = \mathbb{E}_{x \sim D,\, y \sim \pi_\theta}\left[r(x,y) - \beta \log \frac{\pi_\theta(y|x)}{\pi_{ref}(y|x)}\right]$$

At optimum ($\nabla_\theta J = 0$):

Taking functional derivative and setting to zero:

$$\frac{\partial}{\partial \pi_\theta(y|x)}\left[r(x,y)\pi_\theta - \beta \pi_\theta \log \frac{\pi_\theta}{\pi_{ref}}\right] = 0$$

$$r(x,y) - \beta \log \frac{\pi_\theta(y|x)}{\pi_{ref}(y|x)} - \beta = 0$$

$$\log \frac{\pi^*(y|x)}{\pi_{ref}(y|x)} = \frac{r(x,y)}{\beta} - 1$$

$$\boxed{\pi^*(y|x) = \frac{1}{Z(x)} \pi_{ref}(y|x) \exp\left(\frac{r(x,y)}{\beta}\right)}$$

The optimal policy is the reference policy reweighted by exponentiated reward. Higher β → closer to reference. This is the insight that DPO exploits.

(b) Proof: Gradient = 0 when $A_t < 0$ and $\rho_t < 1 - \epsilon$

PPO objective for a single sample:

$$L = \min(\rho_t A_t,\ \text{clip}(\rho_t, 1{-}\epsilon, 1{+}\epsilon) A_t)$$

When $A_t < 0$ and $\rho_t < 1 - \epsilon$:

- Unclipped term: $\rho_t A_t$ (negative, and since $\rho_t < 1 - \epsilon$, it's more negative than the clip)
- Clipped term: $\text{clip}(\rho_t, 1{-}\epsilon, 1{+}\epsilon) = 1 - \epsilon$ (since ρ_t is below the lower bound)
- Clipped objective: $(1 - \epsilon) A_t$ (also negative, but LESS negative since $1 - \epsilon > \rho_t$)

Since $A_t < 0$ and $\rho_t < 1 - \epsilon$:

$$\rho_t A_t < (1 - \epsilon) A_t \quad \text{(more negative < less negative)}$$

$\min$ selects the SMALLER (more negative) = $\rho_t A_t$...

Wait — re-examine: For PPO maximization, we take $\min$ to be PESSIMISTIC. When $A_t < 0$:

- $\rho_t A_t$ → as ρ_t decreases, this becomes MORE negative (larger magnitude penalty)
- $(1 - \epsilon) A_t$ → fixed cap on penalty

The min selects the MORE negative value... BUT PPO maximizes! So we actually want:

$$L^{CLIP} = \min(\rho_t A_t, (1{-}\epsilon) A_t)$$

$\rho_t A_t$ (with $\rho_t < 1 - \epsilon$, $A_t < 0$): magnitude $|\rho_t A_t| < |(1 - \epsilon) A_t|$ because $\rho_t < 1 - \epsilon$ and multiplying a negative by a smaller positive gives LESS negative.

Correction: $\rho_t < 1 - \epsilon$ and $A_t < 0$: $\rho_t A_t > (1 - \epsilon) A_t$ (less negative × negative = less negative result).

$\min$ selects $(1 - \epsilon) A_t$ (the more negative = smaller value).

$(1 - \epsilon) A_t$ is a constant → $\frac{\partial}{\partial \theta}[(1 - \epsilon) A_t] = 0$ ■

Intuition: Policy has already reduced this bad action's probability sufficiently ($\rho_t < 1 - \epsilon$ means probability dropped by more than ϵ). Further suppression is blocked → prevents over-punishment.

(c) Bradley-Terry loss:

$\ell_i = -\log \sigma(r_w - r_l)$

Pair 1: $\Delta = 3.0 - 1.0 = 2.0$

$\sigma(2.0) = 1/(1 + e^{-2}) = 0.881$ → $\ell_1 = -\log(0.881) = \mathbf{0.127}$ ✓ (low loss, correct ranking)

Pair 2: $\Delta = 1.5 - 1.5 = 0.0$

$\sigma(0) = 0.5$ → $\ell_2 = -\log(0.5) = \mathbf{0.693}$ ⚠️ (model can't distinguish — loss = $\log 2$)

Pair 3: $\Delta = 0.5 - 2.5 = -2.0$

$\sigma(-2.0) = 1/(1 + e^{2}) = 0.119$ → $\ell_3 = -\log(0.119) = \mathbf{2.128}$ ❌ **(MODEL FAILS — ranks rejected above preferred!)**

Total loss: $\bar{\ell} = (0.127 + 0.693 + 2.128)/3 = \mathbf{0.983}$

Analysis: Pair 3 dominates the loss (2.128 of 2.948 total). The reward model critically fails on this pair and will receive the largest gradient correction during training.

Q18. Multi-Head Attention: Parameter Analysis and Computation **[15]**

(a) [3 pts] For $d_{model} = 768$, $h = 12$: prove that multi-head and single-head have the same parameter count.

(b) [4 pts] Explain positional encoding. Why are sinusoidal functions used? What's the parameter count for learned positional embeddings with max_len=512?

(c) [8 pts] Full transformer block parameter count: $d_{model} = 768$, $h = 12$, $d_{ff} = 3072$, include LayerNorm and biases. Then compute memory in FP16.

(a) Multi-head vs Single-head parameters:

Multi-head (h=12):

Each head has: $W_Q^i \in \mathbb{R}^{768\times64}$, $W_K^i \in \mathbb{R}^{768\times64}$, $W_V^i \in \mathbb{R}^{768\times64}$

All heads concatenated: effectively $W_Q \in \mathbb{R}^{768\times768}$ (12 heads × 64 = 768 columns)

Total: $W_Q + W_K + W_V + W_O = 4 \times (768 \times 768) = \mathbf{2{,}359{,}296}$

Single-head (h=1):

$W_Q, W_K, W_V \in \mathbb{R}^{768\times768}$, $W_O \in \mathbb{R}^{768\times768}$

Total: $4 \times (768 \times 768) = \mathbf{2{,}359{,}296}$ — **identical!**

Proof: Multi-head splits dimensions internally: $12 \times (768 \times 64) = 768 \times 768$. The overall projection dimensionality is unchanged. Multi-head gives diverse attention patterns at zero extra parameter cost. ■

(b) Positional encoding:

Why needed: Self-attention is permutation-equivariant — without position info, "dog bites man" = "man bites dog".

> **Sinusoidal (original Transformer):**
>
> $$PE_{(pos,2i)} = \sin(pos/10000^{2i/d_{model}})$$
>
> $$PE_{(pos,2i+1)} = \cos(pos/10000^{2i/d_{model}})$$

Why sin/cos? (1) Can represent relative positions: PE_{pos+k} is a linear function of PE_{pos} for any fixed k. (2) No parameters to train. (3) Can extrapolate to unseen sequence lengths.

Learned PE parameters: $\text{max\_len} \times d_{model} = 512 \times 768 = \mathbf{393{,}216}$

Trade-off: learned PEs are more flexible but CANNOT extrapolate beyond max_len.

(c) Full block parameter count:

Component	Parameters
W_Q, W_K, W_V (weights)	$3 \times 768 \times 768 = 1{,}769{,}472$
W_Q, W_K, W_V (biases)	$3 \times 768 = 2{,}304$
W_O (weight + bias)	$768 \times 768 + 768 = 590{,}592$
LayerNorm 1 (γ, β)	$2 \times 768 = 1{,}536$
FFN W_1 (weight + bias)	$768 \times 3072 + 3072 = 2{,}362{,}368$
FFN W_2 (weight + bias)	$3072 \times 768 + 768 = 2{,}360{,}064$
LayerNorm 2 (γ, β)	$2 \times 768 = 1{,}536$
Total per block	**7,087,872**

Memory (FP16): $7{,}087{,}872 \times 2$ bytes = $14{,}175{,}744$ bytes $= \mathbf{13.52\ MB\ per\ layer}$

For BERT-base (12 layers): $12 \times 13.52 = 162.2$ MB for transformer blocks alone.

Q19. Autoencoder Gradient Derivation **[12]**

A 1-hidden-layer autoencoder: Input $\boldsymbol{x} \in \mathbb{R}^3$, hidden $\boldsymbol{h} \in \mathbb{R}^2$ (ReLU), reconstruction $\boldsymbol{x'} \in \mathbb{R}^3$.

$\boldsymbol{h} = \mathbf{ReLU}(\boldsymbol{W_1 x} + \boldsymbol{b_1})$, $\boldsymbol{x'} = \boldsymbol{W_2 h} + \boldsymbol{b_2}$. Loss: $\boldsymbol{J} = \frac{1}{2}\|\boldsymbol{x'} - \boldsymbol{x}\|^2$.

Given: $\boldsymbol{x} = [1, 0, 1]^T$, $\boldsymbol{W_1} = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}$, $\boldsymbol{b_1} = [0, 0]^T$, $\boldsymbol{W_2} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \end{bmatrix}$, $\boldsymbol{b_2} = [0, 0, 0]^T$.

(a) [4 pts] Compute forward pass: $\boldsymbol{h}$ and $\boldsymbol{x'}$.

(b) [4 pts] Compute $\boldsymbol{\partial J / \partial W_2}$.

(c) [4 pts] Compute $\boldsymbol{\partial J / \partial W_1}$.

(a) Forward pass:

$$z_1 = W_1 x + b_1 = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 2 \\ 0 \end{bmatrix}$$

$$h = \mathrm{ReLU}([2, 0]^T) = \mathbf{[2, 0]^T}$$

$$x' = W_2 h + b_2 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 2 \\ 0 \end{bmatrix} = \mathbf{[2, 0, 2]^T}$$

$$J = \tfrac{1}{2}\|(2-1, 0-0, 2-1)\|^2 = \tfrac{1}{2}(1+0+1) = \mathbf{1.0}$$

(b) Gradient $\partial J / \partial W_2$:

$\delta_{out} = \frac{\partial J}{\partial x'} = x' - x = [2-1, 0-0, 2-1]^T = [1, 0, 1]^T$

$$\frac{\partial J}{\partial W_2} = \delta_{out} \cdot h^T = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} [2, 0] = \begin{bmatrix} 2 & 0 \\ 0 & 0 \\ 2 & 0 \end{bmatrix}$$

(c) Gradient $\partial J / \partial W_1$:

Step 1: Backprop through W_2:

$$\delta_h = W_2^T \cdot \delta_{out} = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 2 \\ 0 \end{bmatrix}$$

Step 2: Through ReLU:

$\delta_{z_1} = \delta_h \odot \mathrm{ReLU}'(z_1) = [2, 0] \odot [1, 0] = [2, 0]$

(ReLU'(2) = 1, ReLU'(0) = 0)

Step 3: Gradient of W_1:

$$\frac{\partial J}{\partial W_1} = \delta_{z_1} \cdot x^T = \begin{bmatrix} 2 \\ 0 \end{bmatrix} [1, 0, 1] = \begin{bmatrix} 2 & 0 & 2 \\ 0 & 0 & 0 \end{bmatrix}$$

Note: Row 2 of $\partial J / \partial W_1$ is all zeros because $h_2 = 0$ (ReLU killed it) → that neuron receives no gradient update. This is the "dying ReLU" phenomenon in action.

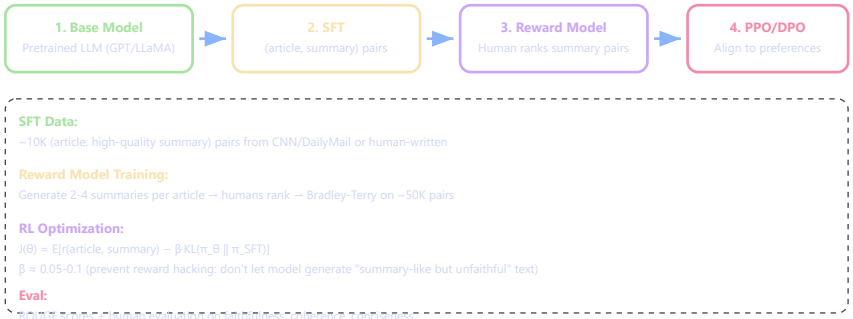
Q20. Design Question: Building an RLHF-aligned Summarization System **[8]**

(a) [3 pts] Outline the complete pipeline: from base model to aligned summarizer.

(b) [3 pts] How would you collect preference data? What biases might arise?

(c) [2 pts] Why might you choose DPO over PPO for this task?

(a) Complete pipeline:



(b) Preference data collection and biases:

Collection: Show annotators an article + 2 summaries (from SFT model with different temperatures/prompts). Ask: "Which summary is better?" Collect ~50K preference pairs.

Biases:

- **Length bias:** Annotators prefer longer summaries (more info) → model learns to be verbose
- **Fluency bias:** Fluent but inaccurate text rated higher than awkward but faithful text
- **Annotator disagreement:** Different annotators have different quality standards → noisy labels
- **Position bias:** First-shown summary may be preferred regardless of quality

Mitigation: Randomize position, use multiple annotators + majority vote, filter low-agreement pairs, add explicit instructions for faithfulness.

(c) DPO over PPO for summarization:

- **Simpler pipeline:** DPO needs only preference pairs + reference model. No RM training, no RL loop, no reward model inference during training.
- **More stable:** PPO on summarization is notoriously unstable (reward hacking: model produces summaries that "look good" to RM but are unfaithful).
- **Compute efficient:** Single training pass vs. iterative RL rollouts. ~3× fewer GPU hours.
- **When PPO is better:** If you need the reward model for other purposes (ranking, filtering) or want iterative improvement with online data collection.

Created by **Kushal Ghosh** (gkushalg01@gmail.com) and **Bharat Das Vaishnav** (bharatvaishnav2025@gmail.com)